

The University of Nottingham

Faculty of Science and Engineering



**University of
Nottingham**
UK | CHINA | MALAYSIA

Edge AI low-power water leak detection

AUTHOR	:	Cheng Jing Siang
ID	:	20409588
SUPERVISOR	:	Dr Hermawan Nugroho
MODERATOR	:	Prof Dr Haider Abbas
DATE	:	2/3/2026

Fourth year project thesis is submitted in partial fulfilment of the requirements of the degree of **Master of Engineering**

Statement of Originality

- I have read and understood the University of Nottingham [Academic Misconduct Policy](#).
- I am aware that failure to act in accordance with the Regulations Governing Academic Integrity may lead to the imposition of penalties which, for the most serious cases, may include termination of programme.
- I consent to the University copying and distributing any or all of my work in any form and using third parties to verify whether my work contains plagiarised material, and for quality assurance purposes.

You must change the statements in the boxes to reflect your individual submission.

You must acknowledge all sources of information (e.g. ideas, algorithms, data) using citations to auditable, authoritative sources.

You must also put quotation marks around any sections of text that you have copied without paraphrasing and cite the original source.

If any figures or tables have been taken or modified from another source, you must explain this in the caption and cite the original source.

I have acknowledged all sources, and identified any content taken from elsewhere.

If you have used any code (e, g. open-source code), reference designs, or similar resources that have been produced by anyone else, you must list them in the box below. In the report, you must explain what was used and how it relates to the work you have done.

I have used the following resources that have been produced by someone else:

- [CMSIS-DSP \(https://github.com/ARM-software/CMSIS-DSP\)](https://github.com/ARM-software/CMSIS-DSP) used to perform FFT and MFCC in the B-U585I-IOT02A.
- [Ai-Thinker-LoRaWAN-Ra-08 \(https://github.com/Ai-Thinker-Open/Ai-Thinker-LoRaWAN-Ra-08\)](https://github.com/Ai-Thinker-Open/Ai-Thinker-LoRaWAN-Ra-08) modified to use for LoRa P2P transmission

I have acknowledged all input (hardware, software, intellectual) taken from elsewhere and explained how this was used.

If you have used generative AI tools in the preparation of your assessed work, or the underlying code/designs/underpinning work, its use must be declared in a signed statement in the box below. You must declare the tools employed and the method and extent of use. You can do this using the table below:

Generative AI tools have been used in the preparation of this work as follows:

<i>Tools</i>	<i>Method</i>	<i>Extent of use</i>
<i>Chat GPT</i>	<i>Drafting code</i>	<i>Draft code for the flow of detection, which was produced with AI assistance, before being edited and verified by me.</i>
<i>Chat GPT</i>	<i>Code debugging</i>	<i>Debug code that has errors.</i>
<i>Chat GPT</i>	<i>Improving grammar and spelling</i>	<i>Throughout the report as a whole.</i>
<i>Chat GPT</i>	<i>Paraphrasing</i>	<i>Paraphrasing sentences in literature review section</i>

I have acknowledged, in context, the use of all generative AI tools in preparation of this assessment, including textual, referencing, hardware, software, and intellectual inputs to my submitted work. All outputs were critically reviewed, edited, and verified by me.

You can consult with teaching staff/demonstrators, but you should not show anyone else your work (this includes uploading your work to publicly accessible repositories e, g. Github, unless expressly permitted by your supervisor). Nor should you help other students with their work.

You must get permission in writing before you seek outside assistance, e.g. a proofreading service, and declare it here.

I completed all of the job on my own and didn't assist anybody else.

You must not have fabricated, modified or distorted any data, evidence, references, experimental results, or other material used or presented in the report. You must clearly describe your experiments and how the results were obtained, and include all data, source code and/or designs (submitted as a separate file) so that your results could be reproduced.

I have included all of my data, code, and designs, and the information in the report is authentic.

If your work involved research studies (including surveys) on human participants, their cells or data, or on animals, you must have been granted ethical approval before the work was carried out, and any experiments must have followed these requirements. You must give details of this in the report and list the ethical approval reference number(s) in the box below.

Not applicable

My responses to these questions are full and complete, and I have discussed them with my supervisor.	Student signature	
	Date	15/4/2026

Abstract

Water leakage in distribution systems remains a significant problem because it causes water loss, increases operating cost, and can lead to infrastructure damage if not detected early. This project presents a proof-of-concept edge AI water leak detection system that combines acoustic and vibration sensing on an embedded microcontroller platform. The system was developed using the STM32U585AI on the B-U585I-IOT02A development board, where leak-related signals were acquired using the on-board MEMS microphone and accelerometer, processed locally using NanoEdge AI Studio generated classification libraries, and transmitted wirelessly through a LoRa peer-to-peer communication link. To support model development and controlled validation, a laboratory-scale leak simulation rig and a dedicated pipe-mountable holder were designed and built. The embedded detection workflow was based on repeated 512-sample inference and a structured late-fusion decision logic, in which a 30s detection cycle was divided into ten 3s segments and the segment-level outputs were aggregated through temporal rules before producing a final verdict. This approach was introduced to improve robustness against short-term fluctuations and unstable raw predictions. Among the evaluated model and logic versions, the best-performing configuration achieved a final overall detection accuracy of 88.0%, with a recall of 1.00, F1 score of 0.91, and balanced accuracy of 0.83 under controlled rig-based validation. The project also evaluated low-power operation and showed that, although the STM32U585AI itself was capable of very low sleep current in MCU-only measurements, the complete development-board-based prototype remained dominated by platform-level current consumption, indicating that custom hardware would be needed for true long-duration battery deployment. In addition, LoRa P2P testing successfully demonstrated wireless transmission of leak-related results, while benchmarking against an alternative YAMNet-based model showed that the NanoEdge AI approach was more suitable for this resource-constrained embedded platform after deployment. Overall, this work demonstrates the feasibility of integrating multimodal sensing, embedded AI inference, temporal decision logic, low-power scheduling, wireless reporting, and mechanical prototype design into a single proof-of-concept smart leak detection system, while also identifying the key limitations that must be addressed for future field deployment.

Acknowledgement

I would like to express my sincere gratitude to my supervisor, Dr Hermawan Nugroho, for his continuous guidance, support, and constructive feedback throughout this project. His advice and technical insight have been very important in shaping the direction of this research and in helping me overcome the challenges faced during the development of the system.

I am also grateful to my moderator, Prof Dr Haider Abbas, for his feedback and thoughtful questions during the project review. His comments encouraged me to view the project from a more professional perspective, which helped improve the overall quality of the work.

In addition, I would like to acknowledge STMicroelectronics for providing the development tools, software ecosystem, and hardware platforms that supported the embedded AI development in this project. I am also thankful to the authors and providers of the research papers, technical documentation, and related resources that served as important references throughout the study.

I would also like to thank George Kent for sharing the system concept and practical industrial direction related to the project. Their input helped to strengthen the industrial relevance of this work and gave useful perspective on the intended real-world application.

Finally, I would like to thank my family and friends for their encouragement, patience, and continuous support throughout this journey. Their understanding and motivation have been a constant source of strength during the completion of this project.

Table of Contents

Abstract	i
Acknowledgement	ii
List of Tables.....	vi
List of Figures	vi
List of Abbreviations.....	ix
1. Introduction.....	1
1.1 Project Background	1
1.2 Project Overview	1
1.3 Aims and Objectives	3
1.4 Stretch goals	3
1.5 Risks Management	3
1.6 Project Schedule	4
2. Literature Review	5
2.1 Water Leakage in Distribution Systems: Problem Context and Motivation	5
2.2 Overview of Water Leak Detection Approaches.....	7
2.3 Acoustic Leak Detection Principles and Challenges.....	8
2.4 Vibration-Based Leak Detection Principles and Advantages	10
2.5 Acoustic-Vibration Multimodal Sensing for Leak Detection	11
2.6 Signal Acquisition and Experimental Design for Leak Datasets	12
2.7 Signal Preprocessing for Leak Detection.....	13
2.8 Time-domain, Frequency-domain, and Time-frequency Representations	14
2.9 Temporal buffering, segmentation, and windowing strategies	15
2.10 Machine learning and deep learning models for leak detection.....	16
2.11 Embedded AI, TinyML, and on-device deployment constraints	18
2.12 AutoML and NanoEdge AI Studio for time-series classification	19
2.13 Low-power IoT architecture for continuous monitoring	20
2.14 Wireless communication for remote leak monitoring	21
2.15 Industrial Relevance and Research Positioning.....	23
3. Methodology.....	24
3.1 Overall Architecture.....	24
3.2 Design and Hardware Integration	25
3.2.1 Microphone	25
3.2.2 Accelerometer.....	26

3.2.3	LoRa P2P Communication Module	27
3.2.4	Power Hardware	29
3.3	Simulation Rig	31
3.4	Device Holder.....	32
3.5	Edge AI Model Development	34
3.5.1	Dataset Collection and Preprocessing.....	35
3.5.2	Model Training.....	36
3.5.3	Benchmarking and Library Export	37
3.5.4	YAMNet Detection Model Training	38
3.6	Deployment on Embedded Hardware.....	39
3.7	Decision Logic (Late Fusion)	40
3.7.1	Three-Second Segment Decision Logic.....	42
3.7.2	Thirty-Second Decision Logic	44
3.7.3	Sensor Score	45
3.7.4	Late Fusion and Final Verdict	45
3.8	Testing and Validation.....	49
3.8.1	Measuring AI Detection Model Accuracy.....	50
3.8.2	Measuring Power Consumption	53
4.	Results and Discussion.....	55
4.1	AI Model Accuracy	55
4.2	AI Model Architecture	63
4.3	YAMNet Model Accuracy Comparison	65
4.3.1	Training Performance Evaluation	65
4.3.2	Model Conversion and Architecture Analysis	66
4.3.3	Embedded Accuracy Evaluation.....	67
4.4	Power Consumption	69
4.4.1	Measurement Overview and Operating States	69
4.4.2	Datasheet Comparison (STM32U585).....	72
4.4.3	Battery Lifetime Estimation	72
4.4.4	Effect of Detection Frequency on Daily Energy Consumption and Battery Lifetime	73
4.4.5	Development Board Limitation in Low-Power Operation.....	76
4.4.6	Implication for the final design	77
4.5	LoRa P2P Transmission.....	77

4.6	Device Holder.....	78
5.	Conclusion and Future Works	80
5.1	Detection System Performance.....	80
5.2	Power Consumption and Low-Power Operation.....	81
5.3	Holder Design	82
5.4	Overall Conclusion.....	83
5.5	Future Improvement.....	84
5.6	Reflection on Project Management.....	85
6.	References	86
	Appendices	97

List of Tables

Table 1.5.1: Table of risks and mitigation.	4
Table 2.2.1: Comparison table of different methods for water leaks detection.	7
Table 3.5.1: Example candidate processing and model options explored during benchmarking in NanoEdge AI Studio.	37
Table 3.7.1: Summary of the thresholds used in the implementation of decision logic.	46
Table 4.1.1: Dataset size of different model version (512 samples per buffer).	55
Table 4.1.2: Type of leak/background recorded in the dataset of each model version.	56
Table 4.1.3: Performance matrices for detection model v2 - DL v2.	62
Table 4.4.1: Entire-board current consumption (measured)	69
Table 4.4.2: MCU-only sleep current drawn by different low-power modes (measured).	69
Table 4.4.3: Estimated maximum duration for each state of the device per day.	72
Table 4.4.4: Current Drawn by MCU at Different Power Mode from Datasheet [15].	72
Table 5.4.1: Summary of the objective and stretch goal status.	83

List of Figures

Figure 1.2.1: System diagram	2
Figure 1.6.1: Ideal project Gantt chart.	5
Figure 2.1.1: International Standard Water Balance and Terminology [20].	6
Figure 2.3.1: Illustration of two-sensor correlation-based localization [33].	9
Figure 2.5.1: Fusion strategies using deep learning [46].	11
Figure 3.1.1: Overview of the hardware and software used.	24
Figure 3.2.1: Proposed system architecture for the low-power leak detection device.	25
Figure 3.2.2: MP23DB01HP [92].	25
Figure 3.2.3: ISM330DHCX [95].	26
Figure 3.2.4: RA-08H-Kit [98].	27
Figure 3.2.5: Antenna used for LoRa modules: node (left), base (right).	28
Figure 3.2.6: Relationship between antenna gain and effective signal coverage [102] [103].	29
Figure 3.2.7: Device circuit diagram.	30
Figure 3.3.1: Simulation Rig setup.	31
Figure 3.3.2: 3D printed pipe section with different sizes of leak hole.	31
Figure 3.4.1: 3D CAD model of the prototype device holder developed in SolidWorks.	33
Figure 3.4.2: Exploded view and bill of materials of the prototype holder assembly developed in SolidWorks.	33

Figure 3.5.1: Samples independently (Left) versus Samples as temporal buffers (Right) [76].	35
Figure 3.5.2: Screenshot of the data logger UI.	36
Figure 3.5.3: Example of ML pipeline used in benchmarking in NEAI Studio.	37
Figure 3.7.1: Segmentation of a 30s detection recording.	41
Figure 3.7.2: Graphical interpretation of the temporal hysteresis detection logic.	43
Figure 3.7.3: Graphical interpretation of the strong fraction detection logic.	43
Figure 3.7.4: Per-segment detection summary output.	44
Figure 3.7.5: 30s late fusion detection summary output.	46
Figure 3.7.6: The device periodically detection flow chart with V2 decision logic implemented.	49
Figure 3.8.1: AI model validating on simulation rig.	50
Figure 3.8.2: Example LoRa P2P payload format for AI detection result extraction.	51
Figure 3.8.3: Setup for MCU current measurement.	53
Figure 3.8.4: Power-tree diagram of the B-U585I-IOT02A showing JP3 in the MCU supply path [117].	54
Figure 3.8.5: Setup for full board current measurement.	54
Figure 4.1.1: Vibration signal buffers uploaded to NEAI Studio for baseline and leak conditions, shown in time-series, FFT, and browser-plot views.	57
Figure 4.1.2: Acoustic signal buffers uploaded to NanoEdge AI Studio for baseline and leak conditions, shown in time-series, FFT, and browser-plot views.	58
Figure 4.1.3: Detection accuracy breakdown by modality and decision stage.	59
Figure 4.1.4: Overall detection accuracy trend across model and logic versions.	60
Figure 4.1.5: Confusion matrices for detection model v2 - DL v2.	61
Figure 4.2.1: Performance matrices extracted from the acoustic (left) and vibration (right) library report.	63
Figure 4.2.2: Algorithm flowchart extracted from the acoustic (top) and vibration (bottom) library report.	64
Figure 4.3.1: Training accuracy for YAMNet acoustic and vibration model versus number of epochs used for training.	65
Figure 4.3.2: Confusion matrix for YAMNet acoustic and vibration best model.	65
Figure 4.3.3: Architecture of the YAMNet acoustic model before and after the optimization in STM32Cube AI Studio.	66
Figure 4.3.4: Architecture of the YAMNet vibration model before and after the optimization in STM32Cube AI Studio.	67
Figure 4.3.5: Comparison of detection accuracy between converted and original YAMNet models.	68
Figure 4.3.6: Comparison of detection accuracy between NEAI and YAMNet models.	68

Figure 4.4.1: Comparison of Sleep Current (Full Board vs MCU-only).	70
Figure 4.4.2: Comparison of Total Daily Energy Consumption (Full Board vs MCU-only)..	71
Figure 4.4.3: Estimated lifetime based on ER34614 capacity.	73
Figure 4.4.4: Total daily energy consumption of the full board versus number of detections per day.....	74
Figure 4.4.5: Minimum lifetime of the full board versus the number of detections per day...74	
Figure 4.4.6: Total daily energy consumption of the MCU-only configurations versus number of detections per day.	75
Figure 4.4.7: Minimum lifetime of the MCU-only configurations versus the number of detections per day.	75
Figure 4.5.1: Example payload received by the base LoRa.....	77
Figure 4.5.2: LoRa P2P communication range test under obstructed conditions.....	78
Figure 4.6.1: Inner view of the prototype device.....	79
Figure 4.6.2: Outer view and mounting mechanism of the holder.	79
Figure 4.6.3: Components that are accessible from outside.	80

List of Abbreviations

AI	Artificial Intelligence
AutoML	Automated Machine Learning
CAD	Computer-Aided Design
COM Port	Communication Port
CSV	Comma Separated Values
DL	Deep Learning
GAN	Generative Adversarial Network
IMU	Inertial Measurement Unit
iNEMO	Inertial Nano Electromechanical Oscillator
LoRa P2P	Long Range Peer-to-Peer
LSTM	Long Short-Term Memory
MCU	Microcontroller Unit
MEMS	Micro-Electro-Mechanical Systems
MFCC	Mel-Frequency Cepstral Coefficients
ML	Machine Learning
MLP	Multilayer Perceptron
NEAI	NanoEdge AI Studio
NRW	Non-Revenue Water
PCB	Printed Circuit Board
P2P	Peer-to-Peer
RF	Random Forest
RTC	Real-Time Clock
SDG	Sustainable Development Goal
STFT	Short-Time Fourier Transform
SVM	Support Vector Machine
TinyML	Tiny Machine Learning
UART	Universal Asynchronous Receiver-Transmitter
UI	User Interface
XGB	eXtreme Gradient Boosting
YAMNet	Yet Another Mobile Network

1. Introduction

1.1 Project Background

Water leakage in distribution systems is a major technical and operational problem because it wastes treated water, increases operating costs, and reduces the overall efficiency and sustainability of water utilities [1] [2]. Hidden leaks are particularly problematic because they may persist for long periods before becoming visible, allowing water loss, infrastructure deterioration, and repair cost to accumulate over time [2] [3]. As a result, there is strong practical interest in monitoring approaches that can identify leakage earlier and more reliably than conventional inspection-based methods [3] [4].

Recent developments in the Internet of Things (IoT), embedded sensing, and artificial intelligence (AI) have created new opportunities for automated leak monitoring [5] [6]. AI-based leak detection systems can learn to distinguish leak-related patterns from background conditions using measured signals such as sound, vibration, or pressure, thereby reducing dependence on purely manual interpretation [7] [6]. Such approaches are attractive because they offer the possibility of continuous monitoring, faster response, and more scalable deployment across distributed infrastructure [5] [6].

At the same time, practical deployment in water infrastructure introduces constraints beyond detection accuracy alone. Monitoring points may be geographically dispersed, difficult to access, and located where wired power and communication are not readily available [8] [9]. Under these conditions, a useful sensing node should not only detect leaks, but should also operate with low energy consumption, perform robust local processing, and communicate results efficiently [8] [9]. This makes low-power edge AI particularly relevant, since local inference can reduce latency and lower the communication burden compared with cloud-dependent architectures [8].

Among the available sensing approaches, acoustic and vibration measurements are especially relevant because water leakage produces both airborne or structure-borne sound and mechanical excitation of the pipe structure [10]. These two sensing modalities therefore provide different but related views of the same physical event, and their combination has the potential to improve detection robustness compared with relying on only one sensing channel [10] [11]. Motivated by this context, the present project investigates a low-power embedded leak detection system that combines acoustic sensing, vibration sensing, on-device AI inference, and wireless result reporting within a single prototype architecture [8] [4] [11].

Beyond its technical motivation, improving the energy efficiency of distributed leak-monitoring systems also supports more sustainable water management, which is consistent with Sustainable Development Goal 6, aimed at ensuring the availability and sustainable management of water and sanitation for all [12].

1.2 Project Overview

This project develops a proof-of-concept low-power edge AI system for water leak detection using acoustic and vibration sensing on a microcontroller-based platform [4] [13]. The system is designed around the STM32U585AI on the B-U585I-IOT02A development board, selected for its embedded processing capability, integrated peripherals, and support for low-power operation [14] [15]. Although STM32L562QE on the STM32L562E-DK development board was initially considered during early project planning, the present prototype was ultimately implemented on the STM32U585AI platform. The device performs sensing and inference

locally, with the aim of reducing dependence on continuous remote processing and limiting unnecessary wireless transmission [8] [16]. Figure 1.2.1 illustrates the overall concept of the proposed leak detection system, showing how acoustic and vibration signals are acquired by the embedded node, processed locally using on-device AI, and then transmitted as compact leak-related results through wireless communication.

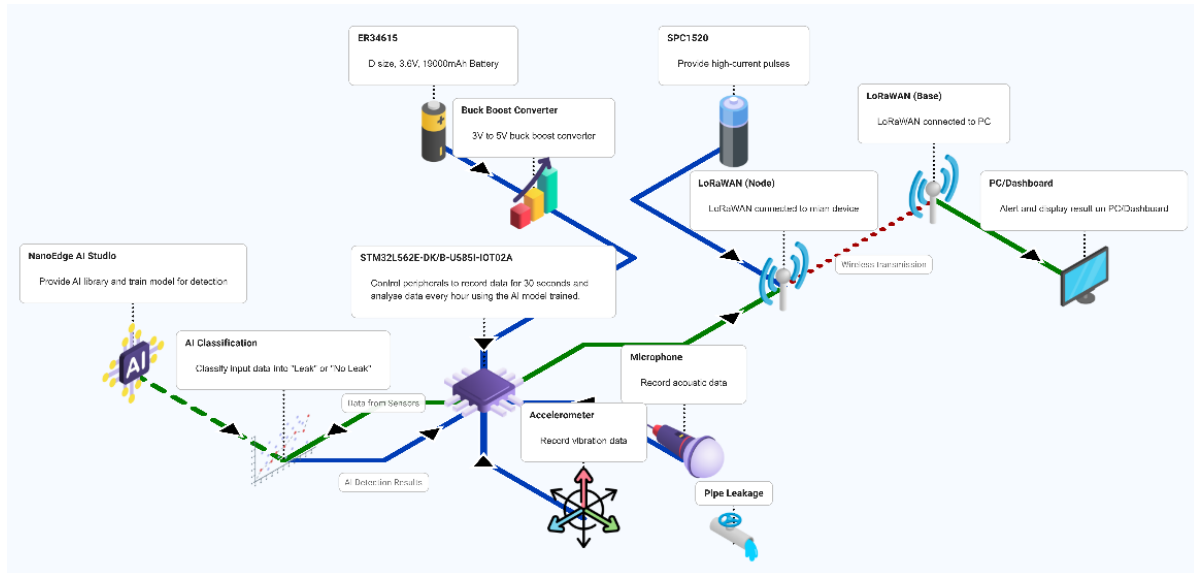


Figure 1.2.1: System diagram

As illustrated in Figure 1.2.1, two sensing modalities are used in the proposed system. Acoustic information is captured using the on-board MEMS microphone, while pipe vibration is measured using the on-board accelerometer [13] [11]. These signals are processed separately using embedded classification models generated through NanoEdge AI Studio, allowing each sensing channel to contribute its own leak-related evidence while remaining compatible with microcontroller deployment constraints [13] [17]. The outputs are then combined using a structured late-fusion decision process rather than relying on a single short inference result alone.

To support model development and controlled evaluation, a laboratory-scale leak simulation rig was constructed to generate repeatable leak and non-leak conditions under different leak sizes and mounting distances [4]. This rig provides the dataset used for model development and also serves as the validation platform for the final embedded prototype. In the deployed workflow, the system records a 30s detection interval, divides it into shorter 3s segments, performs repeated short-buffer inferences, and then aggregates the results through a multi-stage temporal decision logic before issuing a final verdict. This architecture was selected to improve robustness against transient fluctuations and short-term misclassifications during real embedded operation.

The project also investigates two supporting aspects of practical deployment. First, wireless result transmission is implemented using a LoRa peer-to-peer arrangement to demonstrate remote reporting of leak-related outputs without requiring a permanent wired connection [18]. Second, the system's low-power behavior is evaluated through both MCU-level and full-board current measurements in order to examine how effectively the periodic wake-sleep concept can be implemented in the present prototype [15]. In addition, an alternative YAMNet-based model is developed and benchmarked as part of a comparative study to assess whether a more conventional deep-learning approach remains competitive after embedded conversion and deployment.

Overall, the project is intended not as a final field-ready product, but as an integrated proof-of-concept prototype that brings together multimodal sensing, embedded AI, temporal decision logic, low-power scheduling, wireless reporting, and mechanical integration in a single leak-monitoring system.

1.3 Aims and Objectives

The primary goal of this project is to create a reliable low-power edge AI system for pipe leak detection. The system will utilize an embedded microcontroller platform, combined with onboard and optional external sensors, to reliably detect vibration and acoustic signals caused by leaks, thereby enabling early warning of leaks in water distribution networks.

To achieve this goal, the project has set the following specific objectives:

- I. Define and set up a controlled experimental environment, including a leak simulation rig, to collect acoustic and vibration data under various leak conditions and controlled variables.
- II. Develop an AI-based leak detection algorithm utilizing both acoustic and vibration signal analysis.
- III. Ensure low power operation.
- IV. Design and assemble a fully functional prototype for development on a controlled test rig.
- V. Evaluate the complete system implementation by validating the data collection and analysis intervals and verifying that power consumption and leak detection performance meet the defined project goal.

1.4 Stretch goals

If time permits, this project aims to incorporate additional features that enhance the device's usability, user interface, and overall professionalism by utilizing the available onboard peripherals. These enhancements are designed to improve convenience and user interaction with the system. The proposed stretch goals include:

- I. LEDs Indicator: Provide visual status feedback by using different LED activation patterns (i.e., specific combinations of LEDs turned on) to represent the device's current operating mode, such as sleep, idle, detection, or dataset collection.
- II. Wireless Data Transmission: Utilize the onboard ARDUINO® Uno V3 expansion interface to send data (e.g., detection results and timestamps) via LoRa peer-to-peer (P2P) communication.
- III. Wireless Connectivity: Enable Bluetooth communication to either transmit alerts when a leak is detected or allow users to retrieve system logs remotely via a smartphone or PC.
- IV. Leak Localization: In scenarios where multiple devices are deployed, explore strategies to estimate the leak's location using signal strength, detection time, or other triangulation techniques, and include this information in the user alert.
- V. AI Model Performance Comparison: Benchmark the device's deployed AI model against alternative models (e.g., YAMNet) to evaluate and compare detection performance.

1.5 Risks Management

Effective risk management is essential for the successful development and implementation of the proposed leak detection system. Table 1.5.1 lists the key operational and technical risks identified during the planning phase, along with their likelihood of occurrence, potential

consequences, and mitigation methods. These risks include software, hardware, and integration issues that could impact the project schedule or performance outcomes. Throughout the project lifecycle, proactive planning, component testing, and phased verification will be employed to minimize disruptions.

Table 1.5.1: Table of risks and mitigation.

Risk Description	Stages	Likelihood	Severity	Risk (Pre)	Risk (Post)	Mitigation
Hardware failure (e.g. MCU, sensors)	Development	2	2	4	2	Proper maintenance of hardware; perform preliminary hardware checks. Use backup board (B-U585I-IOT02A).
Late delivery of components	Development	3	2	6	3	Use backup components such as backup board (B-U585I-IOT02A)
Software bugs or firmware issues	Development	3	2	6	4	Use modular code structure, version control and iterative testing.
Inaccurate AI model performance	Testing	2	2	4	2	Expand dataset diversity, improve feature extraction, and validate using real test cases.
Excessive power consumption	Testing	2	1	3	1	Optimize the firmware during development to accommodate low power modes and profile consumption.
Back pain, RSI, eye strain	Development	3	1	3	2	Take frequent break to prevent long use of computer.
Dataset quality/inconsistency	Development	3	1	3	1	Preprocess and label data thoroughly; Combine public and locally acquired datasets.
Board/sensor mounting instability	Testing	2	1	2	1	Use rigid 3D-printed housing; test mounting under various conditions

As shown in Table 1.5.1, likelihood indicates how likely the risk will happen in a scale of 1 (Inconceivable) to 5 (Most likely). Severity is the measure of the negative impact of the risk in a scale of 1 (Negligible) to 5 (Catastrophic). Risk is the product of likelihood and severity. The risks for the project are calculated for before (Pre) and after (Post) the risk mitigation. The risks are categorized into low (1-4), medium (5-12), or high (15-25) [19].

1.6 Project Schedule

The project timeline spans from September 25, 2025, to April 29, 2026, ending with the final project presentation. The overall schedule is illustrated in Figure 1.6.1 while a detailed version of the Gantt chart is provided in Appendix C for clearer visibility.

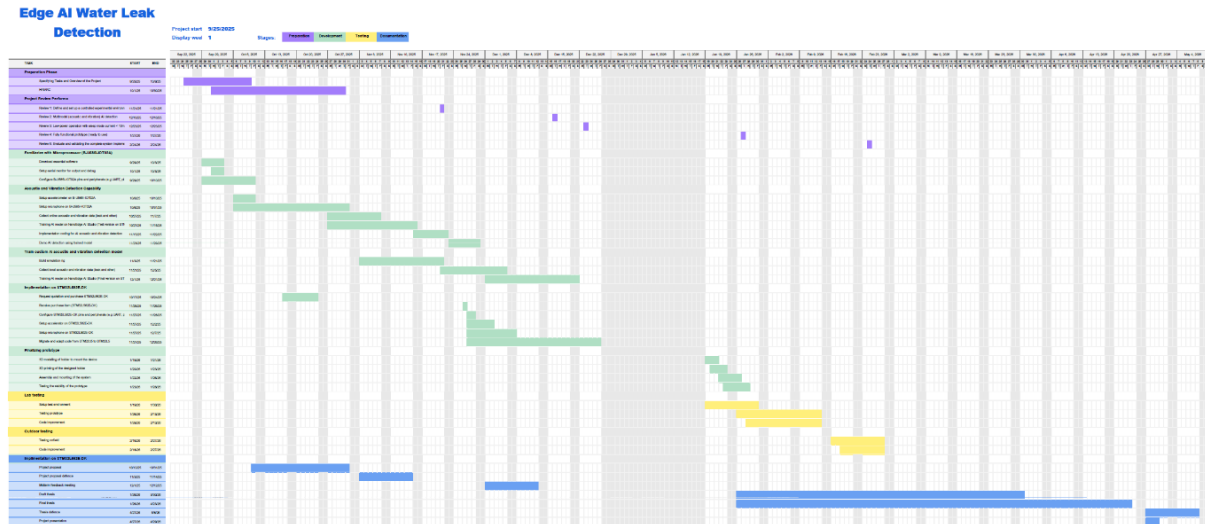


Figure 1.6.1: Ideal project Gantt chart.

As shown in Figure 1.6.1, the estimated time spent for development and testing is around 5 to 6 months. However, the actual project progress only partially followed the ideal timeline. The early preparation stage generally progressed as planned, including initial board familiarization, proposal development, early peripheral setup, and initial AI model testing. However, the timeline later became more iterative due to several technical and logistical factors. The hardware path changed multiple times, first from the B-U585I-IOT02A to the STM32H747I-DISCO and then back again because of unresolved peripheral issues, while the originally intended migration to the STM32L562E-DK as required by the company was delayed by delivery constraints and was not completed within the planned period.

As a result, several tasks such as finalizing the decision logic, expanding the dataset, validating low-power operation, and evaluating embedded accuracy extended into the later stages of the project. In addition, the work shifted away from the originally planned outdoor testing phase and instead focused on controlled rig-based validation, LoRa P2P testing, power-consumption analysis, and thesis completion. Thesis writing also became more concentrated in the later months of the project rather than progressing evenly alongside development. Overall, although the actual timeline differed from the original plan, the main project milestones were still achieved, with the final workflow becoming more iterative and development-driven than initially anticipated.

2. Literature Review

2.1 Water Leakage in Distribution Systems: Problem Context and Motivation

Water leakage in distribution systems is a major technical and operational problem in modern water supply networks. It reduces the amount of treated water that reaches consumers, increases production and pumping costs, and weakens the sustainability of water utilities. In the water sector, this issue is commonly discussed under the concept of non-revenue water (NRW), which refers to water that has been supplied into the system but does not generate income. According to the International Water Association, NRW consists of unbilled authorized consumption, apparent losses, and real losses, while real losses are the physical losses caused by leakage from mains, storage facilities, and service connections [20]. The World Bank also highlighted that high NRW reflects not only physical leakage, but also wider utility

inefficiencies, making leakage reduction both a technical and management challenge [21]. In this sense, water leakage should not be viewed as an isolated pipe defect, but as part of a broader system-performance problem affecting finance, service quality, and water security.

The scale of the problem is substantial. The World Bank estimated that more than 32 billion m^3 of treated water is lost globally each year through leakage in urban systems and argued that reducing these losses could recover major financial value and potentially extend service to millions of people without new production capacity [21]. This shows that leakage is not only an engineering concern, but also a resource-efficiency issue. The Asian Development Bank similarly frames NRW as a persistent burden for utilities, especially in developing regions where infrastructure deterioration, weak asset management, and limited institutional capacity make leakage harder to control [22]. The U.S. Environmental Protection Agency further notes that real losses represent water that has already been abstracted, treated, and pressurized before being lost from the network, meaning utilities lose both water and the energy invested in delivering it [23]. In many regions, NRW can account for approximately 30-50% of the water entering the distribution system, reflecting substantial inefficiencies in water management systems [22] [21] [24] [25].

A key reason leakage is difficult to manage is that many leaks remain hidden for long periods. Some leaks are visible at the ground surface or in accessible facilities, but many occur underground or in locations where the signs are subtle. The EPA explains that, without an active leakage control program, hidden leaks may only be noticed when they finally become visible or are reported by customers, and that the average lifetime of a slowly developing leak can be up to two years [23]. This delay increases cumulative water loss and repair cost. For this reason, leakage management depends not only on repairing known failures, but also on early detection of hidden leaks before they become severe. Beyond water loss, leakage can lead to long-term structural damage to pipelines, soil erosion, and potential contamination of treated water supplies [26] [27] [28]. From an operational perspective, this makes detection latency highly important, because the longer a leak remains undetected, the greater the technical, financial, and environmental impact [23] [29].

The literature also shows that leak management is usually a multi-stage process rather than a single action. At network level, utilities first need to understand where losses are occurring through water auditing and water balance analysis. The IWA standard water balance provides a structured way to separate NRW into its main components and gives more meaningful terminology than older concepts such as unaccounted-for water as shown in Figure 2.1.1 [20]. AWWA similarly promotes the IWA/AWWA water audit method as a best-practice framework for utilities to manage water loss using standard definitions and performance indicators [30]. The EPA describes this process in terms of top-down audits, which begin from existing records and estimates, and bottom-up audits, which involve more detailed investigation of discrete system components and field conditions [23]. This shows that leakage control is not based on sensing alone, but it is closely tied to audit quality, data interpretation, and targeted field intervention.

System Input Volume	Authorized Consumption	Billed Authorized consumption	Billed Metered Consumption	Revenue water
			Billed Unmetered Consumption	
	Water Losses	Unbilled Authorized Consumption	Unbilled Metered Consumption	Non Revenue Water (NRW)
			Unbilled Unmetered Consumption	
	Real Losses	Apparent Losses	Unauthorized Consumption	
			Metering Inaccuracies	
			Leakage on Transmission and/or Distribution Mains	
			Leakage and Overflows at Utility's Storage Tanks	
			Leakage on Service Connections up to point of Customer Metering	

Figure 2.1.1: International Standard Water Balance and Terminology [20].

From a physical perspective, leaks can produce measurable hydraulic and vibroacoustic effects, which provide the basis for modern leak detection methods. However, the detailed classification of these approaches, together with their sensing principles and limitations, is discussed in the following sections.

Overall, the literature consistently shows that water leakage in distribution systems is a serious and persistent issue that affects utility economics, infrastructure condition, and sustainable water management. Standard water-audit frameworks help define and quantify the problem, while modern sensing and data-driven methods aim to reduce the delay between leak occurrence and repair. These findings provide a strong justification for continued research into distributed, automated, and practical leak detection systems for water networks.

2.2 Overview of Water Leak Detection Approaches

Water leak detection methods can be grouped in several ways, but most review papers organize them into three broad classes, which are software- or model-based approaches, hardware sensing approaches, and conventional field inspection methods [31]. This classification is useful because it separates methods that infer leakage from network behavior, such as pressure or flow changes, from methods that directly sense physical leak signatures, such as sound or vibration, and from manual survey practices used by utilities in the field. El-Zahab and Zayed (2019) and Joseph et al. (2023) both describe leak detection as a broad multi-method area rather than a single technology, while Mohd Ismail et al. (2019) and Hu et al. (2021) show that acoustic and vibration techniques occupy an important position when the aim is non-invasive leak detection in pressurized pipelines [29] [31] [32] [33]. In practical terms, this means that leak detection should be understood as a toolbox of methods, where the appropriate choice depends on the monitoring goal, pipe material, surrounding environment, and deployment constraints.

Table 2.2.1 summarizes the main leak detection approaches reported in the literature, each of which serves a different purpose. Some are mainly used for system-level assessment, such as water balance and audit methods, which quantify losses but do not directly detect individual leak events. Others are aimed at detection, where the goal is to determine whether a leak is present. A smaller group is used for localization, where the objective is to estimate where the leak is located. This distinction is important because many studies report strong leak detection performance, but locating the leak accurately is usually harder. Hu et al. (2021) note that acoustic localization often depends on time-delay estimation and cross-correlation, while its success is strongly affected by wave speed estimation, dispersion, and attenuation in the pipe system [33]. For this reason, many embedded studies prioritize reliable leak detection before addressing full localization.

Table 2.2.1: Comparison table of different methods for water leaks detection.

Method	Sensing Modality	Typical Reported Performance	Deployment Complexity	Cost	Robustness to Noise	References
Water balance/audit	Production and consumption metering	Not usually reported as per-event accuracy; mainly used for NRW accounting	Medium	Medium	Not applicable for event detection	[20] [30]
Pressure/flow model-based methods	Pressure and flow sensors with hydraulic/statistical models	Strongly context dependent; often reported by detection rate or localization error	Medium–High	Medium	Medium, because normal demand variation can mask leaks	[29] [31]

Acoustic noise loggers + ML	Contact acoustic sensing, often at hydrants or valves	High performance reported in some real-network studies after denoising and feature extraction	Medium	Medium–High	Moderate; improved by denoising and ML	[34]
Acoustic correlation	Two acoustic or vibroacoustic sensors	Often judged by localization error rather than classification accuracy	Medium	High	Lower in noisy conditions and problematic in plastic pipes	[35] [33]
Vibration sensors + ML	Pipe-wall vibration using MEMS or piezo sensors	Around 95% on non-metal and up to 100% on metal pipes in one real-network study; very high values also reported in some controlled settings	Medium	Low–Medium	Generally stronger than airborne acoustics, but still affected by structural and operational vibration	[36] [37] [32]
Inline inspection	In-pipe acoustic/pressure devices	Often reported by detection success and localization capability rather than simple accuracy	High	High	High, because sensing is inside the pipe	[29] [31]
Non-acoustic NDT	Ground-penetrating radar, thermography, tracer gas and similar methods	Scenario dependent	Medium–High	Medium–High	Variable; less affected by acoustic noise but sensitive to soil, burial depth, and surface conditions	[29] [32]
Fiber optic sensing	Distributed strain, acoustic, or temperature sensing	High sensitivity, but usually limited by infrastructure and cost	Very High	Very High	High	[29]

As shown in Table 2.2.1, different leak detection approaches offer different strengths depending on whether the objective is utility-level water loss assessment, event detection, or leak localization. Audit and water-balance methods are important for long-term management, but they are not designed for direct event-level sensing. Model-based hydraulic approaches can support continuous monitoring, although they may be affected by demand variation and system uncertainty. In contrast, sensing-based methods respond more directly to the physical effects of leakage, making them especially relevant for event detection. Among these, acoustic and vibration approaches are particularly suitable for non-invasive monitoring and are therefore examined in more detail in the following subsections.

2.3 Acoustic Leak Detection Principles and Challenges

Among the sensing-based methods introduced in Section 2.2, acoustic leak detection is one of the most widely used non-invasive approaches. It is based on the fact that pressurized water escaping through a defect produces turbulence, jetting, and interaction with the pipe wall, which together generate broadband vibroacoustic energy [38] [33]. This leak-induced energy can travel through the water inside the pipe, the pipe wall itself, and in buried systems also into the surrounding soil, which is why the measured signal is not a direct leak sound but a filtered version shaped by the full pipe-fluid-ground system [39]. Hunaidi and Chu’s study on plastic water pipes showed that the acoustic characteristics of leak signals vary with leak type, flow rate, pipe pressure, and other test conditions, while later review work by Hu et al. confirmed that this same physical energy is the basis for both acoustic leak detection and acoustic leak localization in pressurized pipelines [38] [33].

In practice, acoustic leak detection is commonly implemented using listening surveys, noise loggers, and correlator-based systems. Listening devices and noise loggers are mainly used to detect whether abnormal leak noise is present, while correlators are more closely associated with localization. In a typical correlator setup, two sensors are placed on accessible contact points on either side of a suspected leak, and the time delay between the measured signals is estimated through cross-correlation as shown in Figure 2.3.1 [33].

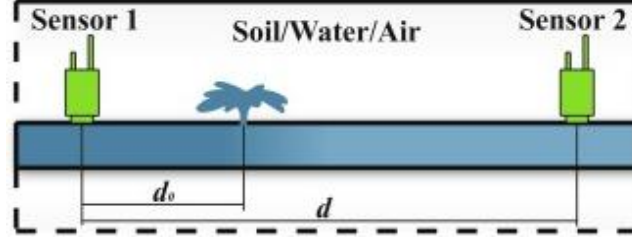


Figure 2.3.1: Illustration of two-sensor correlation-based localization [33].

This time delay, together with an estimate of wave speed, is then used to calculate the leak position using the equation:

$$d_0 = \frac{d - cT_0}{2} \quad (\text{Eq 2.3.1}) [33]$$

Here, d_0 represents the leak location, c denotes the propagation speed of the leak-induced sound, d is the total distance between the sensors, and T_0 corresponds to the time delay, defined as the difference in arrival time of the leak-generated signals at the sensors [33].

However, the signal measured by an acoustic system depends strongly on the pipe and installation conditions. Pipe material is especially important. Plastic pipes are widely recognized as more difficult than metallic pipes for acoustic leak detection because leak noise in plastic pipes is more strongly attenuated and filtered during propagation. Hunaidi and Chu showed that plastic-pipe leak signals do not have a single fixed frequency band and that their behavior changes with test conditions [38]. Gao et al. also showed that sensor choice matters greatly in plastic pipes, because the usefulness of pressure, velocity, and acceleration measurements depends on both signal strength and background noise [35]. More recently, Scussel et al. reported that the frequency range of measured leak noise in buried plastic pipes depends significantly on pipe type, installation location, and sensor type, and that operators often still rely on practical experience because reliable prediction of the usable band remains difficult [39].

These findings help explain why acoustic sensing is often combined with signal processing and machine learning in practical leak-detection systems. In real water distribution networks, the leak signal may be mixed with traffic noise, pumps, consumer usage, and other irregular disturbances. Under these conditions, simple threshold-based detection can become unreliable. Recent studies also show that acoustic ML methods still face structural limitations. One major issue is data scarcity, because labeled real leak events are hard to collect in sufficient quantity and diversity. Liu et al. addressed this by proposing an LSTM-GAN framework that generated synthetic acoustic leak signals to augment real datasets collected from water distribution networks [40]. Their results showed that synthetic data could improve model performance, which directly supports the view that limited data diversity remains an important bottleneck in acoustic leak detection [40]. Another major issue is domain shift between laboratory and field conditions. Liu, Zayed, and Xiao noted in their contrastive-learning study that machine-learning-based acoustic leak detection is still challenged by environmental variability and

limited labeled data, while background interference can reduce reliability and contribute to false alarms if the learned representation is not robust enough [41].

For this reason, acoustic leak detection should be seen as both powerful and vulnerable. It is powerful because it is non-invasive, practical, and well established for both detection and localization. It is vulnerable because its effectiveness depends heavily on propagation conditions, pipe material, sensor placement, and ambient noise.

2.4 Vibration-Based Leak Detection Principles and Advantages

Vibration-based leak detection focuses on the structural response of the pipe rather than the airborne sound around it. When pressurized water escapes through a defect, the resulting turbulence and pressure fluctuations apply dynamic forces to the pipe wall, causing it to vibrate [32]. These vibrations can then propagate along the pipe and be measured by an accelerometer mounted on the pipe or on nearby fittings. This makes vibration sensing a useful leak-detection channel because it captures leak-induced structural excitation directly, rather than relying only on external sound that may be masked by the surrounding environment. Reviews of accelerometer-based leak detection describe this as one of the main reasons vibration sensing has become increasingly important in water-pipeline monitoring [32].

A practical advantage of vibration sensing is that it can remain informative in conditions where acoustic sensing becomes difficult. In particular, Ismail et al. highlighted the growing role of accelerometer-based methods in water pipeline monitoring, especially for non-invasive leak detection in plastic pipes where conventional acoustic methods may be less effective [32]. Field-oriented work by Tariq et al. also showed that low-cost MEMS accelerometers can be used for long-duration monitoring in real water distribution networks, supporting the practical relevance of this sensing approach [36]. In that study, separate machine-learning models were developed for metal and non-metal pipes, and the reported performance differed by material class, reinforcing that vibration-based leak detection is effective but still influenced by pipe properties and deployment conditions [36].

Unlike the acoustic approach, the main concern in vibration-based detection is not airborne sound propagation, but how well the sensor is mechanically coupled to the pipe structure. Accelerometers measure motion along specific axes, so the selected sensing direction matters. Tariq et al. reported that preliminary investigations in their field deployment led them to use the radial direction because this direction successfully captured leak-related vibration under their conditions, whereas other directions were less effective [36]. Their study also showed that practical placement was often on accessible components such as valves rather than arbitrary pipe-wall locations, which reflects a common field constraint in infrastructure monitoring [36].

Another important strength of vibration-based detection is that vibration signals can be readily integrated into machine-learning-based classification pipelines. Lee et al. analyzed vibration data from leak detection sensors installed at water meter boxes and water outlets and reported that XGBoost was the most suitable model in their study, achieving a very high reported accuracy of 99.79% under their evaluated conditions [37]. While such values should be interpreted carefully because performance depends on the specific dataset and test environment, the study still shows that vibration signals can provide highly discriminative features when appropriate preprocessing and model selection are used [37]. This is consistent with the broader review by Ismail et al., which notes that both time-domain and frequency-domain analysis are commonly used and that feature selection should reflect the physical behavior of the sensed vibration [32].

Overall, vibration-based leak detection is attractive because it measures leak-induced structural behavior directly, can be practical with low-cost MEMS accelerometers, and may remain

effective when acoustic conditions are unfavorable. However, it is not free from limitations. Mounting repeatability, sensing direction, pipe material, and distance from the leak can all affect signal quality and classification performance. For this reason, vibration sensing is best viewed not as a perfect replacement for acoustic sensing, but as a strong complementary modality.

2.5 Acoustic-Vibration Multimodal Sensing for Leak Detection

In practical monitoring, a leak can be observed through both acoustic and structural responses. For this reason, acoustic and vibration sensors may capture different but complementary evidence of the same leak event [42] [43]. Reviews of acoustic leak detection commonly describe a “sensor ecosystem” that includes noise loggers, contact microphones, accelerometers, hydrophones, and fiber-optic options, and emphasize that the signal quality depends strongly on pipe material, distance, and background noise [43].

In practice, multimodal sensing is useful because a single modality can fail under conditions that do not necessarily impair the other. For example, an acoustic channel may be more vulnerable to interference from traffic, pumps, or consumption-related noise, while a vibration channel (mounted directly to the pipe) is generally less sensitive to airborne noise but can be affected by structural coupling, mounting repeatability, and attenuation with distance particularly in plastic pipes [36]. When both sensors are available, a system can be designed so that one channel provides supporting evidence when the other channel is uncertain. This fits well with the project goal of reliable early detection using a low-power embedded device, where false alarms are costly because they trigger wasted site visits or unnecessary follow-up actions [44].

The literature on multimodal leak classification shows several ways to combine multiple sensors, typically described as early fusion (sensor/feature-level), Intermediate (joint) fusion, and late (decision-level) fusion as illustrated in Figure 2.5.1 [45] [46].

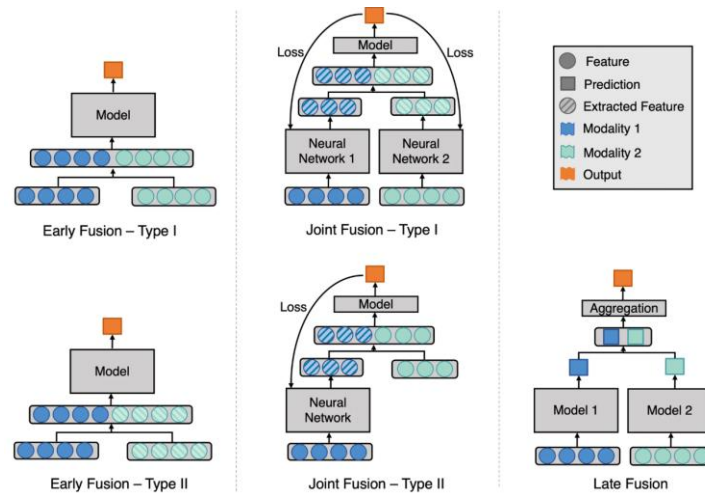


Figure 2.5.1: Fusion strategies using deep learning [46].

As illustrated in Figure 2.5.1, early fusion merges signals or features before feeding into a learning model (for example, concatenating synchronized time series), which can be powerful but is sensitive to sampling-rate differences, alignment errors, and different sensor dynamics [45] [46] [47]. Intermediate (joint) fusion merges extracted features or embeddings from each modality, which can reduce input dimensionality and help models learn complementary patterns [45] [46]. Late fusion combines the outputs of separate unimodal models, for example by averaging probabilities, weighted voting, or rule-based combination [45] [46] [42]. Late fusion is often attractive for embedded deployment because each channel can be processed

independently and the fusion rule can be implemented with minimal additional compute and memory [45] [47] [42].

A recent review reported that for the same task, multimodality fusion models often produced higher accuracy (1.2-27.7%) than conventional single modality models. [46]. A representative recent study that directly motivates late fusion for water leak classification is the work on wavelet scattering transform (WST) features with unimodal classifiers and late fusion across different sensor modalities [42]. The study compares accelerometer/vibration, hydrophone, and dynamic pressure data, then evaluates late fusion that combines unimodal predictions [42]. The results indicate that unimodal performance can differ by modality and network condition, while late fusion can recover some performance by combining complementary evidence [42]. In a related direction, multi-view feature fusion methods have been proposed that explicitly extract representations from time-domain, frequency-domain, and wavelet-domain “views” and fuse them to improve detection accuracy and even leak-size identification [48]. These studies support the core idea that a leak is a multi-faceted phenomenon and that relying on one sensor can discard discriminative information that becomes useful under noise, attenuation, or non-stationary conditions [48].

Despite this promise, there is still a practical gap between multimodal fusion as evaluated in many research papers and multimodal fusion as required for a low-power edge system. Many fusion pipelines are demonstrated offline, use high-dimensional features, and assume abundant compute and memory [45] [47] [42]. Embedded deployment creates immediate constraints where synchronization must be robust, per-inference latency must be bounded, and the processing chain must fit within limited RAM/Flash while still tolerating environmental variability. In this context, AutoML-style workflows for embedded signal classification become relevant because they can search for small, efficient pipelines that trade off accuracy against memory and compute budgets [49].

2.6 Signal Acquisition and Experimental Design for Leak Datasets

Leak detection performance depends heavily on the quality and structure of the dataset used for model development. In real water networks, labeled leak events are comparatively rare, difficult to schedule, and expensive to confirm [50]. This makes real-world datasets costly and often imbalanced, with far more “normal” samples than leak samples [50]. As a result, many studies rely on a combination of controlled laboratory experiments and field data [50]. Reviews of ML-based leak detection commonly describe the modeling workflow as a pipeline that begins with data collection and then moves into data processing and model development, and they repeatedly identify data scarcity and domain shift as continuing barriers to robust deployment [50].

Controlled rigs and experimental testbeds are widely used because they allow repeatable and systematic variation of key factors, such as pipe material, leak type, leak size, pressure/flow conditions, and sensor-to-leak distance. However, laboratory conditions typically have simpler background noise than real networks, so models can look strong in the lab while struggling in the field [34]. Field datasets have the opposite strengths and weaknesses. They include realistic noise sources, demand fluctuations, and installation variability, but often provide weaker control over ground truth and less coverage of controlled factors (such as calibrated leak size) [34]. These field-focused studies also show that increasing dataset quantity alone is not sufficient; the dataset must also cover meaningful variation in environment, pipe type, and sensor coupling to avoid brittle models [36].

It is important to acknowledge the lab-to-field gap explicitly. Controlled rigs simplify geometry (often a straight pipe segment for limited resources), standardize mounting, and reduce

uncontrolled noise sources [50]. Field networks include fittings, bends, soil coupling, variable pressure regimes, and diverse acoustic/vibration backgrounds [36] [50]. A strong dataset strategy therefore treats the rig dataset as an early-stage validation dataset that supports algorithm development and comparative testing, while positioning later field trials (or at least field-inspired augmentation and noise injection strategies) as necessary future work for deployment-level claims [50].

2.7 Signal Preprocessing for Leak Detection

Signal preprocessing is an important stage in leak detection because the measured sensor output is not always immediately suitable for classification. Before a model can distinguish between leak and background conditions reliably, the signal often needs to be conditioned so that irrelevant variation is reduced, and useful leak-related information is retained [43]. In the literature, preprocessing is commonly treated as part of the overall signal-analysis pipeline rather than as a separate optional step. Reviews of acoustic leak detection describe preprocessing and feature extraction as core stages that support later detection and classification, especially when measurements are collected under non-ideal conditions [43].

A broad purpose of preprocessing is to make the signal more informative for later analysis. In practical terms, this may involve suppressing unwanted components, improving the signal-to-noise ratio, or converting the raw measurement into a form that better exposes leak-related structure [51]. This is particularly relevant in leak detection because the useful signal is often weak relative to other measured content. For example, studies using acoustic recordings from water distribution systems have shown that model performance can improve when the signal is filtered or denoised before classification, indicating that preprocessing can materially affect the quality of the final result [34]. More recent work has similarly continued to use signal-enhancement steps when leak signatures are weak or mixed with other disturbances [51] [52].

The literature shows that preprocessing methods vary widely, but they can still be understood through a few general roles. One role is signal conditioning, where the goal is to suppress nuisance variation before features are extracted [43] [53]. Another role is signal transformation, where the raw signal is converted into a representation that makes discriminative patterns easier to analyse [54]. Fan et al. noted that leak-detection studies have used a broad range of preprocessing-related approaches, including time-domain analysis, frequency-domain analysis, time-frequency analysis, correlation-based processing, and other signal-conditioning methods before the final decision stage [43]. This suggests that preprocessing in leak detection is usually chosen according to the sensing problem and the type of information that later stages are expected to extract.

Preprocessing is also important because many leak-detection studies do not use long raw recordings directly as model input but instead derive a more compact representation before classification [50] [43]. Instead, they first derive a more compact description of the signal. Recent review work on machine-learning-based water leak detection highlights that data processing and feature extraction remain central steps in the modeling workflow, particularly when datasets are limited or when raw signals are highly variable [50]. In this context, preprocessing helps reduce irrelevant variation in the input and makes the subsequent classification problem more manageable. It is therefore not only a cleaning step, but also part of how the signal is shaped into a usable learning problem.

A similar pattern is also evident in vibration-based studies. For example, Tariq et al. showed that leak detection using MEMS accelerometer data in real water distribution networks relied on extracted signal features rather than on raw acceleration traces alone [36]. Their results also indicated that the modeling approach needed to account for different pipe classes, which

implies that the preprocessing and feature-design stages must be robust enough to support variation in real sensing conditions [36]. In this way, preprocessing contributes not only to noise reduction, but also to the consistency of the representation used for classification.

More recent studies show that preprocessing is not limited to conventional filtering alone. Some use wavelet-based denoising, decomposition methods, or enhanced spectrogram-based processing to better isolate leak-related structure before classification [51] [52]. Others combine signal enhancement with machine learning or deep learning so that the transformed signal becomes more suitable for later feature learning [55]. Although the specific methods differ, the literature shows a common trend in which classification becomes more reliable when the signal is first processed into a representation that reduces nuisance variation and preserves useful leak-related information [43] [53] [54].

Overall, the literature supports a broad and defensible view of preprocessing for leak detection. It is not limited to one particular operation but refers to the set of steps used to improve signal quality, reduce irrelevant variation, and prepare the data for reliable classification. Therefore, rather than fixing a single manually selected preprocessing method in advance, it is often more appropriate to consider a workflow that can evaluate multiple candidate signal-processing and model combinations under embedded deployment constraints.

2.8 Time-domain, Frequency-domain, and Time-frequency Representations

One of the key design choices in leak-detection research is how the measured signal is represented before classification. Even when the same sensing modality is used, different signal representations can emphasize different characteristics of the leak phenomenon. For this reason, the literature commonly organizes leak-related signal representations into three broad categories, time-domain, frequency-domain, and time-frequency representations [43]. This grouping is widely used in review papers because it provides a clear framework for discussing how leak information is captured and later supplied to machine-learning or deep-learning models [43].

Time-domain representations describe the signal directly as it changes over time. In leak-detection studies, this may involve the raw waveform itself, or statistical descriptors calculated from it, such as amplitude-based measures, RMS, kurtosis, crest factor, or other indicators of temporal behavior [34] [43]. The main strength of time-domain representations is that it preserves the original temporal structure of the signal, making it useful for analysing how leak-related behavior changes directly over time [43]. It is also relatively simple to compute, which can be attractive in low-complexity monitoring systems. However, time-domain representations may be less informative when the differences between leak and background conditions are expressed more clearly in spectral or time-frequency structure [43].

Frequency-domain representations describe how signal energy is distributed across frequency. In the leak-detection literature, these representations are commonly derived using Fourier-based methods and are used to identify spectral characteristics associated with turbulence, vibration response, or other leak-related phenomena [33] [38] [43]. Studies using acoustic and vibration sensing often report frequency-domain descriptors, including dominant spectral components and energy-related features extracted from selected frequency regions [43] [37]. The advantage of this representation is that it can reveal patterns that are less obvious in the raw waveform. However, a pure frequency-domain view does not show when a particular frequency component occurs within the observation window. This can limit its usefulness when the signal changes over time or contains transient events.

For this reason, time-frequency representations have become especially common in modern leak-detection studies [43] [54]. A time-frequency representation retains both temporal and spectral information, which is valuable in leak detection because measured leak signals are often influenced by time-varying operating and environmental factors [43] [54]. Methods such as the short-time Fourier transform, spectrograms, wavelet transforms, and wavelet scattering have therefore been widely used in the literature to represent leak-related acoustic or vibroacoustic patterns before classification [42] [43] [54]. Compared with a single global time-domain or frequency-domain description, these methods can provide a more structured representation of signals that vary over time, which is often useful for classification [42] [54].

Recent studies have also shown growing interest in more structured time-frequency representations, especially for difficult or noisy leak cases [54]. For example, some works have explored enhanced Mel spectrograms, wavelet scattering features, and related multi-view signal representations to better capture subtle leak signatures [42] [48] [54]. These approaches are attractive because they provide a more structured representation than a small set of scalar handcrafted features, while also avoiding the direct use of long raw signals as model input [54]. In this sense, time-frequency representations often act as a middle ground between raw waveforms and heavily compressed summary features.

This broader literature also provides a reasonable basis for including audio-inspired representations such as Mel spectrograms and MFCCs in leak-detection benchmarking. Although these features are more commonly associated with speech and general audio classification, they have also been applied in leak-detection studies, particularly when the sensing channel is acoustic or acoustic-emission based [56] [57]. Published work has shown that Mel-based and cepstral representations can provide useful classification performance in pipeline leak detection and localization tasks, especially when combined with machine-learning or deep-learning models [56] [57].

Overall, the literature does not indicate that one signal representation is always best for every leak-detection problem. Instead, each representation offers a different view of the measured phenomenon. Time-domain methods preserve temporal behavior, frequency-domain methods highlight spectral structure, and time-frequency methods combine both perspectives in a more detailed form.

2.9 Temporal buffering, segmentation, and windowing strategies

In practical leak detection, the model input is usually not treated as a single isolated sensor sample, because classification is typically based on short temporal sequences rather than individual points [58] [59]. For this reason, time-series classification systems usually operate on short sequences or subsequences of data, where each input preserves a portion of the temporal structure of the measured phenomenon [58]. In embedded workflows, this temporal input is commonly organized as a fixed-length buffer, defined as a continuous block of consecutive samples that serves as the basic unit processed by the classifier [59]. In NanoEdge AI Studio, for example, the dataset is explicitly organized into buffers, where each buffer corresponds to one line of time-series input and is used consistently during both benchmarking and deployment [59].

The choice of buffer length is important because it affects both the information content of the input and the resource demand of the embedded system [59] [60]. A longer buffer provides the model with more temporal context, which can be useful when the target pattern extends over a longer portion of the signal [59]. However, longer buffers also increase memory usage, processing cost, and latency, which can become restrictive on microcontroller-class hardware [60]. Conversely, if the buffer is too short, it may not capture enough temporal structure for

stable classification, especially when the signal varies across time [61]. Buffer design is therefore a compromise between temporal richness and deployment feasibility, especially when inference must run repeatedly on a low-power MCU [59] [60].

In addition to the use of fixed buffers, many time-series classification systems apply a sliding-window strategy in which consecutive subsequences are extracted and processed repeatedly [58]. This approach is widely used because it converts a long recording into multiple analysis windows without requiring the signal to be manually divided into isolated events beforehand [58] [62]. A sliding-window strategy is also useful in monitoring applications because it allows the model to examine local behavior continuously across time, rather than depending on one arbitrarily chosen segment [62] [63]. When overlap is introduced between consecutive windows, informative signal content is less likely to fall entirely at a single window boundary [62]. This is particularly useful in leak detection, where informative behavior may not begin or end neatly within one fixed segment [61] [62].

Recent leak-detection studies show that overlapping temporal segmentation is not only common, but often deliberate [62] [63]. For example, Essouabni et al. used a 3s sliding window with 90% overlap and explicitly described this as a deterministic way to transform a limited set of raw recordings into a much larger number of analysis windows [62]. Their study demonstrates that segmentation is not merely a preprocessing convenience, but also part of how the dataset is structured for learning [62]. In cases where leak signatures are intermittent or weak relative to the background, overlapping windows allow nearby temporal regions of the same recording to be evaluated repeatedly [61] [62]. This makes the method especially relevant when a single short window may not be sufficiently representative of the full leak event [61] [62].

Temporal segmentation also contributes to decision stability [63]. Predictions produced from short windows can fluctuate, especially when the environment is noisy, when the sensed leak is weak, or when the signal contains both informative and uninformative intervals [61] [63]. For this reason, practical systems often combine multiple short-window predictions over a longer decision horizon instead of relying on a single instant-level classification [63]. Müller et al., for example, described an acoustic leak-detection framework in which repeated measurements are evaluated across a predefined decision interval, partly to support practical deployment constraints and partly to avoid dependence on one isolated prediction [63]. This logic is closely related to the idea of temporal aggregation, where local predictions are accumulated into a more stable final decision over a longer observation period [63].

From a methodological point of view, buffering, segmentation, and windowing serve different but related purposes [58] [59] [63]. Buffering defines the temporal unit seen by the classifier [59]. Sliding windows determine how the recording is scanned across time [58] [62]. Segmentation and aggregation then help stabilize the final decision by combining information from multiple short inferences rather than trusting any single one in isolation [63]. Taken together, these strategies allow time-series models to remain sensitive to short local patterns while still supporting robust decisions over longer intervals [61] [63].

These studies provide the theoretical basis for adopting buffer-based classification, repeated temporal scanning, and multi-level aggregation in embedded leak-detection systems [58] [59] [61] [63].

2.10 Machine learning and deep learning models for leak detection

Machine learning and deep learning models are now widely used in water leak detection, particularly when the sensing pipeline includes acoustic, vibration, pressure, or multimodal data [50]. Recent reviews describe the leak-detection workflow as a sequence of stages that

usually include data acquisition, signal conditioning, feature extraction or representation learning, and final model selection [43] [50]. In this context, the choice of model is closely tied to the form of the input data [50]. Some studies use engineered features derived from time-domain, frequency-domain, or time-frequency analysis, while others use high-dimensional representations such as raw waveforms, scalograms, or spectrogram-like images [43] [50] [54]. For this reason, it is useful to distinguish between classical machine-learning models that operate on extracted features and deeper models that can learn directly from richer signal representations [50] [64].

Classical machine-learning models remain common in leak detection, especially when the signal has already been converted into a compact set of engineered features [50]. Support vector machines (SVMs) are commonly reported in vibroacoustic classification pipelines, including recent work that combines wavelet scattering features with unimodal classifiers across different sensing modalities [42] [50]. Random Forest (RF) models are also widely used in water leak detection, and a real-network MEMS accelerometer study reported that Random Forest outperformed several other classical models, with different performance observed for metal and non-metal pipes [36]. XGBoost and related gradient-boosted tree models are also reported as competitive baselines in feature-based pipelines, including vibration-sensor studies that compare multiple learning algorithms under controlled experimental conditions [37] [50]. These models remain attractive because they can achieve good performance without requiring extremely large datasets or end-to-end deep architectures [50].

Lightweight neural models such as multilayer perceptrons (MLP) also appear in leak-detection studies, particularly when the model input has already been reduced to engineered features rather than long raw sequences [50] [65]. In such cases, the neural network is used mainly as a classifier on top of an already reduced feature representation rather than as a full end-to-end feature-learning model [50] [65]. This makes MLP-based pipelines relevant when the representation stage has already captured the most important information, and the remaining task is to separate leak and background classes efficiently [50] [65]. For embedded and resource-aware workflows, such models may be preferable to deeper architectures when the available training data and hardware budget are limited [60] [66].

Deep learning models become more attractive when the input representation is high-dimensional or when the learning problem benefits from automatic feature extraction [64]. Convolutional neural networks are among the most widely used deep models in leak detection [53] [55] [56] [57]. One-dimensional CNNs are commonly applied to raw temporal signals or narrow-band sequences, while two-dimensional CNNs are often used when acoustic or vibroacoustic data are converted into scalograms, spectrograms, or other image-like time-frequency representations [53] [55] [56]. Recent studies have reported strong results using CNN-based pipelines on acoustic-emission data after wavelet or cepstral transformation, showing that local spatial or temporal patterns can be learned effectively from such representations [53] [56] [57].

Recurrent models such as long short-term memory (LSTM) networks are used when temporal dependence across longer sequences is important [40] [67]. In acoustic leak detection, LSTM-based models have been explored both as standalone temporal classifiers and as components in hybrid pipelines [40] [67]. Some recent studies discuss CNN-LSTM combinations, where convolutional layers extract local features and recurrent layers model how those features evolve across time [67]. This kind of hybrid structure is useful when the signal contains both local leak signatures and longer-range temporal dependencies [67]. However, such architectures are generally more demanding in terms of computation and memory than simpler feature-based models [64] [66].

More recent work has also introduced transformer-based architectures into leak detection [62] [68]. These models reflect a broader trend in time-series learning, where attention mechanisms are used to model long-range dependencies and complex temporal structure without relying entirely on recurrence [64] [68]. For example, transformer-based and frequency-informed architectures have recently been proposed for leak analysis in pipeline signals, showing that attention-based models can be competitive when the dataset and representation are rich enough to support them [62] [68]. Even so, these models usually require greater training and deployment resources than classical ML or lightweight embedded classifiers [64] [66].

Taken together, the literature does not point to one universally best model family for every leak-detection problem [50] [64]. Instead, model suitability depends on the sensing modality, the signal representation, the size and diversity of the dataset, and the intended deployment environment [43] [50] [64]. Classical ML models remain strong choices when informative features have already been engineered, while deeper architectures become more appealing when feature learning from high-dimensional input is needed [50] [64]. This distinction is especially important in embedded leak-detection research, where the choice between lightweight classifiers and more complex deep-learning models depends not only on accuracy, but also on memory, computation, and deployment constraints [60] [66].

2.11 Embedded AI, TinyML, and on-device deployment constraints

Embedded AI refers to deploying machine learning inference directly on small devices at the edge, typically without a full operating system and without access to cloud-scale computation and memory [66]. The TinyML literature emphasizes that this setting differs from PC and cloud AI because embedded processors operate under much tighter constraints in computation capability, memory availability, and power consumption [66]. A key practical implication is that the model-development workflow must be treated as a closed loop that jointly considers accuracy and deployability, rather than selecting a model solely based on predictive performance [60].

One of the most immediate constraints is memory, because microcontroller-class devices often have limited SRAM for activations and limited Flash for program code and model parameters [66]. The TensorFlow Lite Micro paper highlights that embedded devices have far less compute and memory than mobile systems [69]. It also notes that embedded inference frameworks often operate under very small memory budgets and avoid mechanisms common in larger systems, such as dynamic memory allocation and virtual memory [69]. Saha et al. similarly emphasizes that conventional ML models often have a high memory and compute footprint that hinders direct deployment on ultra resource-constrained microcontrollers, motivating specialized workflows and model optimization [60]. These constraints create a strong incentive to keep models compact and to control intermediate activation sizes, since SRAM usage can become a practical limiting factor even when Flash is sufficient for storing model weights [69].

Compute throughput and latency constraints are equally important because embedded devices typically operate under real-time sampling and must complete inference within tight timing budgets [69]. The TinyML review by Ray frames TinyML as shifting inference from high-end systems to low-end edge devices, while noting that maintaining accuracy under processing-capacity and reliability constraints remains a major challenge [66]. In time-series sensing applications, latency also affects whether the system can produce decisions at the required monitoring rate, and it can influence the practicality of duty-cycling and scheduling in low-power operation [60].

Model optimization techniques are therefore central to embedded AI, and quantization is among the most widely cited approaches [70]. Quantization reduces model size and can

improve inference efficiency by replacing floating-point operations with integer arithmetic, which is typically faster and more energy efficient on many embedded processors [71]. Jacob et al. propose an integer-only quantization scheme and show that quantization-aware procedures can preserve accuracy while improving the accuracy-latency tradeoff for on-device inference [71]. However, the literature also emphasizes that accuracy retention is not guaranteed and can depend on factors such as architecture sensitivity, calibration quality, and the degree of compression required by the target platform [66] [71]. This general result also helps explain why large, high-capacity models may lose accuracy when they are aggressively compressed or quantized to meet microcontroller deployment constraints [71].

Energy consumption is another defining constraint because many embedded devices are battery-powered and must operate for long durations without frequent maintenance [72]. The literature often presents on-device inference as beneficial because it can reduce communication and enable local processing, but it also recognizes that inference cost still needs to be managed through careful model selection, scheduling, and optimization [60]. TinyML surveys also highlight that embedded deployment should be evaluated not only by accuracy but by a multi-metric profile including memory footprint, latency, and energy [73].

Finally, embedded deployment is shaped by toolchain fragmentation and integration overhead, since real products require converting trained models into formats compatible with embedded inference engines and integrating them into firmware [69]. The TensorFlow Lite Micro paper describes fragmentation challenges (e.g., variation in instruction sets and FPU support) and motivates design decisions that enable portability while maintaining efficiency on constrained systems [69]. These considerations help explain why embedded AI workflows often prioritize practical deployability and reproducibility, such as model export, deterministic inference, and resource profiling, alongside algorithmic performance [66] [69].

2.12 AutoML and NanoEdge AI Studio for time-series classification

AutoML is broadly defined as automating parts of the machine learning pipeline, including model selection, hyperparameter tuning, feature engineering, and (in some cases) neural architecture search [70]. In embedded systems, AutoML is particularly attractive because the most optimized model is often not the most accurate model in isolation, but the model that satisfies a combined objective over accuracy, memory footprint, latency, and energy [74]. The Tiny-to-TinyDL survey explicitly discusses the role of optimization methods and tool chains, including quantization, pruning, and AutoML/NAS approaches, as enabling technologies for practical on-device learning and deployment [70].

Neural architecture search provides a well-known example of constraint-aware AutoML for edge deployment [74]. MCUNet, for example, frames TinyNAS as automatically handling constraints such as latency, energy, and memory under low search cost, illustrating how AutoML can be used to produce architectures that fit microcontroller budgets [74]. More broadly, embedded-AI surveys often present AutoML methods as practical responses to the difficulty of manual architecture and parameter tuning under hardware constraints [70] [74]. These NAS-focused approaches are mainly designed for deep learning architectures, but the same principle applies to time-series pipelines where preprocessing choices and classical ML model selection can dominate embedded feasibility [60].

In addition to NAS-oriented approaches, embedded AI workflows for sensor data increasingly rely on specialized toolchains that search not only model structure but also preprocessing and deployment-ready pipeline configurations under hardware constraints. NanoEdge AI Studio may be viewed as an embedded AutoML tool chain specialized for sensor time-series data, with a workflow intended for non-expert embedded developers [75]. ST’s official

documentation describes NanoEdge AI Studio capabilities as searching for optimal AI libraries given user data and explicitly frames an AI library as including both signal treatment (preprocessing) and a machine learning model [76]. The ST wiki also states that the Studio can search for optimal AI libraries composed of preprocessing and model components, implying that the search space spans both signal-processing blocks and classifier families rather than only tuning one fixed model [76]. This is directly relevant to time-series classification problems because performance often depends as much on representation and preprocessing as on the final classifier [60].

In embedded time-series projects, data collection strategy strongly influences AutoML outcomes because the search procedure can only optimize within the statistical range represented by the collected data [60]. ST’s datalogging guidelines emphasize that buffer size selection should be meaningful relative to the sampling rate and note that buffer sizes should be powers of two for optimization reasons, while also warning that overly short buffers may fail to capture relevant phenomena [59]. ST also documents a “Sampling Finder” process where sampling rate and buffer size are treated as tunable parameters to be selected based on the phenomenon and the intended model development [77]. These points align closely with the broader microcontroller-ML literature, which frames dataset design and parameter selection as part of a closed-loop workflow in resource-limited applications rather than as a one-shot choice [60].

AutoML toolchains are also attractive because they can reduce the engineering burden of manually testing many candidate pipelines, which is especially valuable under embedded constraints where each candidate may require careful profiling and integration [60]. The capability claims in NanoEdge AI Studio (e.g., selecting an ML algorithm for a given MCU and exporting a library) reflect a broader trend in TinyML toward end-to-end train-to-deploy workflows that emphasize deployability and iteration speed [75]. At the same time, the literature suggests that AutoML does not remove the need for rigorous experimental design, because poor dataset coverage or non-representative operating conditions can still lead to overfitting and brittle deployment behavior [60]. For this reason, NanoEdge AI Studio can be understood as a practical engineering workflow that automates candidate exploration under constraints, while still requiring carefully controlled data collection and structured evaluation protocols [60] [76].

2.13 Low-power IoT architecture for continuous monitoring

Continuous leak monitoring is not only a sensing problem, but also a system-architecture problem, particularly when sensor nodes are deployed in distributed infrastructure and are expected to operate for long periods with limited access to fixed power or maintenance support [78] [73]. For this reason, low-power design is a central requirement in IoT monitoring systems rather than a secondary implementation detail [78] [73].

A widely adopted strategy in low-power wireless sensing is duty cycling, in which the node remains in a sleep state for most of the time and wakes periodically to sense, process, and communicate only when necessary [79] [80]. This approach is effective because the active states of the processor, sensors, and radio generally consume much more energy than sleep states, so reducing active time can substantially extend operating lifetime [79] [80] [78]. Accordingly, practical low-power embedded systems are often designed around short active intervals followed by rapid return to deep sleep modes, while still preserving the timing and wake-up resources needed for reliable operation [80] [78].

For long-duration monitoring, the sensing policy is also important. Instead of sampling and transmitting continuously, many IoT systems adopt periodic or event-driven operation so that

sensing and communication occur only during scheduled checks or when a condition of interest is detected [80] [78]. This reduces unnecessary energy consumption during long periods of normal operation and is especially relevant to leak monitoring, where abnormal events are typically sparse relative to the total observation time [81] [78].

Another important architectural decision is the distribution of computation between the device and remote infrastructure. In conventional cloud-centered IoT systems, sensed data are transmitted to remote servers for storage and analysis, but this increases communication overhead, latency, and dependence on network availability [73] [82]. Recent reviews on TinyML and edge computing therefore emphasize the value of performing inference locally on the embedded node, particularly in applications that require timely response, reduced data transfer, and operation under constrained bandwidth or intermittent connectivity [78] [73] [82]. This shift toward on-device inference is closely related to energy efficiency because wireless transmission is often among the most energy-expensive operations in a sensor node [81] [78] [83]. If raw continuous sensor data are transmitted frequently, communication can dominate the overall energy budget and substantially reduce battery lifetime [81] [83]. A more efficient architecture is often to process the sensed signal locally and transmit only compact outputs such as alarms, health indicators, or summary information [78] [73] [82]. This system-level shift helps explain why edge AI has become increasingly attractive for continuous infrastructure monitoring [78] [73].

The communication layer must also be selected to support long-term remote deployment. Low-power wide-area network technologies are widely recognized as suitable for IoT applications because they are designed to provide long communication range while maintaining low power consumption [81] [83]. In water distribution monitoring, prior work has specifically shown that LPWAN technologies can support near real-time leak-related sensing in locations where external power is unavailable and long maintenance-free operation is required [81]. More broadly, LPWAN reviews describe these technologies as particularly relevant to distributed monitoring scenarios in which communication distance, battery life, and infrastructure cost must all be considered together [83].

As a whole, the literature suggests that practical low-power IoT monitoring nodes are usually designed around several coordinated principles [78] [79] [80] [83]. Sensing is often scheduled rather than fully continuous whenever the application allows it [79] [80] [78]. Processing is often kept sufficiently local to reduce unnecessary communication burden [78] [73] [82]. Wireless transmission is often kept brief and infrequent, especially when battery lifetime is a design priority [81] [83]. At the same time, the hardware and firmware must support reliable transitions between active and sleep states so that low-power operation does not compromise system availability [80] [78]. These considerations are especially important in distributed water infrastructure monitoring, where sensing points may be geographically dispersed, difficult to access, and expected to remain operational for long intervals between maintenance visits [81] [78]. Under such conditions, a monitoring solution is better assessed not only by detection accuracy, but also by its latency, communication load, and expected operating lifetime [78] [73] [82]. Overall, the literature indicates that effective low-power continuous monitoring depends on the combined design of sensing policy, embedded inference, sleep-state management, and communication strategy, rather than on any one subsystem in isolation [81] [78] [73] [82] [83].

2.14 Wireless communication for remote leak monitoring

Wireless communication is an important enabling layer in remote leak monitoring because sensing nodes may be deployed away from operators, fixed infrastructure, and convenient

wired access points [81] [29]. In distributed water networks, this makes wireless reporting important not only for convenience, but also for the practical deployment of continuous monitoring across multiple sensing locations [81] [29]. Recent leak-monitoring and smart-infrastructure literature increasingly treats communication as part of the wider sensing architecture through which field data and alarms can be transferred to supervisory or maintenance systems for timely response [81] [29] [82].

In this context, the communication layer is mainly used to deliver useful information from the sensing node to a remote receiver rather than to replace local sensing or decision-making [73] [84] [82]. Depending on the application, this transmitted information may include alarms, timestamps, device status, health indicators, or selected summary measurements that help the user interpret conditions at the sensing point [73] [82]. This reporting function becomes especially valuable in distributed monitoring because a leak-detection device is more operationally useful when its result can be accessed remotely without requiring repeated physical inspection or manual retrieval of stored data [81] [29].

The literature also shows that low-power wide-area communication is particularly relevant to water infrastructure monitoring [81] [85]. LPWAN technologies are attractive because they combine relatively long communication range with low power demand, which is important when sensing nodes are battery-powered and deployed in locations without permanent external power supply [81] [83] [86]. In the context of water distribution systems, Pointl and Fuchs-Hanusch note that LPWAN technologies can support near real-time leak-related monitoring while still enabling long maintenance-free operating periods [81]. This is especially important because monitoring points may be located at the periphery of the network, where both power access and communication infrastructure are limited [81] [85].

Within this broader LPWAN context, LoRa-based communication is relevant because it is designed for long-range, low-data-rate, and low-power IoT applications [87] [85] [86]. A useful distinction in the literature is that LoRa refers to the underlying radio technology, whereas LoRaWAN refers to a higher-level network protocol that is typically organized around gateway-based star topologies [87] [85]. LoRaWAN has been widely studied for smart water monitoring and leak detection because it can support distributed sensing over broad coverage areas with low energy demand [85] [86]. However, the literature also indicates that not all applications require the full overhead of a managed LoRaWAN architecture [87]. For some prototype systems or simpler reporting tasks, lighter LoRa-based arrangements may be more suitable when the main objective is reliable low-bitrate transmission rather than full network management [87].

A central architectural trade-off in wireless leak monitoring is whether to transmit raw sensor data or only compact inference results [73] [82]. Transmitting raw data offers greater flexibility for remote analysis, but it also increases communication load, airtime, and energy demand [73] [86] [82]. By contrast, transmitting compact outputs such as event flags, short summaries, or confidence-related indicators reduces bandwidth use and can significantly lower communication burden [73] [84] [82]. Recent TinyML and edge-inference reviews identify this trade-off as an important motivation for performing inference locally and sending only the most relevant information to the remote side [73] [82] [84]. Therefore, wireless communication in an edge-monitoring system is often more efficient when it is used to report distilled results rather than to stream continuous raw sensing data [73] [84] [82].

This distinction is particularly important in low-power leak monitoring because communication cost is not only a networking issue, but also a system-level energy issue [86] [82]. If raw acoustic, vibration, or pressure signals are transmitted frequently, communication can dominate the node's resource budget and reduce achievable battery lifetime [86] [84]. A

compact reporting strategy, by contrast, allows the radio stage to remain lightweight while preserving the essential function of remote notification [73] [86] [82]. This makes wireless transmission especially compatible with low-power edge AI architectures, in which the sensing node performs the main interpretation locally and uses the radio mainly for concise reporting [73] [82].

Collectively, the literature suggests that wireless communication is better understood as an enabling component of practical remote leak monitoring rather than as an optional add-on [81] [29] [82]. Its value lies in allowing distributed sensing nodes to report useful leak-related information without depending on permanent wired infrastructure [81] [85]. LPWAN technologies strengthen this possibility by making long-range and low-power reporting more feasible in water-network environments [81] [85] [86]. At the same time, communication becomes most efficient when paired with local processing so that only compact detection outputs or selected summaries need to be transmitted [73] [84] [82]. For prototype and early-stage deployment studies, lightweight LoRa-based reporting arrangements can still be useful for validating remote result transmission without requiring a full large-scale network architecture from the outset [87] [86].

2.15 Industrial Relevance and Research Positioning

The literature reviewed in this chapter indicates that water leakage remains both a technical and an operational challenge, which is why the topic has strong industrial relevance. Utilities are under increasing pressure to reduce non-revenue water, improve service reliability, and modernize aging water infrastructure without depending only on labor-intensive inspection methods [21] [22] [23]. In this context, leak detection is not only an academic signal-classification problem, but also an industrial need linked to maintenance cost, asset management, and sustainable water delivery [21] [22].

This relevance is reflected in the direction of the water industry itself. George Kent (Malaysia) Sdn Bhd is one example of an established industrial player in water metering and infrastructure, with long-standing involvement in the manufacture of water meters, valves, fittings, and related water-system components [88]. Its product portfolio includes residential, commercial, and industrial metering systems, including units that support automatic meter reading and digital monitoring functions [88]. More recently, the company has also expanded toward smart metering through the introduction of GK Ultra, described as a locally branded ultrasonic water meter developed with digital capabilities for remote monitoring, predictive maintenance, and leak-related functions [89]. This shift reflects a broader industrial movement away from purely mechanical metering and toward more connected, data-enabled water infrastructure [89].

The wider utility sector shows a similar trend. Industrial and municipal water systems are increasingly adopting sensor-based monitoring technologies to improve visibility of network behavior and reduce hidden leakage [90] [91]. In particular, low-power acoustic and IoT-enabled leak-monitoring systems are attractive because they can support distributed deployment, long operating life, and remote reporting in locations where wired infrastructure is impractical [90] [91]. These developments show that industry demand is no longer limited to detecting large failures after they become obvious [90] [91]. Instead, there is growing interest in solutions that can identify smaller or developing leak conditions earlier and with less dependence on manual field inspection [90].

At the same time, the literature reviewed in Sections 2.1 to 2.14 also shows that several important gaps remain. Many reported leak-detection studies focus on one sensing modality, such as acoustics, vibration, or pressure, rather than combining complementary sensing channels in a compact embedded workflow [32] [33] [42]. Many multimodal studies also

remain computationally heavy, offline, or oriented toward larger computing platforms rather than toward practical microcontroller-class deployment [42] [45] [47]. In addition, although embedded AI and TinyML research has advanced significantly, the combination of multimodal leak sensing, embedded model generation, temporal decision logic, low-power operation, and lightweight wireless reporting is still not widely documented in the literature as one integrated water-monitoring system [60] [66] [70] [78].

This gap is important from both research and industrial perspectives. From a research perspective, it means that strong classification performance alone is not sufficient if the resulting method cannot be deployed under realistic memory, energy, and communication constraints [60] [66]. From an industrial perspective, it means that there is still a need for leak-detection systems that are not only accurate, but also compact, interpretable, energy-aware, and suitable for long-duration field use [78] [81] [83]. The literature therefore points toward a practical need for embedded leak-detection approaches that combine multiple sensing modalities while remaining feasible for low-power distributed monitoring [42] [46] [78].

Viewed in this context, the present study can be positioned at the intersection of several strands of literature reviewed in this chapter. It draws on leak-detection sensing principles, multimodal classification, embedded AI deployment, temporal decision processing, and low-power IoT design, while bringing them together in a single edge-oriented leak-monitoring prototype [42] [50] [66] [78]. Rather than treating sensing, classification, power management, and communication as isolated problems, the study is motivated by the need to examine how these components may function together within one practical monitoring architecture [60] [73] [82].

3. Methodology

3.1 Overall Architecture

Figure 3.1.1 illustrates the overall architecture of the proposed system. Sensor inputs (vibration and acoustic) are fed into the B-U585I-IOT02A microcontroller, where a lightweight AI model performs leak classification locally. The NanoEdge AI Studio generated inference engine is embedded within the MCU firmware. The device remains in low-power mode and activates periodically. An alert signal (such as an alert message via LoRa P2P) is triggered upon leak detection. The board's available expansion interfaces and ports support future upgrades, including LoRa P2P communication through the Arduino header, Bluetooth connectivity, and LED-based mode indication.

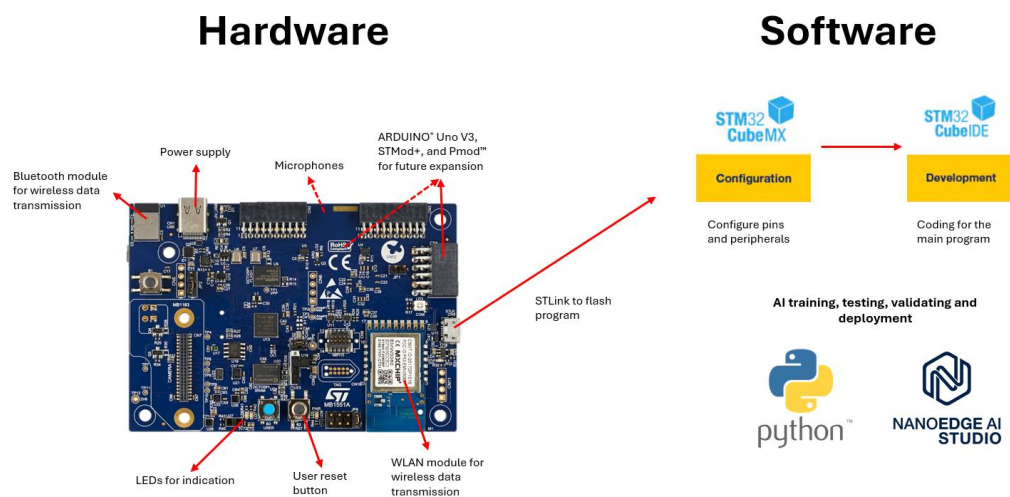


Figure 3.1.1: Overview of the hardware and software used.

3.2 Design and Hardware Integration

Using the B-U585I-IOT02A board as the base platform, create a prototype leak detection device. It was selected due to its integrated sensors and low-power Cortex-M33 MCU. This involves developing a 3D-printed casing to firmly place the device on water pipes, as well as incorporating an acoustic sensor (MEMS microphone) and vibration accelerometer (iNEMO 3D accelerometer) on the board.

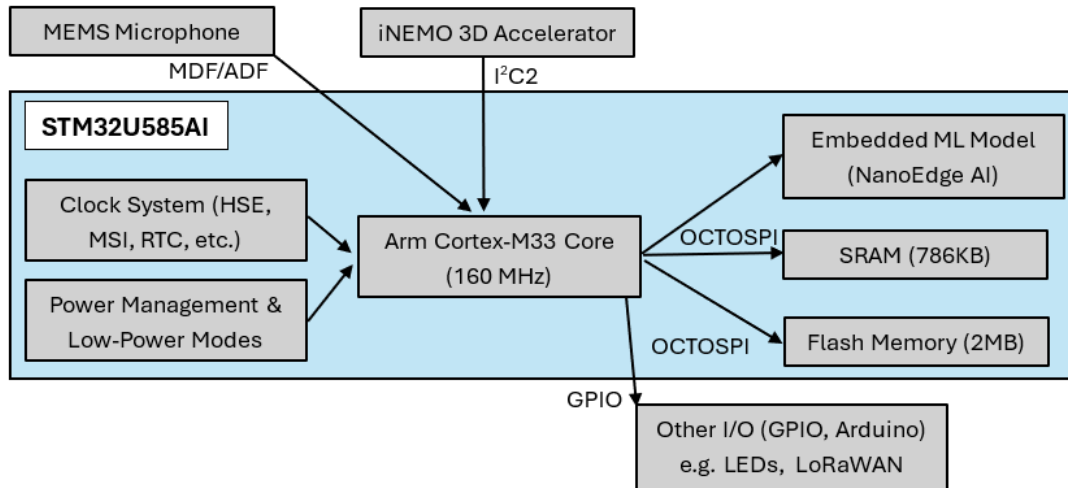


Figure 3.2.1: Proposed system architecture for the low-power leak detection device.

Figure 3.2.1 shows a block diagram of the B-U585I-IOT02A microcontroller, along with its main internal components and external peripherals. In addition to on-chip memory (2 MB flash and 786 KB SRAM) for storing code, data, and machine learning models [14], the STM32U585AI MCU (highlighted in blue) integrates an Arm Cortex-M33 CPU (160 MHz) with TrustZone. It includes a flexible power management unit with deep sleep mode support, and an internal clock system (MSI RC clock, HSE oscillator, etc.) [15]. The iNEMO 3D accelerometer provides vibration data via an I²C interface, while a MEMS microphone connects to the MCU to record acoustic leakage sound. These sensor inputs are read by the Cortex-M33 core and combined in real time. To classify leakage signatures, an embedded machine learning model (created using NanoEdge AI Studio) is stored in flash memory and executed on the MCU. Arrows indicate data paths: output signals (or stored data) leave the MCU, and sensor data enters the MCU.

3.2.1 Microphone

In this project, acoustic sensing is implemented using one of the two on-board digital microphones, MP23DB01HP, to capture sound signatures associated with water leakage. The MP23DB01HP is a compact, omnidirectional, low-power ($2\mu A - 800\mu A$) digital MEMS microphone that integrates a capacitive sensing element together with an interface IC, and it supports configurations where two microphones can be operated in a stereo arrangement if required [92].

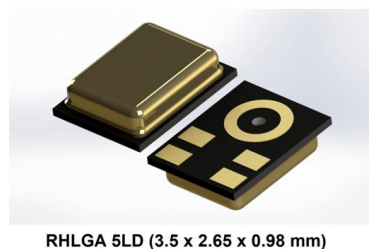


Figure 3.2.2: MP23DB01HP [92].

Using an on-board microphone reduces hardware complexity and improves repeatability because the acoustic front-end (microphone model, biasing, routing, and mechanical placement on the PCB) is fixed by the development platform rather than relying on external sensors or hand-wired assemblies. This also provides a stable baseline for dataset collection and subsequent model validation, since variations in sensor type and mounting can otherwise introduce unwanted shifts in the recorded signal characteristics.

The microphone signal path is configured to produce an audio stream suitable for embedded machine learning. Specifically, the acquisition is set to mono with a 16 kHz sampling rate and 16-bit sample resolution. At 16 kHz, the digitized audio preserves frequency content up to 8 kHz by the Nyquist criterion, which offers a practical balance between retaining informative acoustic details and controlling memory footprint, processing time, and energy consumption on the MCU [93] [94]. In implementation terms, the microphone’s digital output is acquired by the MCU’s digital audio capture path and converted into a 16-bit PCM stream suitable for subsequent preprocessing (such as framing/windowing and feature extraction as required by the model). Selecting a single-channel (mono) configuration further simplifies buffering and ensures a consistent input shape for inference, while leaving the second on-board microphone available for potential future enhancements such as spatial filtering, redundancy, or noise-robust dual-microphone processing.

To ensure deterministic acquisition behavior across operating cycles, the microphone is initialized as part of the device’s startup sequence and re-initialized whenever the system transitions from sleep back to an idle state before starting audio recording. The same initialization routine is also executed following a manual reset so that sampling parameters (rate, channel mode, resolution, buffer handling) are restored to a known configuration before any data collection or inference begins. This approach prevents inconsistent audio formatting caused by partial peripheral states after low-power operation and helps ensure that the recorded signals remain comparable across experiments. In addition, the microphone is only enabled when the device is actively acquiring data, and it is disabled during sleep intervals to reduce system power consumption, which is essential for achieving the project’s long-life operational objective.

3.2.2 Accelerometer

In this project, vibration sensing is implemented using the on-board ISM330DHCX inertial measurement unit (IMU) to capture mechanical vibrations propagating along the pipe during leak and non-leak conditions. The ISM330DHCX as shown in Figure 3.2.3 integrates a high-performance 3-axis digital accelerometer and 3-axis digital gyroscope in a compact system-in-package, designed for high stability, low noise, low power ($5.5\mu A - 430\mu A$) and synchronized sensing across axes, which is beneficial for repeatable vibration acquisition in embedded monitoring applications [95].

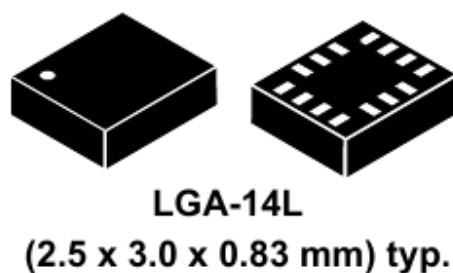


Figure 3.2.3: ISM330DHCX [95].

Leveraging the on-board IMU simplifies hardware integration because the sensor is already electrically and mechanically coupled to the development board, reducing variability that could arise from external sensor mounting, wiring, or inconsistent sensor placement. This provides a consistent sensing front-end for both dataset collection and subsequent model validation.

For the leak detection pipeline, the accelerometer is configured to a full-scale range of ± 4 g with a maximum sampling rate of 6.667 kHz, enabling the capture of high-frequency vibration components that may arise from turbulent flow and structural excitation near a leak site. The ± 4 g range was selected to provide adequate sensitivity for small-amplitude pipe vibrations while still providing headroom to avoid saturation under stronger mechanical disturbances such as handling shocks or higher flow-induced vibration [96] [97]. The sampled vibration stream is then buffered and segmented according to the project’s inference windowing strategy before being used for preprocessing and classification. Maintaining a fixed sampling configuration during both dataset recording and deployment is important to ensure that the statistical properties of the input data remain aligned with the training distribution, which directly affects embedded model reliability.

To ensure deterministic behavior across operating cycles, the accelerometer is initialized during device startup and re-initialized whenever the system wakes from sleep or after a manual reset. This approach ensures that key parameters, such as measurement range and output data rate return to a known state before any recording or inference begins. In addition, the sensor can be placed into a low-power state during sleep intervals to reduce idle energy consumption, aligning with the project’s long-life operation objective. Overall, the ISM330DHCX provides a robust vibration sensing modality that complements the acoustic channel, improving detection resilience in scenarios where environmental noise may mask leak sound, while vibration signatures remain detectable through structural propagation along the pipe.

3.2.3 LoRa P2P Communication Module

Wireless transmission is implemented using the RA-08H-Kit, shown in Figure 3.2.4, to provide long-range communication between the prototype node mounted on the pipe and a receiving unit connected to the monitoring computer. The communication module is included as a stretch goal to support the project objective of edge deployment, where the leak detection result can be transmitted without requiring a permanent wired connection to the sensing location.

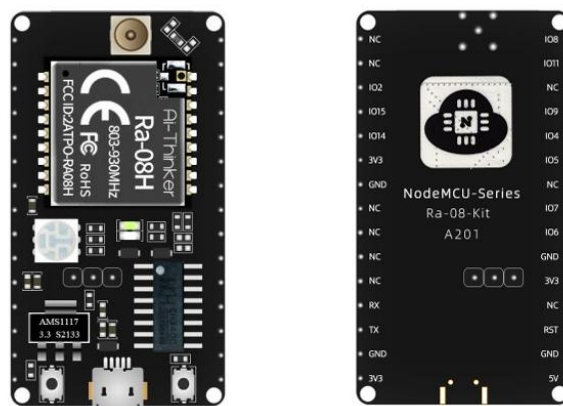


Figure 3.2.4: RA-08H-Kit [98].

The RA-08H family is designed for low-power wide-area applications and supports sub-GHz communication with UART-based host control and AT-command configuration. According to available module specifications, the RA-08H operates from a low supply voltage, provides high receiver sensitivity, and supports long-range communication with transmit current substantially higher than its receive current, making it suitable for intermittent burst transmission rather than

continuous radio activity [99]. This aligns with the current operating strategies of the device, which involves performing recording and inference locally on the B-U585I-IOT02A and transmitting only a compact result payload after each predetermined detection cycle. The transmitted payload consists only compact information such as timestamp, operating state, classification result, and confidence-related fields, thereby keeping the airtime short and limiting radio energy consumption.

In this project, two communication modules are used and are configured using a LoRa P2P approach, in which the sensing node sends its detection payload directly to a receiving base node connected to a computer. This was achieved by modifying a ping-pong example code from [100] for payload transmission and reception. This method was selected because water distribution systems may cover large and obstructed areas, making gateway-based communication less practical due to coverage limitations and the possible need for multiple gateways. A P2P arrangement provides a simpler and more suitable solution for the current prototype.

In a larger future deployment, results could be forwarded from one node to another until they reach the base node, allowing communication across extended infrastructure while keeping neighboring nodes within effective transmission range. Such an approach would require additional consideration of node identification, transmission delay, and node failure handling, but these aspects are beyond the scope of the current study and are reserved for future development.

Different antennas were used for the node and base communication modules, as shown in Figure 3.2.5. The base module was equipped with a physically larger antenna to improve transmission/reception capability and practical communication distance. In contrast, the node module used a shorter antenna because the prototype device is intended to be mounted on a pipe, where overall device size must remain compact. This design reflects a practical trade-off between wireless performance and mechanical integration. Since the node is part of the mounted leak-detection device, its antenna length and form factor must remain compatible with the size limitations of the holder and the surrounding pipe installation environment.



Figure 3.2.5: Antenna used for LoRa modules: node (left), base (right).

Antenna gain also influences the effective communication range by shaping how efficiently radiated power is concentrated in a preferred direction [101]. In general, a higher-gain antenna can improve signal strength at the receiver and extend effective coverage, although this is achieved by concentrating the radiation pattern rather than creating additional power [101]. The theoretical relationship between antenna gains and signal coverage is illustrated in Figure 3.2.6. In the present project, the larger antenna at the base side was selected to provide stronger reception performance, while the smaller node antenna was retained to satisfy device size constraints. This arrangement is suitable for the prototype because the sensing node prioritizes compact deployment, whereas the base receiver can accommodate a less size-restricted antenna.

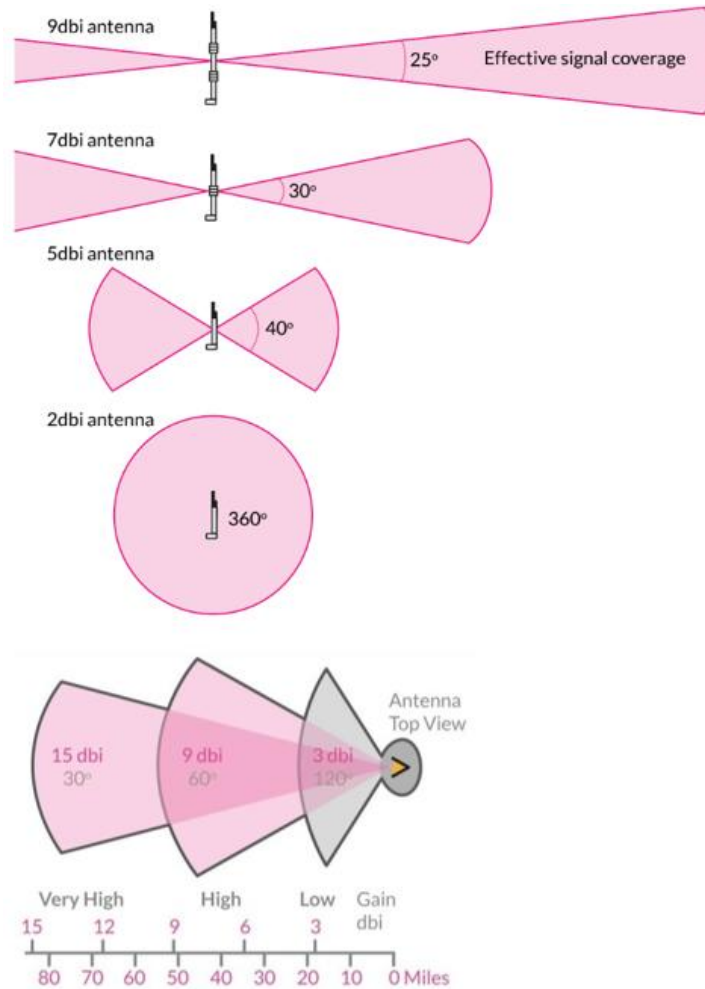


Figure 3.2.6: Relationship between antenna gain and effective signal coverage [102] [103].

To support reliable communication, a simple handshake mechanism was also implemented in the firmware using a ping-pong exchange with a nonce value. On the STM32 side, the host generates a message in the format PING:<nonce> and waits for a matching PONG:<nonce> reply within a defined timeout period. The node-side LoRa firmware forwards the ping message over the radio link and opens a temporary receive window, while the base-side firmware responds with the corresponding pong message upon successful reception. This lightweight procedure provides a direct method to verify bidirectional wireless communication before transmitting the actual detection payload and is therefore used as part of the communication validation workflow.

Overall, the RA-08H-Kit provides a suitable communication interface for the prototype because it allows leak decisions generated at the edge to be forwarded wirelessly using a lightweight, low-duty-cycle transmission strategy. Its integration through UART keeps the firmware architecture modular, such that sensing, inference, and communication remain separable blocks. This is beneficial for debugging, staged validation, and future deployment development, where the sensing node may participate in a wider multi-node relay structure across extended water infrastructure.

3.2.4 Power Hardware

The power subsystem is made to accommodate the various electrical requirements of the communication module and processing board while supporting battery-powered operation. The overall arrangement is illustrated in Figure 3.2.7.

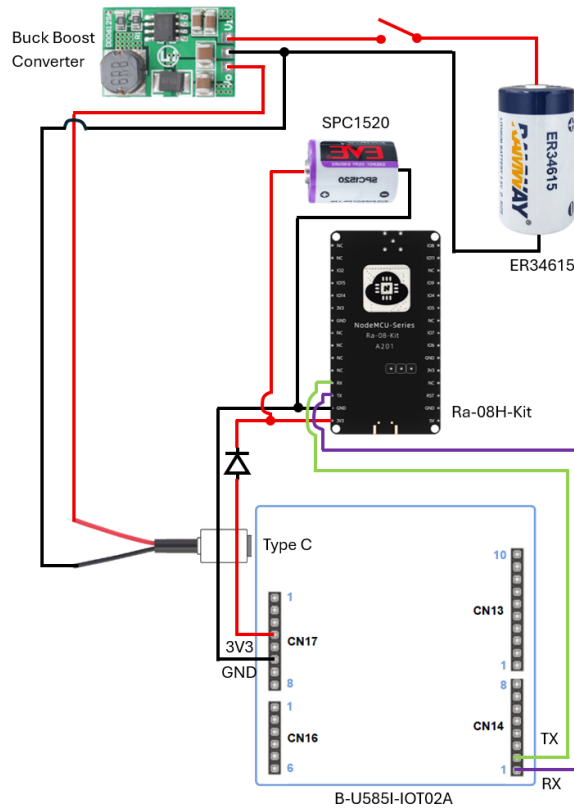


Figure 3.2.7: Device circuit diagram.

Figure 3.2.7 shows the connection between the battery ER34625, the DC-DC conversion stage, the pulse-assist capacitor, the user power switch, the B-U585I-IOT02A board, and the external LoRa P2P communication module. The design goal of this power hardware is not only to provide the required operating voltages, but also to manage transient current demand in a way that is compatible with long-life primary-battery operation.

The main energy source selected for the prototype is the ER34615 lithium thionyl chloride primary cell. This battery chemistry is widely used in long-life industrial and metering applications because it offers high energy density, low self-discharge, and long shelf life [104]. The ER34615 datasheet specifies a nominal voltage of 3.6 V and a nominal capacity of 19,000 mAh [105]. However, the datasheet also shows that while the cell is well suited to low average-current operation, its continuous and pulse current capability is limited compared with the short burst demands typically associated with radio transmission.

A DC-DC boost converter is used in the power path to increase the output voltage of the ER34615 battery to a level suitable for powering the development board. With the converter included, the board was able to power up successfully and remained operational.

Figure 3.2.7 shows the SPC1520 super pulse capacitor, connected in support of the Ra-08H-Kit power path. The purpose of this capacitor is to buffer short-duration high-current bursts during wireless transmission. A diode was added to prevent current backflow and damage the B-U585I-IOT02A board. This design choice is particularly important for lithium thionyl chloride battery, since these batteries are very suitable for long-duration sensing but are less ideal for repetitive high pulse loads if used without an auxiliary storage element. Application guidance for ER34615 systems commonly recommends pairing the battery with a capacitor or pulse-assist energy-storage element when short transmit bursts are required, so that the capacitor can supply the instantaneous current while the battery replenishes the charge more gradually between events [104].

The power hardware also includes a manual ON/OFF switch, allowing the entire device to be isolated during handling, transport, and maintenance. This is a practical feature for experimental work because it provides a clear hardware-level means of disabling the prototype without relying only on firmware state. In addition, it allows the power-up and reset sequence of the full system to be repeated consistently during testing, which is useful when validating startup behavior, peripheral reinitialization, and low-power wake-up transitions.

3.3 Simulation Rig

To enable controlled dataset collection and systematic validation of the proposed edge AI leak detection system, a specialized laboratory-scale water leak simulation rig was designed and constructed as shown in Figure 3.3.1. In methodological terms, the simulation rig functions as a controlled physical system model of a leaking water pipe, allowing the sensing device to be evaluated under repeatable and deliberately varied leak conditions before wider real-world deployment.

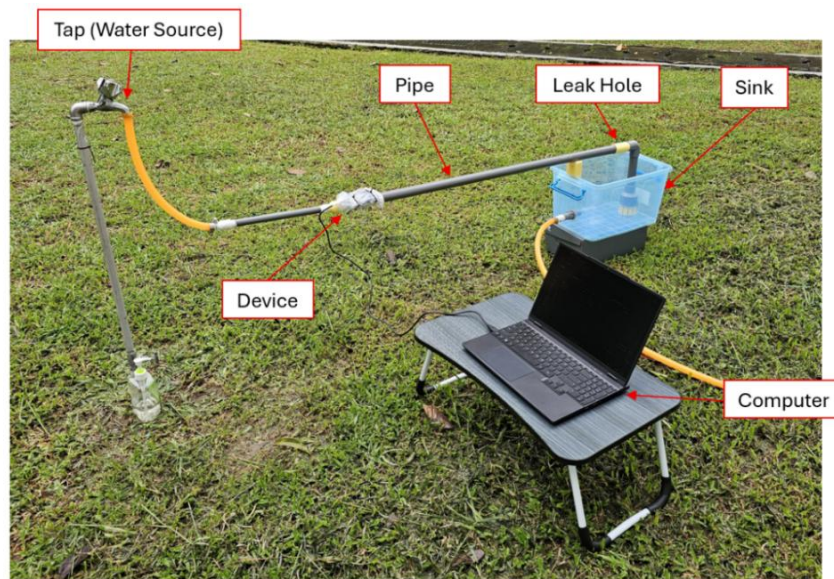


Figure 3.3.1: Simulation Rig setup.

As illustrated in Figure 3.3.1, the rig consists of a water tap acting as the source of water, which feeds water into a 25mm diameter main PVC pipe section where the detection device is mounted. Downstream of the mounting point, just before the water flows into the sink, a 3D-printed replaceable pipe with a calibration leak hole as shown in Figure 3.3.2 is installed to simulate a leak.



Figure 3.3.2: 3D printed pipe section with different sizes of leak hole.

The leak generation mechanism is implemented using a custom 3D-printed 25 mm pipe section with different sizes of leak hole, as shown in Figure 3.3.2. Multiple calibrated hole diameters were fabricated, including a baseline condition (no hole), 0.5 mm, 1 mm, 2.5 mm, 5 mm, and 10 mm. This modular design allows precise control of leak severity and flow characteristics and also allows for the easy creation of leaks of different sizes if needed in the future. Smaller leak holes simulate early-stage pinhole leaks, while larger leak holes represent more severe structural failures. The baseline configuration without any hole is used to collect non-leak background data for model training and false alarm evaluation. The ability to change hole sizes ensures that the dataset captures a broad spectrum of leak conditions, improving the generalization capability of the trained AI model.

The main pipe section has an internal diameter of 25 mm, selected to represent typical domestic or small-scale distribution pipelines, which is consistent with common sizes used in building cold-water plumbing (e.g., 20 mm minimum branches and 25 mm feeders in some internal plumbing specifications), but it does not capture the wider range of diameters found in service connections and larger distribution mains [106] [107]. The device's mounting position along this pipe can be adjusted to evaluate detection performance at different distances from the leak location. The maximum practical mounting distance from the leak hole is approximately 1.14 m, while the minimum distance is approximately 30 cm. This adjustable range enables investigation of signal attenuation characteristics for both acoustic and vibration signals as they propagate along the pipe. By systematically varying the mounting distance, the robustness and sensitivity of the AI model can be assessed under realistic deployment scenarios.

The simulation rig serves two primary purposes in this project. First, it provides a repeatable and controlled environment for dataset collection, enabling the collection of synchronized acoustic and vibration signals under well-defined leak and non-leak conditions. Second, it functions as the final verification platform for evaluating the deployed embedded AI model. By comparing detection outcomes across different leak sizes and distances, the system's classification accuracy, sensitivity, and false alarm rate can be quantitatively assessed. The controlled nature of the rig ensures consistency in experimental conditions, thereby improving the reliability and reproducibility of performance evaluation results.

The simulation rig provides a controlled and repeatable environment for leak detection development, but it remains a simplified physical representation of a real water distribution system. The simulation rig is limited to a straight 25 mm PVC pipe, a maximum sensor-to-leak separation of approximately 1.14 m, and a defined set of replaceable leak-hole geometries. These constraints are acceptable for controlled comparative testing and dataset generation, but they do not fully represent the wider range of pipe materials, diameters, geometries, and leak forms encountered in practical infrastructure.

3.4 Device Holder

A dedicated device holder was developed to provide secure attachment of the leak detection prototype device to the pipe during dataset collection and validation tests, while ensuring consistent sensor coupling so that measured vibration and acoustic characteristics remain comparable across experiments. Before fabrication, the holder was first designed and evaluated in SolidWorks to ensure that the complete prototype could be assembled into a compact and mechanically stable structure suitable for mounting on the simulation rig. The 3D CAD model, shown in Figure 3.4.1, was used to determine the overall enclosure size, the arrangement of internal components, and the positioning of external access features such as the ON/OFF switch, LoRa antenna and ST-LINK port (for flashing and debugging) opening.

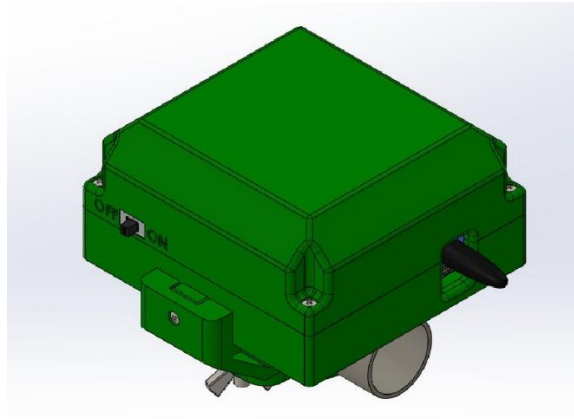


Figure 3.4.1: 3D CAD model of the prototype device holder developed in SolidWorks.

This modelling stage was important because the prototype had to accommodate the B-U585I-IOT02A development board, battery holder, power hardware, and LoRa P2P communication module within a single enclosure while still remaining sufficiently compact for pipe mounting. By developing the holder first in CAD, dimensional compatibility and assembly feasibility could be checked before printing, thereby reducing fabrication iteration and helping ensure that the final holder matched the physical requirements of the prototype. A more detailed exploded view of the holder assembly is shown in Figure 3.4.2, together with the bill of materials. This figure provides a clearer representation of how the printed parts, fasteners, development board, battery assembly, and pipe-clamping structure are integrated into the final prototype.

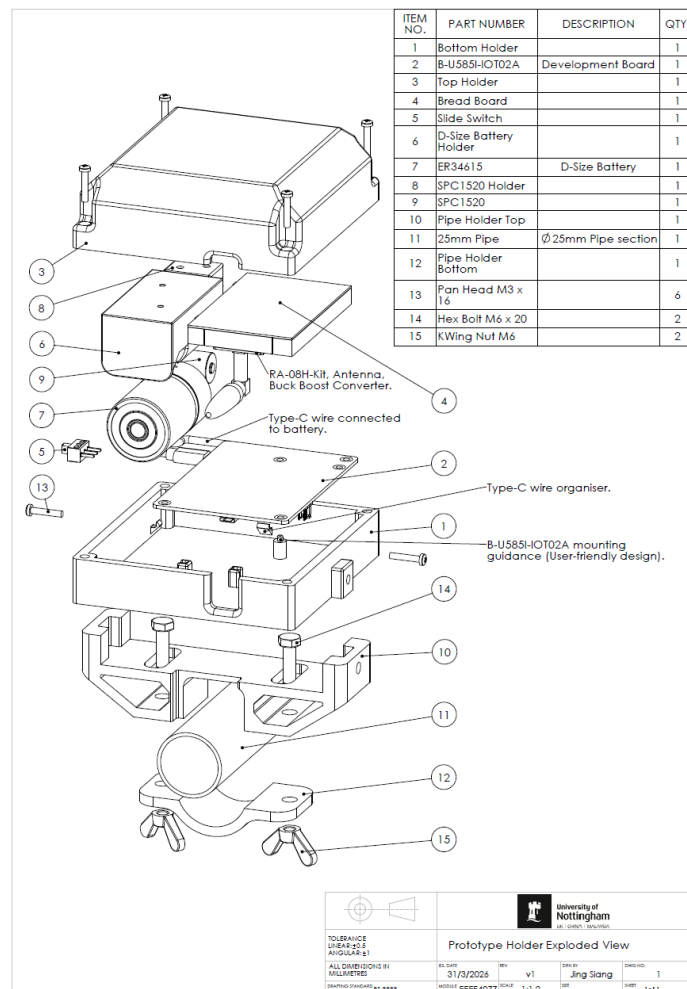


Figure 3.4.2: Exploded view and bill of materials of the prototype holder assembly developed in SolidWorks.

As shown in Figure 3.4.2, the exploded view also highlights several user-friendly design features introduced to simplify assembly and maintenance. In particular, the bottom holder includes a dedicated mounting guidance feature to help position the B-U585I-IOT02A correctly before fastening, thereby making installation easier and reducing the likelihood of incorrect placement during repeated assembly. The board is intended to be mounted in only one correct orientation, such that the side requiring external access is aligned toward the enclosure openings. In addition, a Type-C wire organizer was incorporated to manage cable routing more neatly and reduce mechanical interference with surrounding components. These design considerations improve assembly consistency and usability, which is beneficial during repeated experimental installation and removal of the prototype.

Item number 10, 11, 12, 14, and 15 in Figure 3.4.2 are the parts of the mounting mechanism that were developed around a clamp-based attachment method so that the enclosure could be fixed to the pipe without permanently modifying the test rig. This approach was selected to achieve a stiff, repeatable mechanical connection while avoiding destructive installation methods such as drilling or tapping into the pipe. Such a choice aligns with established vibration measurement guidance that emphasizes rigid mounting to preserve resonant frequency and maintain measurement fidelity [108]. In addition, clamp-style mounting methods are commonly used in experimental vibration monitoring when drilling or tapping is impractical; published evaluation work shows that non-destructive mounting approaches, including clamp-based methods, can provide usable bandwidth for monitoring applications when care is taken to ensure secure attachment [109]. As a result, the holder was designed to use a two-point fastening arrangement based on bolts and nuts that apply controlled compression to the pipe-contacting surfaces.

Beyond mechanical retention, the holder geometry was also designed to avoid obstructing the sensing elements, particularly the MEMS microphone. Dedicated microphone clearance features were incorporated so that the acoustic port is not blocked by the holder during operation. This design choice aligns with established MEMS microphone integration guidelines, which emphasize that the housing ports and acoustic paths must be properly positioned and kept unobstructed to avoid unnecessary attenuation, distortion, or abnormal frequency response due to poor acoustic path design or partial obstruction [110]. The holder and enclosure were therefore co-designed so that key device interfaces remain accessible when mounted, including the switch and antenna routing required for practical system operation.

The external LoRa antenna was intentionally routed outside the enclosure wall, while the ON/OFF switch was placed in a position that allows straightforward user access without removing the unit from the pipe. Locating the antenna externally, while avoiding close proximity to conductive parts, is a standard RF integration practice because nearby metal objects and unfavorable enclosure configurations can detune antennas and degrade radiation efficiency; hardware integration guidance typically recommends maintaining a keep-out region from conductive materials and validating antenna performance in the final mechanical assembly [111]. Overall, the CAD modelling stage was not only used to define the shape of the holder, but also to integrate the electrical, sensing, and usability requirements of the prototype into a manufacturable mechanical design before fabrication.

3.5 Edge AI Model Development

STMicroelectronics' NanoEdge AI Studio is used to train a lightweight machine learning model for leak detection. NanoEdge AI Studio is an AutoML tool that generates optimized C libraries of AI algorithms specifically designed for STM32 microcontrollers [13]. For training, the sensor data were preprocessed into a format suitable for NanoEdge AI Studio. An optimized

signal-classification library that satisfies the STM32U585AI's memory and processing limitations was automatically generated using the studio [76].

3.5.1 Dataset Collection and Preprocessing

Dataset collection involves using the prototype on a controlled leak simulation rig to record baseline (no-leak) and leak event data. To prevent the model from overfitting specific scenarios, it is important to focus on capturing a variety of conditions, such as varying leak sizes, distance from leak, and background noise. Prior research has demonstrated the importance of this diversity [16].

The first stage of the NanoEdge AI workflow is to prepare the recorded sensor data in a format that matches the input requirements of the Studio. For time-series classification projects, NanoEdge AI Studio expects the data to be provided as temporal buffers rather than as isolated samples [76]. In this format, each line of the dataset corresponds to one independent signal example containing multiple samples collected at a fixed sampling frequency. ST states that, for time-series data, the buffer size and the sampling frequency must remain consistent throughout the whole project, including dataset preparation, benchmarking, and later inference [76]. The Studio also recommends using a buffer size that is a power of two. In this project, a buffer length of 512 samples was selected for both the acoustic and vibration channels, and this same buffer size was preserved throughout model development and embedded deployment. This choice is consistent with the Studio requirements and also fits the later firmware design, where the deployed inference libraries are called repeatedly using 512-sample sliding input windows. The motivation for working with buffers instead of individual samples is illustrated conceptually in Figure 3.5.1.

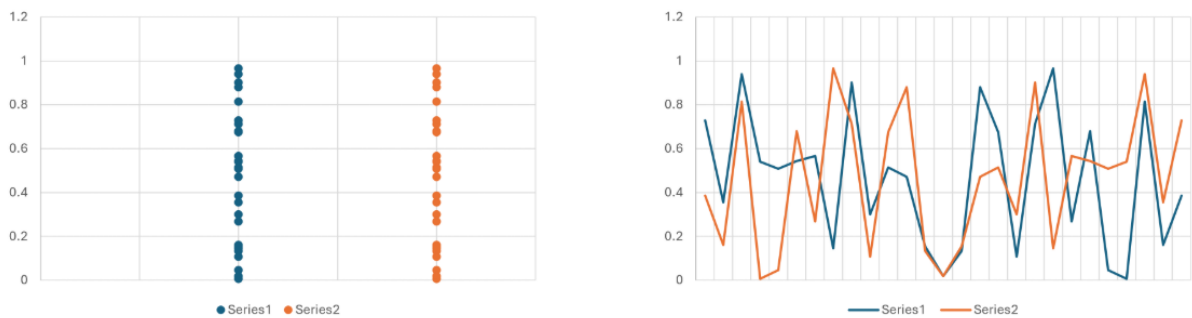


Figure 3.5.1: Samples independently (Left) versus Samples as temporal buffers (Right) [76].

As explained in the NanoEdge AI Studio documentation, a single sample represents only one point in time and therefore contains limited information about the physical phenomenon being studied [76]. A temporal buffer, in contrast, preserves the trace of the signal over time, making it easier for the model to distinguish patterns that would be impossible to classify from isolated scalar values alone [76]. ST's example specifically shows that two classes can contain the same individual values while still being distinguishable once their temporal ordering is preserved [76]. For this reason, temporal buffers are more informative than standalone samples when the classification problem depends on time-varying behavior, which is the case for leak-induced acoustic and vibration signals.

To support practical dataset generation, a dedicated Python data logger was developed to automate the conversion of raw prototype output into the required training format. In the implemented workflow, the B-U585I-IOT02A prototype transmits sensed acoustic and vibration data over the ST-LINK virtual COM port using structured UART markers such as `--BEGIN_SAMPLE---`, `---BEGIN_AUDIO_CSV---`, and `---BEGIN_VIB_CSV---`. The logger running on the PC listens to these markers, extracts the incoming data streams, and saves them

into raw one-column CSV files for both modalities. It then automatically converts the raw recordings into 512-sample buffer CSV files for NanoEdge AI training, placing them into the respective directory under the selected label and modality. The script also keeps the sampling assumptions explicit, using 16 kHz for the acoustic channel and approximately 6.667 kHz for the vibration channel. A screenshot of the data logger user interface is shown in Figure 3.5.2 and the Python code can be found in this [link](#).



Figure 3.5.2: Screenshot of the data logger UI.

In practical use, dataset collection was performed by mounting the prototype on the simulation rig and connecting the B-U585I-IOT02A board to a PC through the ST-LINK port while the logger script was running. The user first selected the required label (folder name) and base name (of the file) in the logger interface, then operated the simulation rig under either leak or background conditions. During recording, the firmware sent the acoustic and vibration samples in the predefined UART format, and the PC software converted them automatically into the required dataset outputs. This arrangement reduced manual formatting effort and helped keep the training data consistent across repeated recording sessions. It also made the collection process easier to scale when multiple leak sizes, distances, and noise conditions were being recorded.

In addition to the NanoEdge AI training format, the logger also generated an optional secondary dataset format for later benchmarking against an alternative model under Stretch Goal V. Specifically, the raw acoustic CSV was converted into a 16 kHz mono WAV, while the vibration CSV was converted into a one-column signal file. This additional export path did not affect the NanoEdge AI workflow directly, but it was introduced so that the same rig recordings could later be reused for YAMNet-based comparison, thereby avoiding duplicate recording effort and preserving consistency across the two model-development tracks.

3.5.2 Model Training

For this project, model development did not follow a conventional manual training pipeline in which the user explicitly selects feature extractors, defines the model architecture, and tunes hyperparameters separately. Instead, NanoEdge AI Studio automates much of this process. After the project type, target MCU constraints, sensor type, and dataset files are defined, the Studio searches for suitable combinations of signal preprocessing methods, machine-learning

models, and internal hyperparameter settings [76]. In this sense, the training stage for the present classification task is integrated into the Studio workflow itself rather than being handled as a separate coding step by the user.

Accordingly, the main engineering task at this stage was not to hand-design the classifier, but to ensure that the imported data, sampling conditions, and project setup were coherent with the physical phenomenon being studied. This included keeping the buffer size and sampling methodology constant, ensuring that the collected leak and background signals were representative of the intended operating conditions, and iteratively improving the dataset when necessary.

3.5.3 Benchmarking and Library Export

In NanoEdge AI Studio, the main model-development step for a classification project is carried out through the benchmarking process. Rather than testing only one fixed pipeline, the Studio evaluates a large number of candidate libraries built from different combinations of signal preprocessing, machine-learning model type, and hyperparameter settings. ST describes this as a search for the most relevant NanoEdge AI library given the user’s MCU target, memory constraints, sensor type, and imported signal examples [76]. The resulting candidate libraries may differ in their preprocessing flow, classifier family, internal parameters, and resource usage. An example of the type of generated ML pipeline explored during this process is shown in Figure 3.5.3.

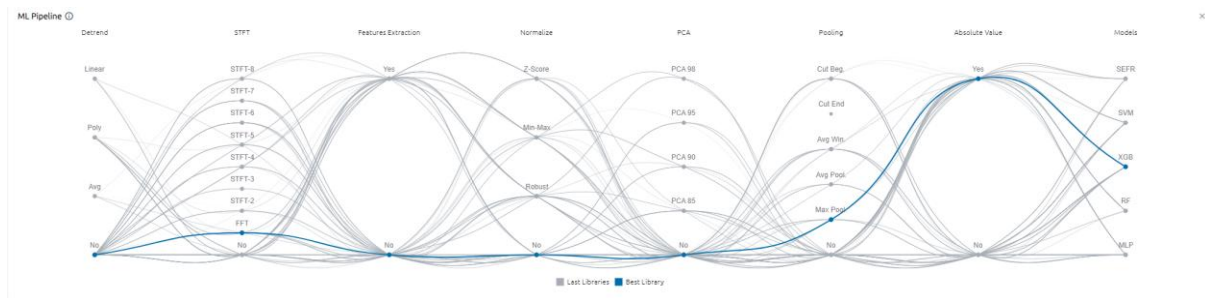


Figure 3.5.3: Example of ML pipeline used in benchmarking in NEAI Studio.

Table 3.5.1 listed out the example candidate processing and model options explored during benchmarking in NanoEdge AI Studio.

Table 3.5.1: Example candidate processing and model options explored during benchmarking in NanoEdge AI Studio.

Stage	Candidate options
Detrend	Linear, Poly, Aug, No
STFT/FFT	STFT-0, STFT-1, STFT-3, STFT-4, STFT-5, FFT, No
Feature extraction	Yes, No
Normalize	Z-score, Min-Max, Robust, No
PCA	PCA 80, PCA 85, PCA 90, PCA 95, No
Pooling	Cut Beg, Cut End, Avg Ws, Avg Pool, Max Pool, No
Absolute value	Yes, No
Model	SEFR, SVM, XGB, RF, MLP

As illustrated by Figure 3.5.3 and summarized in Table 3.5.1, benchmarking in NEAI Studio does not evaluate only one fixed processing pipeline. Instead, the Studio explores multiple candidate combinations across several stages, including optional detrending, frequency-domain conversion, feature-extraction, normalization, dimensionality reduction, pooling strategy, absolute-value transformation, and the final machine-learning model type. The presence of “No” in several stages indicates that the Studio may also choose to skip a processing step entirely if that produces a better candidate library. In this way, benchmarking functions as an automated search across multiple preprocessing and model-design alternatives before selecting the best-performing embedded library. The final selected acoustic and vibration pipelines are discussed later in Section 4.2.

During benchmarking, candidate libraries are ranked using a primary performance indicator called the score, which is derived from several secondary metrics. For classification projects, ST states that these secondary indicators include balanced accuracy, accuracy, F1-score, Matthews correlation coefficient (MCC), a custom certainty-related measurement, and the RAM and Flash memory requirements of the resulting library [76]. The benchmark therefore does not optimize accuracy alone. Instead, the Studio searches for a library that provides strong classification performance while still fitting within the memory limits of the selected embedded target. The documentation also notes that execution time can be estimated during the benchmark, although this is provided as an estimation rather than a direct measurement on the final hardware [76]. This makes benchmarking particularly suitable for an embedded-AI workflow, where predictive performance must be balanced against resource constraints rather than being treated as an isolated objective.

For this project, benchmarking was used as the practical equivalent of model selection and training. Multiple datasets were imported into the Studio, and the benchmark was allowed to explore a large range of candidate libraries. From these, the best performing classification library for each sensing modality was identified using the Studio’s ranking and validation outputs. As emphasized in the ST documentation, benchmark quality depends strongly on the coherence of the imported signals, the constancy of sampling conditions, and the appropriateness of the chosen buffer size for the physical phenomenon under study [76]. Thus, if benchmark quality is poor, the recommended corrective actions include adjusting the sampling frequency, changing the buffer size, improving signal quality, and revising the diversity or quantity of the imported signal examples rather than only rerunning the search unchanged [76].

Once a suitable library had been identified, NanoEdge AI Studio was used to export the deployment package. The Studio provides a deploy function that generates the files required to integrate the selected library into embedded firmware. For deployment in the STM32 codebase, the key files are the static precompiled NanoEdge AI library file, typically provided as `libneai.a`, together with the associated main API header file `NanoEdgeAI.h` [76]. The export package also includes additional metadata files such as a JSON description of the generated library and benchmark information [76]. In the present implementation, the compiled library file and the main header file were the essential files required for deployment on the STM32U585AI, since they allow the firmware to initialize the model and call the inference functions from C code. The resulting exported libraries were then integrated into the embedded firmware as described later in Section 3.6.

3.5.4 YAMNet Detection Model Training

This section describes the development of an alternative AI model based on a YAMNet-inspired approach, implemented as part of Stretch Goal V, AI Model Performance Comparison, where the deployed embedded model (NEAI) is benchmarked against another model

(YAMNet) to evaluate detection performance. To achieve this, a Python-based training pipeline was developed to construct both acoustic and vibration classification models. The objective was not to replace the primary embedded model, but to provide a reference for performance comparison under similar dataset conditions.

For the acoustic channel, the model adopted a feature extraction approach based on Mel-Frequency Cepstral Coefficients (MFCCs). Raw audio signals sampled at 16 kHz were first converted into time–frequency representations using Mel spectrograms, followed by MFCC transformation to capture perceptually relevant features. The extracted features were segmented into overlapping temporal patches (approximately 0.96 s per patch with 50% overlap) to preserve temporal dynamics while maintaining compatibility with embedded deployment constraints. A compact two-dimensional convolutional neural network (2D CNN) was then trained using these MFCC patches. The dataset was split into training (70%), validation (15%), and testing (15%) subsets. The model was trained using a batch size of 32, a learning rate of 0.001, and up to 300 epochs, with reproducibility ensured through fixed random seeds.

For the vibration channel, a similar training strategy was applied, but without MFCC feature extraction. Instead, raw vibration signals sampled at approximately 6.6 kHz were segmented into overlapping time windows of similar duration. These windows were used directly as input to a one-dimensional convolutional neural network (1D CNN), allowing the model to learn features directly from the temporal signal.

After training, both models were converted into TensorFlow Lite format and further processed using STM32Cube AI Studio. The STM32Cube AI Studio was used to analyse the models, optimize them for embedded deployment, and generate C code for integration into the STM32-based prototype. The conversion process ensures that the trained models can be executed within the memory and computational constraints of the embedded system [112]. The optimized acoustic and vibration models' architecture are shown and discussed in Section 4.3.2.

3.6 Deployment on Embedded Hardware

After benchmarking and exporting the trained models from NanoEdge AI Studio, the final deployment step was to integrate the inference libraries into the STM32 firmware and execute them within a periodical embedded workflow. In the final implementation, two separate classification models were deployed, one for the acoustic channel and one for the vibration channel. The system performs independent on-device inference for each sensing modality and combines the results later using a late-fusion decision layer. This structure was selected because it preserves modality-specific behavior, allows each model to be tuned separately, and simplifies debugging during deployment. At firmware level, the acoustic and vibration models are integrated as separate NanoEdge AI libraries with independent initialization and inference calls.

The deployed firmware follows a periodic operating sequence of sleep → wake → sensing → inference → LoRa P2P transmission → sleep. In practical operation, the MCU remains in a low-power state for most of the time and wakes up only at scheduled intervals. Upon wake-up, the required peripherals are re-enabled, the sensors are configured to their known operating states, and a 30s detection cycle is started. During this active period, the board records both microphone and accelerometer data, runs local inference, performs a compact leak decision, transmits the result through the LoRa P2P module, and then returns to low-power mode. This section focuses on the embedded deployment and execution of the models, while the detailed electrical behavior of the wake-sleep cycle is discussed later in section 4.4 Power Consumption.

The inference process was constrained by the data shape expected by the exported NanoEdge AI libraries. Each individual inference call receives a fixed-size input buffer of 512 samples, which is the same signal length used during model training. Therefore, the firmware does not classify an entire 30s recording in one pass. Instead, the 30s recording is made out of 10 segments and each segment is a 3s recording, and the segment is scanned repeatedly using a sliding input buffer. For both sensing channels, the firmware performs 30 sliding inference slots per segment, where each slot is a buffer containing 512 raw samples. Each slot produces a pair of class probabilities, namely Leak and Background, together with the predicted class label. These slot-level outputs are then accumulated into a higher-level decision for the full 3s segment. This sliding arrangement is explicitly implemented in both the acoustic and vibration inference modules.

The use of sliding inference slots serves two purposes. First, it allows the model to examine local signal behavior throughout the whole 3s segment instead of depending on only one short snapshot. Secondly, it reduces the risk that a short but meaningful leak signature is missed simply because it occurs near the boundary of a single inference buffer. This is especially relevant for leakage phenomena, where the signal may appear either as a more persistent turbulent pattern or as short transient droplet-like events. Prior work in sound event detection has shown that post-processing and temporal smoothing of frame-level probabilities can materially affect the final detection score, while acoustic-emission leak studies have similarly used sliding-window thresholding to retain both burst-type and continuous-type leak behavior [113]. The present deployment therefore adopts sliding inference not only for compatibility with the embedded model input length, but also as a practical way to strengthen temporal robustness before making the final decision.

A further design choice was to keep the number of inference slots fixed at 30 for both sensing channels, even though the sampling rates differ. The audio channel runs at 16 kHz, while the vibration channel runs at approximately 6.67 kHz. By the Nyquist criterion, these correspond to approximate upper retained frequency contents of 8 kHz and 3.33 kHz, respectively [93]. However, by keeping the inference buffer length fixed at 512 samples and using the same number of sliding slots per 3s segment, both sensing modalities still produce temporally aligned decision streams over the same observation period. In other words, the physical duration represented by one slot is different for audio and vibration, but the overall segment-level decision structure remains synchronized in time. This is advantageous for future development because it preserves a common timing framework for dual-sensor fusion without forcing both sensors to share an identical sampling rate.

The final embedded implementation therefore consists of three tightly connected layers. The first layer is sensor acquisition, where raw acoustic and vibration signals are buffered during the active interval. The second layer is modality-specific inference, where each sensor stream is processed by its own NanoEdge AI model using repeated 512-sample sliding slots. The third layer is the decision layer, where the slot outputs are converted into a 3s segment verdict, then into a 30s sensor-level decision, and finally into a late-fusion final verdict. This layered deployment strategy keeps the system modular, the sensing front end, the two AI models, and the final decision logic can be updated independently without redesigning the whole firmware.

3.7 Decision Logic (Late Fusion)

In this project, the final leak decision was not taken directly from a single NanoEdge AI output, because the embedded classifier operates on short fixed-length temporal buffers rather than on a full longer recording. A structured multi-stage decision layer was therefore introduced to combine multiple local predictions into one stable device-level verdict. This was expected to

improve robustness because leak signals are time-varying in nature and may not appear uniformly throughout the recording. Instead, leak evidence may appear either as a continuous leak signal over time or as short, irregular leak-related bursts. For this reason, the proposed decision logic was designed to evaluate not only the confidence of individual inferences, but also how that evidence behaves across time. It therefore combines two complementary ideas: temporal hysteresis, which favors persistent evidence over time, and strong fraction, which allows short but confident evidence to remain influential. Each detection cycle comprises a 30 s recording, which is subsequently partitioned into shorter segments and processed through the proposed decision logic to determine the final classification result. A graphical overview of the 30 s segmentation is shown in Figure 3.7.1.

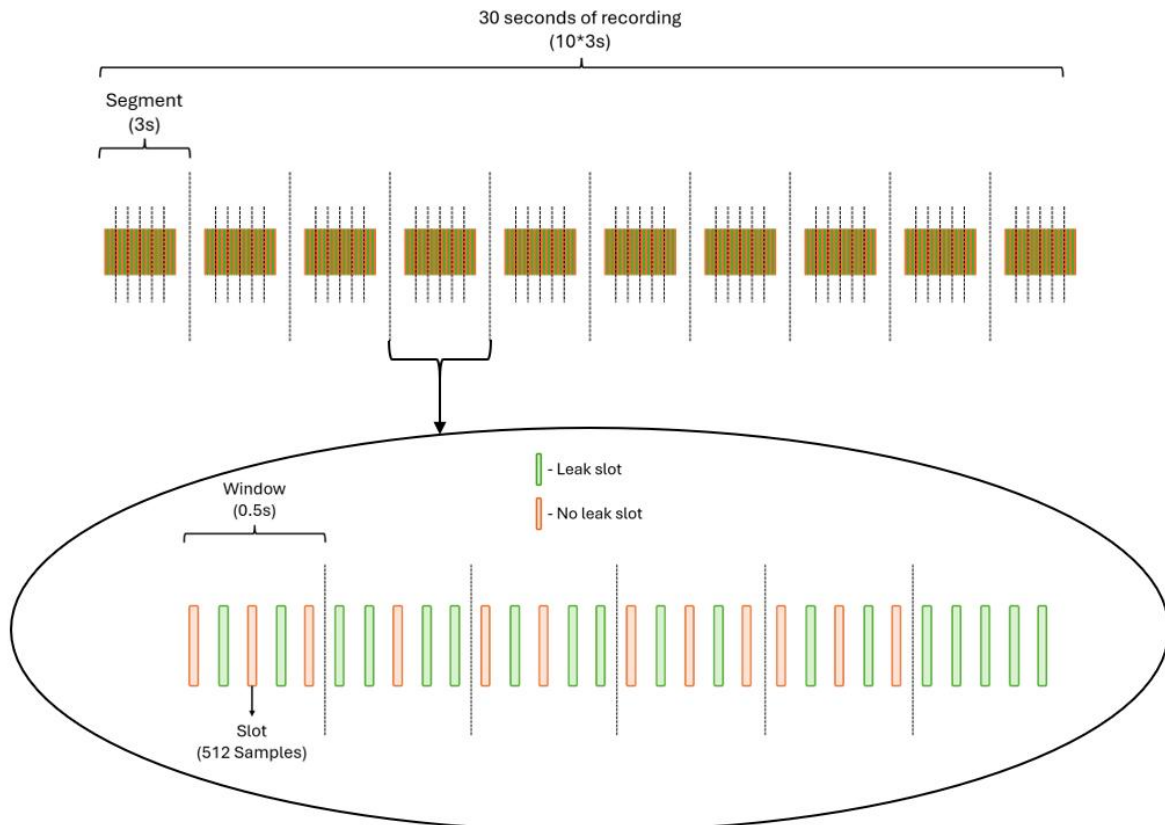


Figure 3.7.1: Segmentation of a 30s detection recording.

For clarity, the terminology used in this thesis is standardized as follows. As shown in Figure 3.7.1, one 3s recording is referred to as a segment. Each segment is divided, for decision purposes, into six 0.5s windows. Each 0.5s window consists of five slots, and each slot corresponds to one 512-sample inference input. This terminology is adopted to make the explanation easier to follow, especially when comparing slot-level model outputs with window-level and segment-level decisions.

The 30s recording duration was selected as a practical compromise between temporal robustness, response time, and energy consumption. The intention was not to increase the model input size, since each inference still uses only a 512-sample buffer, but to provide repeated opportunities to observe whether leak evidence is persistent, intermittent, or isolated. Dividing the 30s recording into ten 3s segments created ten comparable decision units that were long enough to contain meaningful temporal behavior while still remaining manageable for an embedded low-power implementation. Within each 3s segment, the signal was further organized into six windows, with each window containing five slots, giving a total of 30 slots per segment. This corresponds to an effective interval of about 0.1s per slot. This multi-level

structure was chosen so that the segment decision would depend on short-term temporal consistency rather than a single isolated prediction. Grouping several nearby slots within one window smooths fluctuations in the raw model output, while keeping six windows across the 3s segment still preserves enough temporal resolution to distinguish a sustained leak pattern from a brief spike. Using substantially fewer windows would make the decision too coarse, while using substantially more would make it more sensitive to random variation.

A buffer length of 512 samples was selected from the combined constraints of NanoEdge AI requirements, temporal resolution, and embedded resource limits. NanoEdge AI requires a fixed input size and recommends a power-of-two buffer length that remains consistent throughout training and deployment. Within those constraints, 512 was considered an appropriate compromise because it is short enough to preserve local time-domain behavior while keeping RAM usage, computational cost, and latency within a practical range for the B-U585I-IOT02A. Larger powers of two such as 1024 or 2048 would increase the duration represented by each inference buffer, but this would also reduce temporal sensitivity and increase the risk of mixing leak and background behavior within the same input. For this project, such a trade-off was not preferred, because the decision logic was intended to observe how evidence evolves over time using repeated short inferences rather than fewer but much longer ones.

The choice of 512 samples was especially suitable for the vibration channel because, at an effective sampling rate of about 6.67 kHz, each 0.1 s slot contains only about 667 samples. This means that 512 samples already use most of the data available within a short local interval. The same input length was then retained for the acoustic channel, not because the microphone required the same physical time duration, but because a common buffer size simplified the firmware structure, dataset preparation, and timing alignment between the two sensing modalities. In the final late-fusion design, the two channels do not need identical physical window durations, but it is beneficial that they follow the same segmentation framework so that both contribute comparable streams of evidence over the same 3s segment and 30s recording structure.

3.7.1 Three-Second Segment Decision Logic

For each 512-sample inference slot, the classifier outputs two probabilities, the probability of the Leak class and the probability of the Background class. These slot-level probabilities are not used directly as the final segment decision. Instead, they are aggregated within the 3s segment using two complementary rules.

The 3s segment decision was intentionally based on two complementary rules namely temporal hysteresis and strong fraction. Temporal hysteresis was introduced to favor sustained evidence across time, which is useful when leak behavior is present but only at moderate confidence. Strong fraction was introduced to preserve short-duration but highly confident leak events that might otherwise be suppressed by averaging. These two rules were expected to complement each other. A persistence-only rule would be more suitable for continuous leak behavior but could miss intermittent events, whereas a strong-event-only rule would be too sensitive to isolated spikes. Combining both allowed the logic to detect two possible leak patterns, weaker signals that continue over time, and shorter signals that stand out more clearly.

The first rule is temporal hysteresis, implemented through the 0.5s windows. Within each 0.5s window, the leak probabilities of all slot outputs that fall inside that window are averaged. A window is considered leaky when its average leak confidence is above a predefined threshold (thres.5 in Table 3.7.1 that summarizes the threshold value of the final version used.). For the acoustic channel, the segment is classified as leak by this rule only when more than half

(thres.6) of the six windows are leaky as graphically illustrated in Figure 3.7.2. This makes the acoustic channel intentionally conservative, because the microphone is generally more exposed to environmental disturbances and should not trigger a leak verdict solely because of one short spike. By contrast, the vibration channel uses a less permissive but still sensitive persistence rule where it requires at least two (thres.14) leaky windows to be classified as leak.

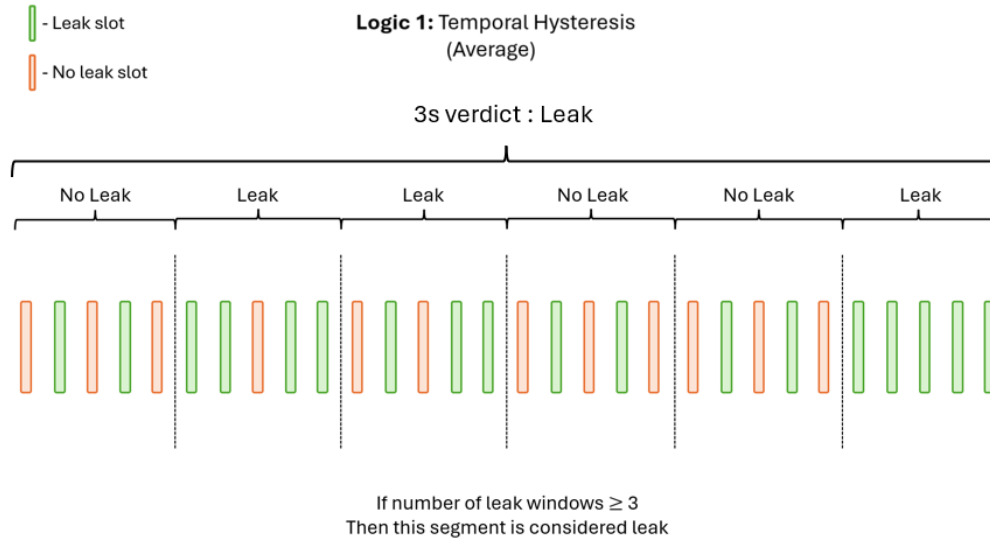


Figure 3.7.2: Graphical interpretation of the temporal hysteresis detection logic.

The second rule is strong fraction. This rule is intended to preserve short-duration but highly confident events that may be too brief to satisfy temporal hysteresis. In this rule, the system counts how many sliding inference slots within the segment having a leak confidence above a strong-confidence threshold (thres.7). If the fraction of such strong slots is sufficiently high (thres.8), the segment is still classified as leak even if the persistent-window rule is not satisfied as graphically illustrated in Figure 3.7.3. Conceptually, this protects the system from missing leak events that are sharp, sparse, or intermittent, such as weak droplets or short bursts that may not dominate the full segment but are nevertheless meaningful. In this way, temporal hysteresis and strong fraction are not competing rules, but they are designed to complement one another. Temporal hysteresis covers persistent leak behavior, while strong fraction covers short but convincing leak evidence.

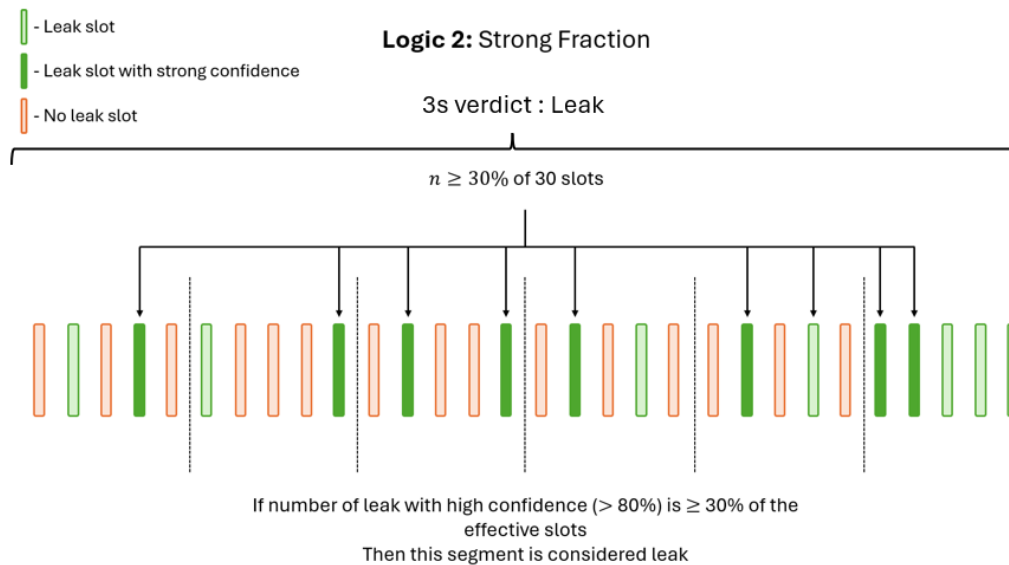


Figure 3.7.3: Graphical interpretation of the strong fraction detection logic.

A 3s segment is therefore declared as Leak when either temporal hysteresis or strong fraction is triggered. If neither rule is triggered, the segment is declared as Background. The result of each segment is stored together with its average leak confidence, average background confidence, and strong-slot fraction. These stored values are then passed to the 30s decision stage rather than being discarded. This makes the 3s logic meaningful beyond local classification where it acts as a structured filter that produces interpretable evidence for the higher-level decision layer. An example of this per-segment summary output is illustrated by the detection log in Figure 3.7.4.

```
[NEAI] 07/02/2026 00:01:50.36
[NEAI] win=0 time: 00:01:50.38 offset=0 class_id=2 (Background) probs=[0.088, 0.912]
[NEAI] win=1 time: 00:01:50.48 offset=1637 class_id=2 (Background) probs=[0.166, 0.834]
[NEAI] win=2 time: 00:01:50.58 offset=3274 class_id=1 (Leak) probs=[0.683, 0.317]
[NEAI] win=3 time: 00:01:50.69 offset=4911 class_id=2 (Background) probs=[0.027, 0.973]
[NEAI] win=4 time: 00:01:50.79 offset=6548 class_id=1 (Leak) probs=[0.986, 0.014]
[NEAI] win=5 time: 00:01:50.89 offset=8185 class_id=2 (Background) probs=[0.017, 0.983]
[NEAI] win=6 time: 00:01:50.99 offset=9822 class_id=2 (Background) probs=[0.001, 0.999]
[NEAI] win=7 time: 00:01:51.10 offset=11459 class_id=1 (Leak) probs=[0.996, 0.004]
[NEAI] win=8 time: 00:01:51.20 offset=13096 class_id=2 (Background) probs=[0.109, 0.891]
[NEAI] win=9 time: 00:01:51.30 offset=14733 class_id=2 (Background) probs=[0.105, 0.895]
[NEAI] win=10 time: 00:01:51.40 offset=16370 class_id=2 (Background) probs=[0.113, 0.887]
[NEAI] win=11 time: 00:01:51.50 offset=18007 class_id=2 (Background) probs=[0.130, 0.870]
[NEAI] win=12 time: 00:01:51.61 offset=19644 class_id=2 (Background) probs=[0.016, 0.984]
[NEAI] win=13 time: 00:01:51.71 offset=21281 class_id=2 (Background) probs=[0.012, 0.988]
[NEAI] win=14 time: 00:01:51.81 offset=22918 class_id=1 (Leak) probs=[1.000, 0.000]
[NEAI] win=15 time: 00:01:51.91 offset=24555 class_id=1 (Leak) probs=[1.000, 0.000]
[NEAI] win=16 time: 00:01:52.02 offset=26192 class_id=2 (Background) probs=[0.056, 0.944]
[NEAI] win=17 time: 00:01:52.12 offset=27829 class_id=1 (Leak) probs=[1.000, 0.000]
[NEAI] win=18 time: 00:01:52.22 offset=29466 class_id=1 (Leak) probs=[1.000, 0.000]
[NEAI] win=19 time: 00:01:52.32 offset=31103 class_id=1 (Leak) probs=[1.000, 0.000]
[NEAI] win=20 time: 00:01:52.43 offset=32740 class_id=2 (Background) probs=[0.048, 0.952]
[NEAI] win=21 time: 00:01:52.53 offset=34377 class_id=1 (Leak) probs=[0.800, 0.200]
[NEAI] win=22 time: 00:01:52.63 offset=36014 class_id=1 (Leak) probs=[0.989, 0.011]
[NEAI] win=23 time: 00:01:52.73 offset=37651 class_id=1 (Leak) probs=[0.778, 0.222]
[NEAI] win=24 time: 00:01:52.83 offset=39288 class_id=2 (Background) probs=[0.004, 0.996]
[NEAI] win=25 time: 00:01:52.94 offset=40925 class_id=1 (Leak) probs=[0.969, 0.031]
[NEAI] win=26 time: 00:01:53.04 offset=42562 class_id=1 (Leak) probs=[1.000, 0.000]
[NEAI] win=27 time: 00:01:53.14 offset=44199 class_id=1 (Leak) probs=[1.000, 0.000]
[NEAI] win=28 time: 00:01:53.24 offset=45836 class_id=2 (Background) probs=[0.398, 0.602]
[NEAI] win=29 time: 00:01:53.35 offset=47473 class_id=2 (Background) probs=[0.281, 0.719]
[NEAI] Summary: 3s acoustic chunk verdict = leak decision logic = 2 (StrongFrac)
[NEAI] Leak windows = 2/6 Leak slots = 14/30 Leak slots (high confidence) = 11/30
[NEAI] Leak Average = 0.492 Background Average = 0.508
```

Figure 3.7.4: Per-segment detection summary output.

3.7.2 Thirty-Second Decision Logic

A single device decision is not made from only one 3s segment. Instead, the firmware records and analyzes ten (thres.17) consecutive 3s segments, giving a total observation interval of 30s for each sensing channel. This second aggregation stage was introduced to strengthen robustness against short disturbances and to make the final decision depending on a pattern observed over time rather than on one local event. Importantly, the 30s logic does not ignore the 3s verdicts. The 30s logic explicitly uses the segment-level leak decisions generated by the 3s stage, thereby making the earlier decision layer functionally meaningful in the final result.

At the 30s stage, a segment is first labelled as a persistent leak segment when two conditions are simultaneously satisfied:

- 1) the 3s segment has already been classified as Leak; and
- 2) its average leak confidence is at least 0.58 (thres.18).

A segment is labelled as a strong leak segment when the 3s segment has already been classified as Leak and, in addition, either its average leak confidence is at least 0.75 (thres.21) or its strong-slot fraction is at least 0.35 (thres.22). These definitions specifically place the 3s leak verdict first. This makes the 30s stage more robust, because it builds on prior accepted leak evidence instead of bypassing it.

The 30s temporal hysteresis rule is then applied at segment level. It is triggered when either at least five (thres.19) persistent leak segments are observed out of the ten segments, or when there is a consecutive run of at least three (thres.20) persistent leak segments. The first condition captures distributed persistence across the full 30s observation period, while the second condition preserves the ability to react to a shorter but clearly sustained run of leak behavior.

The 30s strong fraction rule is triggered when at least three (thres.23) strong leak segments are present within the 30s interval. This again provides a complementary pathway whereas, even if the leak does not dominate the full 30s record, multiple highly confident segments can still provide persuasive evidence of abnormality.

The 3s and 30s stages together form a temporal decision model, in which short-duration inference outputs are progressively aggregated into higher-level decisions to improve robustness against transient errors and unstable predictions.

3.7.3 Sensor Score

After the 30s logic is evaluated separately for the acoustic and vibration channels, each channel is converted into a continuous sensor score between 0 and 1. This score is not produced from a single threshold alone; instead, it combines several pieces of evidence. Specifically, the score contains contributions from:

- 1) the ratio of 3s segments classified as leak,
- 2) the ratio of persistent leak segments,
- 3) the ratio of strong leak segments, and
- 4) the mean leak confidence among the segments already classified as leak.

Additional small bonuses are then added if the 30s temporal hysteresis rule is triggered and if the 30s strong-fraction rule is triggered. This scoring method was chosen because it preserves richer information than a purely binary decision. A channel that meets both of the 30s conditions should be regarded as more convincing than one that does not, and a channel with regular moderate evidence should not be regarded as being the same as a channel with only one isolated spike. The resulting score therefore acts as a compact but evidence-aware summary of each sensing modality.

In the current implementation, the sensor score for each channel is computed as

$$\begin{aligned} \text{Sensor Score} = & 0.25(3s \text{ leak ratio}) + 0.35(\text{persistent leak ratio}) \\ & + 0.20(\text{strong leak ratio}) \\ & + 0.20(\text{mean leak confidence on detected segments}) + \text{bonuses} \end{aligned}$$

where the temporal-hysteresis bonus is 0.05 (thres.28) and the strong-fraction bonus is also 0.05 (thres.29) when the corresponding rule is triggered. The stronger weighting on the persistent leak ratio reflects the design intention that sustained leak evidence should be more influential than isolated evidence.

3.7.4 Late Fusion and Final Verdict

The final device decision is obtained by late fusion of the acoustic and vibration sensor scores. In the current implementation (final version, V2), the acoustic score is assigned a weight of 0.80 (thres.30), while the vibration score is assigned a weight of 0.20 (thres.31). The heavier acoustic weighting reflects the stronger standalone performance of the acoustic model observed during development, while the vibration channel is retained as a complementary cue that can strengthen the final decision in cases where structural vibration remains informative. Additional fusion bonuses are added when both channels simultaneously trigger 30s temporal

hysteresis (thres.32) and when both channels simultaneously trigger 30s strong fraction (thres.33). This rewards cross-modal agreement without forcing both channels to be identical in strength.

The final fusion score is therefore,

$$\text{Fusion Score} = 0.80(\text{Acoustic Score}) + 0.20(\text{Vibration Score}) + \text{agreement bonuses.}$$

A final verdict of Leak is issued when the fusion score is greater than or equal to 0.58 (thres.34). Otherwise, the final verdict is Background. The firmware also reports the overall average leak confidence and average background confidence across all processed slots, which provide an interpretable confidence summary for logging and LoRa P2P transmission. The example late-fusion detection output in Figure 3.7.5 demonstrates this final stage, showing the per-sensor scores, fusion weights, fusion score, and final verdict.

```
[FUSE] ===== FUSE =====
[AUD ] Chunk average leak confidence = {0.060, 0.062, 0.043, 0.216, 0.347, 0.058, 0.109, 0.323, 0.114, 0.144}
[AUD ] 3s leak verdicts = {0, 0, 0, 0, 1, 0, 0, 0, 0, 0}
[AUD ] 3s leak chunks = 1/10 persistent leak chunks = 0/10 strong leak chunks = 0/10
[AUD ] longest consecutive persistent leak run = 0
[AUD ] average leak confidence = 0.148 average background confidence = 0.852
[AUD ] temporal hysteresis = not triggered strong fraction = not triggered sensor score = 0.094
[VIB ] Chunk average leak confidence = {0.996, 0.994, 0.995, 0.974, 0.810, 0.997, 0.995, 0.952, 0.997, 0.997}
[VIB ] 3s leak verdicts = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1}
[VIB ] 3s leak chunks = 10/10 persistent leak chunks = 10/10 strong leak chunks = 10/10
[VIB ] longest consecutive persistent leak run = 10
[VIB ] average leak confidence = 0.971 average background confidence = 0.029
[VIB ] temporal hysteresis = triggered strong fraction = triggered sensor score = 1.000
[FUSE] Audio score = 0.094 Vibration score = 1.000
[FUSE] Audio weight = 0.80 Vibration weight = 0.20
[FUSE] Fusion score = 0.276 < 0.58 (threshold) -> Final decision = BACKGROUND
[FUSE] Leak average confidence = 0.559 Background average confidence = 0.441 Window count = 600
```

Figure 3.7.5: 30s late fusion detection summary output.

Table 3.7.1 summarizes the thresholds used in the implementation for first (V1) and second (V2) version. These values are firmware-defined and correspond to the deployed decision logic. The values in Table 3.7.1 are further justified in Results and Discussion Section 4.1.

Table 3.7.1: Summary of the thresholds used in the implementation of decision logic.

Stage	thres.	Parameter	Value Type	Value (V1)	Value (V2)	Meaning
3s acoustic segment	1	Segment duration	Duration	3s	3s	One acoustic decision segment
	2	Sliding inference slots per segment	Value	30	30	Number of 512-sample slot inferences
	3	Decision windows per segment	Value	6	6	Six 0.5s windows per segment
	4	Slot size	Value	512 samples	512 samples	Raw samples passed into one inference
	5	Window leak threshold	Threshold	0.60	0.60	A 0.5s window is leaky if its average leak confidence $\geq$ 0.60
	6	Temporal hysteresis threshold	Threshold	> 50% of windows	> 50% of windows	Acoustic leak if more than half of the six windows are leaky

	7	Strong slot threshold	Threshold	0.80	0.80	A slot is strong if leak confidence ≥ 0.80
	8	Strong fraction threshold	Threshold	0.30	0.30	Acoustic leak if strong slots / all slots ≥ 0.30
3s vibration segment	9	Segment duration	Duration	3s	3s	One vibration decision segment
	10	Sliding inference slots per segment	Value	30	30	Number of 512-sample slot inferences
	11	Decision windows per segment	Value	6	6	Six 0.5s windows per segment
	12	Slot size	Value	512 samples	512 samples	Raw samples passed into one inference
	13	Window leak threshold	Threshold	0.60	0.60	A 0.5s window is leaky if its average leak confidence ≥ 0.60
	14	Temporal hysteresis threshold	Threshold	at least 1 leaky window	at least 2 leaky windows	Vibration leak if at least n windows are leaky ($V1: \geq 1$, $V2: \geq 2$)
	15	Strong slot threshold	Threshold	0.80	0.80	A slot is strong if leak confidence ≥ 0.80
	16	Strong fraction threshold	Threshold	0.30	0.35	Vibration leak if strong slots / all slots ≥ 0.35
30s sensor logic	17	Number of segments	Value	10	10	Ten consecutive 3s segments form one 30s decision interval
	18	Persistent segment confidence threshold	Threshold	Not used in V1	0.58	In V2, Segment must already be leak and have average leak confidence ≥ 0.58
	19	Persistent segment count threshold	Threshold	Segment Average ≥ 0.60 and count > 5	5	V1 temporal rule based directly on segment average leak confidence. In V2, temporal hysteresis if at least 5 persistent segments occur
	20	Consecutive persistent run threshold	Threshold	Not used in V1	3	In V2, temporal hysteresis also triggers if 3 persistent segments occur consecutively

	21	Strong segment confidence threshold	Threshold	Segment Average ≥ 0.80	0.75	V1 strong rule based directly on segment average leak confidence. In V2, strong segment if already leak and average leak confidence ≥ 0.75
	22	Strong segment slot-fraction threshold	Threshold	Not used in V1	0.35	In V2, strong segment also if already leak and strong-slot fraction ≥ 0.35
	23	Strong segment count threshold	Threshold	3	3	Strong fraction triggers if at least 3 strong segments occur
Sensor score	24	3s leak-ratio weight	Weight	Not used in V1	0.25	Contribution from number of 3s leak segments
	25	Persistent-ratio weight	Weight	Not used in V1	0.35	Contribution from number of persistent leak segments
	26	Strong-ratio weight	Weight	Conditional in V1	0.20	V1 uses different score weighting depending on triggered logic. In V2, the value is the contribution from number of strong leak segments
	27	Mean leak-confidence weight	Weight	Conditional in V1	0.20	V1 uses different score weighting depending on triggered logic. In V2, the value is the contribution from average leak confidence on detected segments
	28	Temporal bonus	Score	Not used in V1	0.05	Added if 30s temporal hysteresis triggers
	29	Strong bonus	Score	Not used in V1	0.05	Added if 30s strong fraction triggers
Late fusion	30	Acoustic weight	Weight	0.70	0.80	Weight of acoustic sensor score
	31	Vibration weight	Weight	0.30	0.20	Weight of vibration sensor score
	32	Both-sensors temporal bonus	Score	Not used in V1	0.05	Added when both channels trigger temporal hysteresis
	33	Both-sensors strong bonus	Score	Not used in V1	0.05	Added when both channels trigger strong fraction
	34	Final fusion threshold	Threshold	0.60	0.58	Final verdict is Leak if fusion score ≥ 0.58

As shown in Table 3.7.1, the first version (V1) operates slightly different than the second version (V2). In V1, the 30s sensor logic operated directly on the average leak confidence of each 3s segment, rather than on the persistent-segment formulation introduced in V2. In V1, the 30s sensor score was not formed using the V2 ratio-based weighting structure. Instead, it was computed conditionally from the overall average leak confidence and the average confidence of strong segments, using different weight combinations depending on whether the temporal hysteresis, strong fraction, or both were triggered. This will be further justified in Results and Discussion Section 4.1. Figure 3.7.6 illustrates the flow chart of the prototype's periodically detection with V2 decision logic implemented.

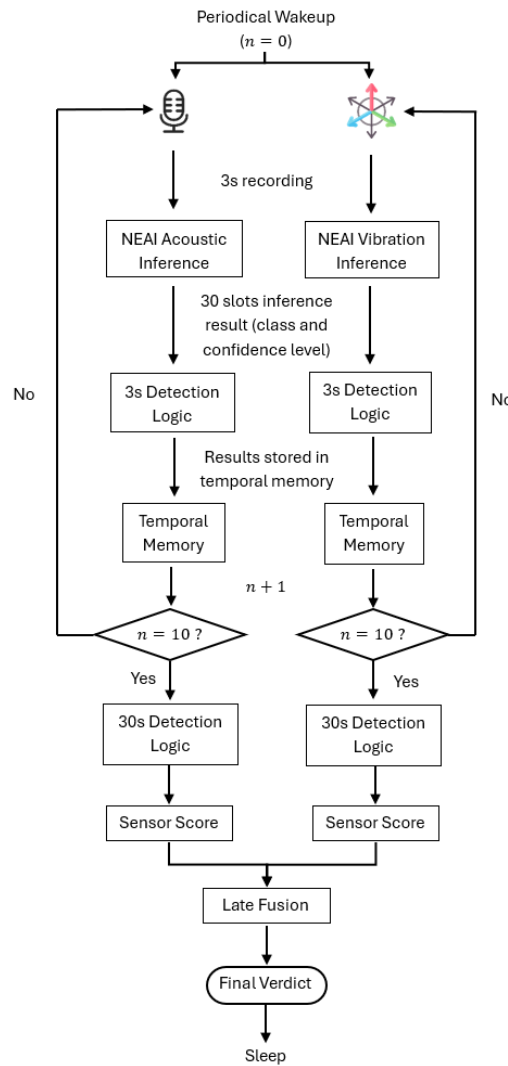


Figure 3.7.6: The device periodically detection flow chart with V2 decision logic implemented.

3.8 Testing and Validation

To evaluate whether the proposed system meets its functional and low-power objectives, testing and validation were carried out from two perspectives. First, the leak detection performance was assessed using labelled leak and background trials to quantify classification accuracy and related metrics. Second, measurements were performed to evaluate the current drawn by both the MCU alone and the entire board under different operating states. Together, these tests provide evidence on whether the system is capable of reliable embedded leak detection while also demonstrating the extent to which low-power operation is achieved in the current implementation of the prototype.

3.8.1 Measuring AI Detection Model Accuracy

The accuracy of the proposed AI leak detection system was evaluated by running inference directly on the embedded prototype during operation on the simulation rig, rather than by exporting recorded files and testing them offline on a PC as shown in Figure 3.8.1.

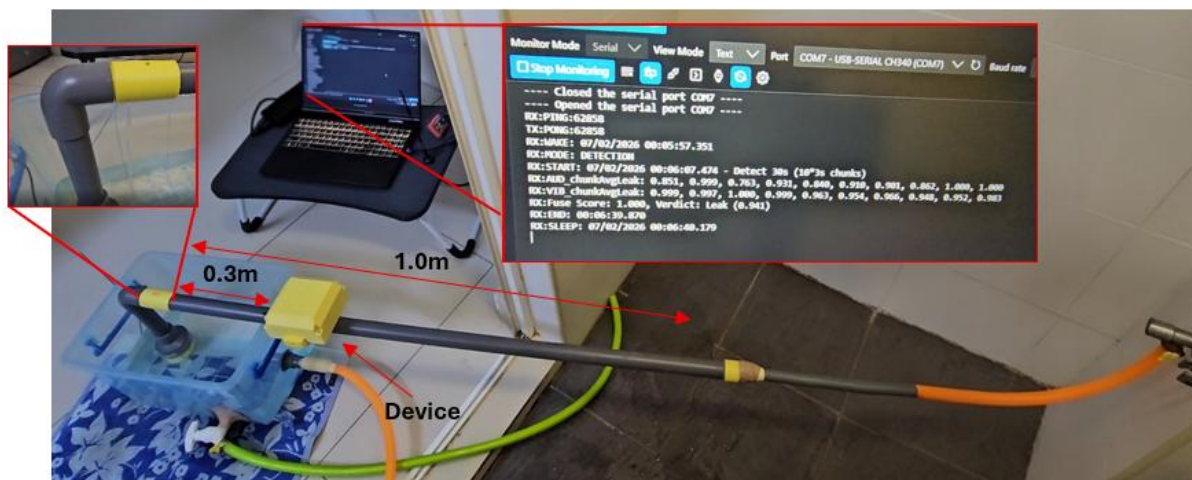


Figure 3.8.1: AI model validating on simulation rig.

This approach was selected because the final system is intended to operate as a standalone embedded device, where the combined effects of sensing, buffering, embedded inference, decision logic, and wireless communication all influence the actual detection outcome. Evaluating the model in this way therefore provides a more realistic measurement of deployed performance than testing only pre-recorded signals in an offline software environment. Similar studies in water-pipeline monitoring have also validated embedded or IoT-based leak detection systems using a physical prototype and a real or representative testbed, rather than relying only on offline file-based analysis. For example, Boujelben et al. reported a lightweight deep-learning leak detection system evaluated on a real small-scale pipeline testbed using an IoT device prototype, with the embedded model deployed on the sensing node itself [114].

In this project, the detection pipeline produces three levels of verdict, namely the raw inference result, the 3-second verdict, and the 30-second verdict. The raw inference result corresponds to the immediate class output produced by the NanoEdge AI model for each inference slot. The 3-second verdict is the result after the slot-level outputs are processed by the first stage of the decision logic described in Section 3.7.1. The 30-second verdict is the final aggregated result after temporal decision logic and late fusion have been applied over the full observation interval, as described in Sections 3.7.2 to 3.7.4. Recording all three verdict types is important because it allows the contribution of the decision logic to be assessed explicitly. In other words, the methodology does not only measure whether the AI model can classify leak and background conditions, but also whether the added decision structure improves the practical detection accuracy of the full embedded system.

To make this evaluation manageable during repeated rig experiments, the STM32 firmware was modified to transmit every detection result in a compact comma-separated format through the LoRa P2P link. Each output line was structured as:

result type, class name, confidence level

For example, a raw slot-level result, a 3-second result, and a 30-second result can all be transmitted using the same general format while being distinguished by the first field. An example of the transmitted output is shown in Figure 3.8.1.

```

RX:Aud, Background, 0.990
RX:Aud, Leak, 0.990
RX:Aud, Leak, 0.962
RX:Aud, Background, 0.998
RX:Aud, Background, 0.998
RX:Aud, Background, 0.997
RX:Aud, Background, 0.851
RX:Aud, Background, 0.994
RX:Aud, Background, 0.578
RX:Aud, Background, 0.995
RX:Aud, Background, 0.879
RX:Aud, Background, 0.864
RX:Aud, Leak, 0.972
RX:Aud, Background, 0.989
RX:Aud, Leak, 0.521
RX:Aud, Leak, 0.835
RX:Aud, Leak, 1.000
RX:Aud, Background, 0.816
RX:Aud, Background, 0.996
RX:Aud, Background, 0.756
RX:3s-Aud, background, 0.651
RX:Vib, Background, 0.591
RX:Vib, Leak, 0.995
RX:Vib, Leak, 0.755
RX:Vib, Background, 0.586

```

Figure 3.8.2: Example LoRa P2P payload format for AI detection result extraction.

By separating the three fields with commas, the text captured from the serial monitor of the base LoRa P2P module can be copied directly into a text file or spreadsheet software without manual rewriting. This reduces transcription error and makes it possible to sort, filter, and group the results efficiently according to verdict type and predicted class. Compared with manual logging, this method is faster, more repeatable, and better suited for repeated experimental trials.

After collecting, the transmitted results were imported into Microsoft Excel for organization and counting. For each verdict type, the predicted class was compared against the known ground-truth condition of the experiment, namely Leak or Background. Based on this comparison, the outputs were grouped into the four standard entries of a confusion matrix:

- 1) True Positive (TP): a leak condition correctly classified as leak
- 2) True Negative (TN): a background condition correctly classified as background
- 3) False Positive (FP): a background condition incorrectly classified as leak
- 4) False Negative (FN): a leak condition incorrectly classified as background

Since the proposed device is intended for practical monitoring, all four outcomes are important: missed leaks reduce safety and usefulness, while false alarms reduce trust in the system and may increase unnecessary inspection effort. Once the confusion matrix was populated, several standard classification metrics were calculated in Excel using formula-based cell references. These metrics provide a more complete view of performance than accuracy alone. The metrics are:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (Eq\ 3.8.1)$$

Accuracy gives the proportion of all classifications that were correct. It provides an overall summary of performance, but by itself it may be misleading if one class is easier to predict than the other [115].

$$Precision = \frac{TP}{TP + FP} \quad (Eq\ 3.8.2)$$

Precision measures how often a predicted leak is truly a leak [115]. In practical terms, it indicates how trustworthy a positive alarm is, and a high precision means the system generates fewer false alarms [115].

$$Recall = \frac{TP}{TP + FN} \quad (Eq\ 3.8.3)$$

Recall, also referred to as sensitivity or true positive rate, measures how many actual leaks are successfully detected [115]. In this project, recall is especially important because missed leaks may reduce the usefulness of the device in real deployment.

$$Specificity = \frac{TN}{TN + FP} \quad (Eq\ 3.8.4)$$

Specificity measures how well the system correctly identifies non-leak conditions [115]. This metric is useful for evaluating resistance to false alarms during background operation. Since the false positive rate (FPR) is defined as $\frac{FP}{FP+TN}$, specificity can equivalently be understood as $1 - FPR$.

$$F1\ Score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (Eq\ 3.8.5)$$

The F1 score combines precision and recall into a single value using their harmonic mean [115]. It is useful when both missed detections and false alarms must be considered together, rather than focusing on only one type of error.

$$Balanced\ Accuracy = \frac{1}{2} (Recall + Specificity) \quad (Eq\ 3.8.6)$$

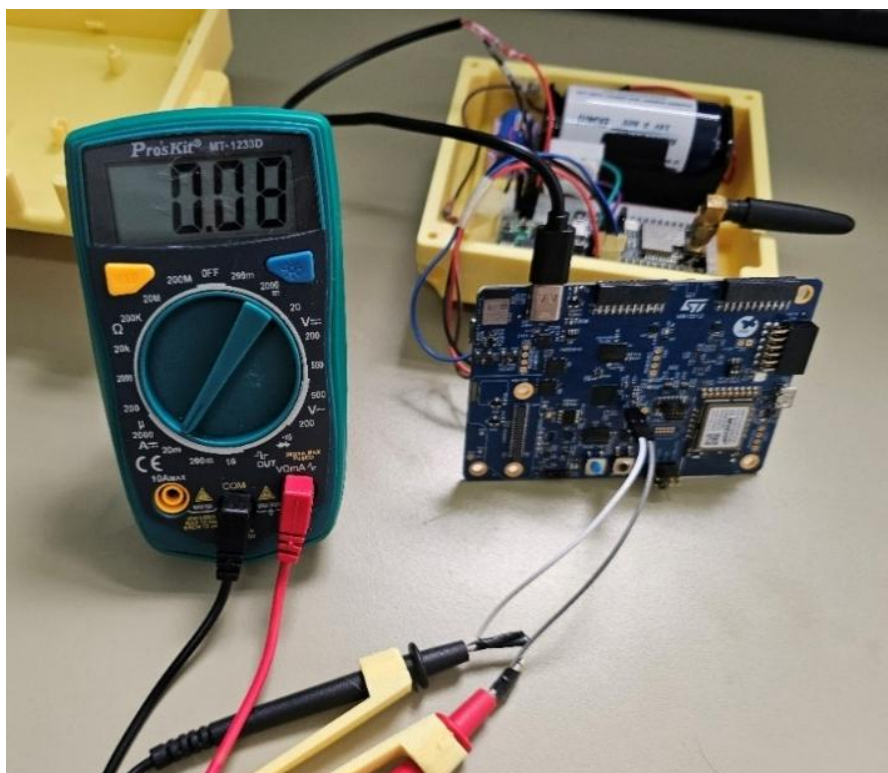
Balanced accuracy is the average of sensitivity and specificity [116]. This is particularly useful when comparing performance in cases where the numbers of leak and background samples are not perfectly balanced, because it gives equal importance to both classes.

Using multiple performance metrics is important in this project because the device is intended for real-world monitoring, where a high overall accuracy alone does not necessarily imply good practical performance. For example, a model could achieve a seemingly good accuracy if it predicts the majority class often, while still performing poorly at detecting true leaks. By reporting precision, recall, specificity, F1 score, and balanced accuracy alongside accuracy, the evaluation more clearly reflects both the leak-detection capability and the false-alarm behavior of the system.

To improve fairness and repeatability, the same labelled test conditions were used across all compared verdict types. In other words, the raw inference results, 3-second results, and 30-second results were extracted from the same experimental runs so that differences in

performance could be attributed to the decision logic rather than to changes in the test conditions. This is important because the purpose of the methodology is not only to measure final system accuracy, but also to determine whether the intermediate and final decision layers contribute meaningful improvement over direct slot-level inference.

3.8.2 Measuring Power Consumption



From Figure 3.8.2, a digital multimeter was connected in series across JP3 on the B-U585I-IOT02A. This jumper location is suitable because it lies on the supply path associated with the target MCU domain, allowing the current drawn by the MCU to be observed without directly measuring the total current consumed by the rest of the board. Figure 3.8.3 shows the power tree diagram extracted from the datasheet that shows the position of the jumper JP3 in relation to the MCU supply path [117].

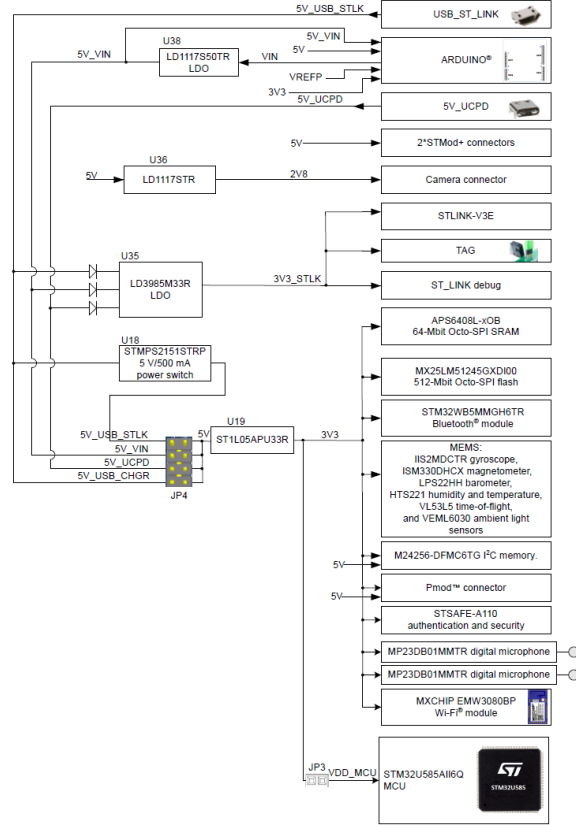


Figure 3.8.4: Power-tree diagram of the B-U585I-IOT02A showing JP3 in the MCU supply path [117].

By opening the jumper at JP3 shown in Figure 3.8.3, and inserting the ammeter in series, the current flowing into the MCU section can be measured while the firmware transitions the device into different low-power modes, namely STOP2, STOP3, and SHUTDOWN. This method is appropriate for characterizing the low-power behavior of the controller itself, which is necessary because datasheet-level low-power claims apply primarily to the MCU rather than to the full development platform [15]. The MCU-only current measurement results are presented and discussed in section 4.4.

Although MCU-only measurement is useful for verifying low-power firmware operation, it does not represent the actual current required by the full prototype in deployment. Therefore, a second measurement was performed on the entire board-level system. In this setup, the multimeter was connected in series with the output voltage terminal (V_{out}) of the buck boost converter supplying the development board, as shown in Figure 3.8.4.

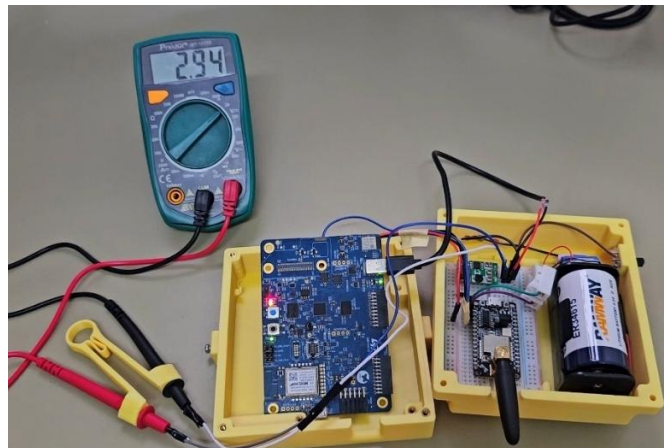


Figure 3.8.5: Setup for full board current measurement.

The configuration shown in Figure 3.8.4 is able to capture the current drawn by the entire B-U585I-IOT02A board under a given test condition. Unlike the JP3 method, this measurement reflects the practical current that must be supplied to keep the prototype operating as an assembled device. The full board current measurement results are presented and discussed in section 4.4.

4. Results and Discussion

4.1 AI Model Accuracy

This section presents the classification accuracy achieved by each version of the embedded AI leak detection model under different decision logic schemes. In total, three AI model versions and two decision logic versions were evaluated. For clarity, each model configuration is labeled as V_n - DL V_m , where V_n denotes the model version, with later versions trained using larger datasets, and DL V_m denotes the decision logic version. The decision logic was updated once, from DL V1 to DL V2, by adjusting the parameters and thresholds listed in Table 3.7.1. Accordingly, V1 - DL V1 refers to the first model trained on the smallest dataset with the first decision logic, whereas V1 - DL V2 refers to the same model and dataset evaluated using the updated logic. Similarly, V2 - DL V2 represents the larger model trained with an expanded dataset and evaluated using the updated logic, while V3 - DL V2 represents the largest model version with the same updated logic. The number of training samples used for each model version is summarized in Table 4.1.1.

Table 4.1.1: Dataset size of different model version (512 samples per buffer).

Model version	Dataset size			
	Acoustic - Buffers (Samples)		Vibration – Buffers (Samples)	
	Leak	Background	Leak	Background
V1	24,628 (12,609,536)	12,168 (6,230,016)	10112 (5,177,344)	4,992 (2,555,904)
V2	49,920 (25,559,040)	39,936 (20,447,232)	34,304 (17,563,648)	20,224 (10,354,688)
V3	71,136 (36,421,632)	49,296 (25,239,552)	56,320 (28,835,840)	30,080 (15,400,960)

As shown in Table 4.1.1, the dataset size was increased progressively from V1 to V2, and then to V3, for both the acoustic and vibration channels, rather than maintaining a fixed training volume across all versions. This progressive increase was intended to improve the robustness of the trained models by exposing them to a wider range of signal patterns and reducing the likelihood that the classifier would learn only a narrow subset of leak conditions. As discussed earlier in Section 3.5.1, improving model performance depends not only on increasing the number of samples, but also on increasing the diversity of training conditions. In this sense, the progression from V1 to V3 should be understood as a deliberate attempt to improve generalization by expanding both the quantity of data and the variety of signal characteristics presented during training.

A dataset that is too small, such as V1, may not provide sufficient variation for the model to learn stable representations of leak and background conditions, which can reduce performance when the embedded system is tested under unseen scenarios. Conversely, although a larger

dataset is generally expected to improve performance, a much larger dataset such as V3 may still introduce difficulties if the additional data are not sufficiently consistent, are more challenging, or are not well matched to the model capacity and decision logic. Therefore, the impact of increasing dataset size cannot be assessed purely from sample count alone, and its effect must be interpreted together with the later accuracy results presented in this section. The composition of the collected dataset for each model version is further summarized in Table 4.1.2.

Table 4.1.2: Type of leak/background recorded in the dataset of each model version.

Type of leak/background collected as dataset		Model version
Hole (mm)	Distance (m)	
2.5	0.3	V1, V2, V3
	0.5	V2, V3
	1.0	V1, V2, V3
5.0	0.3	V1, V2, V3
	0.5	V2, V3
	1.0	V1, V2, V3
10.0	0.3	V3
	1.0	V3
Baseline	0.3	V1, V2, V3
	0.5	V2, V3
	1.0	V1, V2, V3
2.5 (with noise)	0.3	V2, V3
5.0 (with noise)	0.3	V3
10.0 (with noise)	0.3	V3
Baseline (with noise)	0.3	V2, V3

As shown in Table 4.1.2, the changes between model versions reflect not only an increase in dataset size, but also a progressive increase in dataset diversity. Across the three model versions, the data collected were expanded to include different leak hole sizes, different distances between the prototype and the leak location, and both quiet and noisy operating conditions. In addition to the baseline background condition, both the leak and background datasets included recordings collected with and without added environmental disturbance, while the leak dataset further covered multiple hole diameters. The noisy cases were introduced by playing factory sound obtained online through a speaker placed near the detection prototype during recording, thereby exposing the acoustic channel to more realistic environmental interference instead of only controlled laboratory silence.

Compared with V1, which was trained on a more limited combination of leak sizes, distances, and noise conditions, V2 introduced additional distances together with noisy leak and baseline recordings. V3 further extended the dataset to include larger leak openings and more challenging noise-influenced scenarios. This indicates that the model versions differ not only in total data volume, but also in the breadth of physical conditions represented during training. Consequently, the progression from V1 to V3 should be interpreted as an expansion in both dataset quantity and scenario coverage, which is important when assessing why certain model versions perform better than others in the subsequent accuracy comparison. For reproducibility and future reference, the dataset used in this study has been uploaded to Kaggle and can be accessed through the following [link](#).

To give a clearer view of the signal content used for training, representative baseline and leak buffers uploaded to NanoEdge AI Studio are shown in Figure 4.1.1 for the vibration channel and Figure 4.1.2 for the acoustic channel. These plots were extracted directly from the Studio and are included to illustrate how the two sensing modalities differ visually before classification. In each figure, the time-series plot shows the variation of signal amplitude over the buffer, the FFT plot shows the distribution of signal energy across frequency components, and the browser plot provides an overlaid view of multiple buffers to help visualize the spread and repeatability of the dataset. Together, these views help explain why the leak and background classes are distinguishable and also show the differences in character between the vibration and acoustic signals.

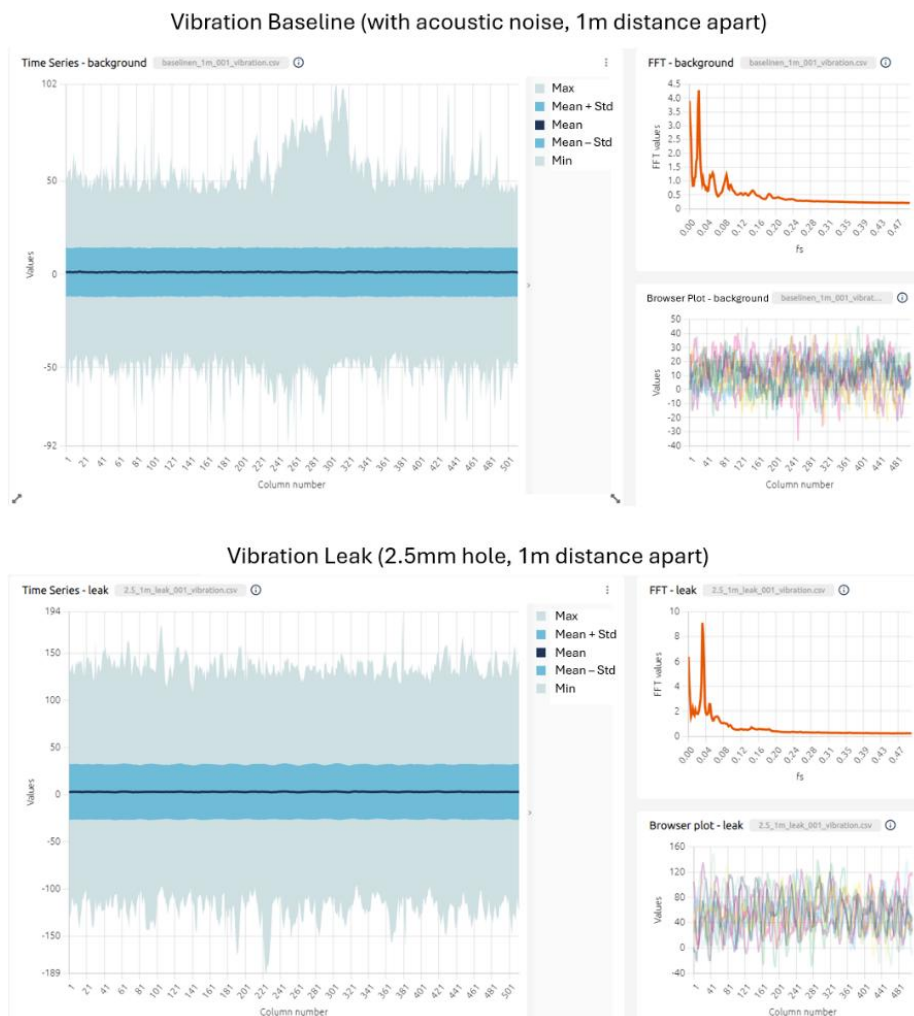


Figure 4.1.1: Vibration signal buffers uploaded to NEAI Studio for baseline and leak conditions, shown in time-series, FFT, and browser-plot views.

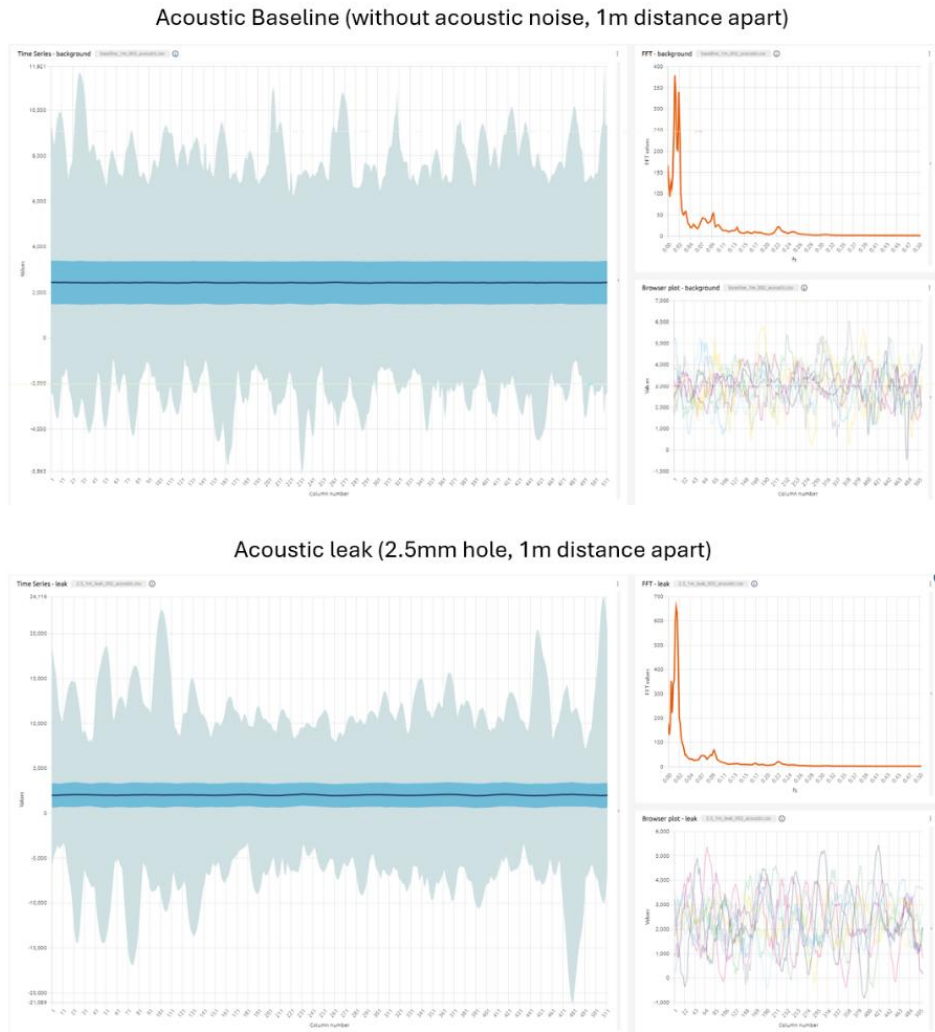


Figure 4.1.2: Acoustic signal buffers uploaded to NanoEdge AI Studio for baseline and leak conditions, shown in time-series, FFT, and browser-plot views.

From Figure 4.1.1, the vibration leak buffers show a visibly larger signal range than the baseline case in the time-series view, indicating stronger mechanical excitation when leakage is present. A similar difference is observed in the FFT view, where the leak case shows higher spectral magnitude than the baseline case, suggesting that the leak condition introduces stronger vibration energy across the frequency range. As illustrated in Figure 4.1.2, a similar overall contrast is observed in the acoustic channel. Comparing both figures, the acoustic signals show a significantly higher amplitudes compared to the vibration signals. This observation is anticipated, as the microphone directly captures both airborne and structure-borne leak noise, whereas the accelerometer records a more attenuated structural vibration signals transmitted through the pipe. Overall, these plots support the expectation that leak and background classes are separable in both sensing modalities, while also showing that the acoustic and vibration channels provide different forms of physical information. Figure 4.1.3 presents the detection accuracy by sensor modality (audio and vibration) and by decision stage (raw inference, 3s decision logic, and 30s fused logic).

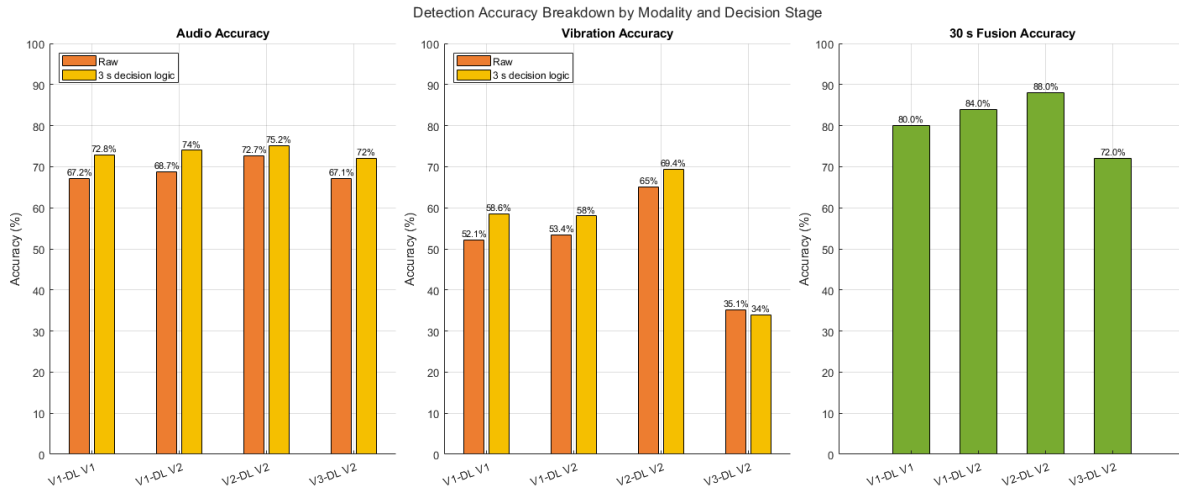


Figure 4.1.3: Detection accuracy breakdown by modality and decision stage.

Figure 4.1.3 compares the detection accuracy of the different model and logic configurations by separating the results into audio accuracy, vibration accuracy, and 30s fusion accuracy. For the audio and vibration plots, each model configuration on the horizontal axis is represented by two bars, the raw inference result and the corresponding 3s decision logic result. A consistent pattern can be observed in the audio channel, where the 3s decision logic improves the accuracy over the raw model output for all evaluated configurations. For example, in V1 - DL V1, the audio accuracy increases from 67.2% in the raw inference stage to 72.8% after the 3s decision logic is applied. Similar improvements are also seen in V1 - DL V2, V2 - DL V2, and V3 - DL V2, although the magnitude of improvement varies. This indicates that the short-term temporal decision stage is able to refine the raw model output by reducing isolated misclassifications and producing a more stable intermediate decision.

A similar trend is also evident in the vibration channel, although the degree of improvement differs more clearly between versions. In V1 - DL V1, the vibration accuracy increases from 52.1% to 58.6%, while in V1 - DL V2 it increases from 53.4% to 58.0%. The largest vibration performance is achieved by V2 - DL V2, where the accuracy improves from 65.0% in the raw stage to 69.4% after the 3s decision logic. These results demonstrate that 3s logic has a positive effect not only on the acoustic channel but also on the vibration channel, which would otherwise result in unstable initial predictions due to signal fluctuations and transient noise. However, the vibration results also show that the improvement is not always large enough to compensate for weak raw model performance, as seen in V3 - DL V2, where both the raw and 3s accuracies remain low.

The rightmost plot in Figure 4.1.3 shows the 30s fusion accuracy, which represents the final decision after longer temporal aggregation and multimodal fusion have been applied. In all cases, the 30s fusion stage produces a further increase in accuracy compared with the preceding raw and 3s stages, confirming that the final decision logic strengthens the overall detection reliability. The highest final performance is achieved by V2 - DL V2 with 88.0%, followed by V1 - DL V2 with 84.0% and V1 - DL V1 with 80.0%. Although V3 - DL V2 also benefits from the 30s fusion stage, its final accuracy remains lower at 72.0%, indicating that temporal fusion can improve performance but cannot fully compensate for weaker intermediate model outputs. Overall, Figure 4.1.3 demonstrates that the proposed decision hierarchy, from raw inference to 3s decision logic and finally to 30s fusion, consistently improves detection accuracy, thereby supporting the effectiveness of the embedded decision-logic design. Figure 4.1.4 shows the overall average detection accuracy trend across model and detection logic versions.

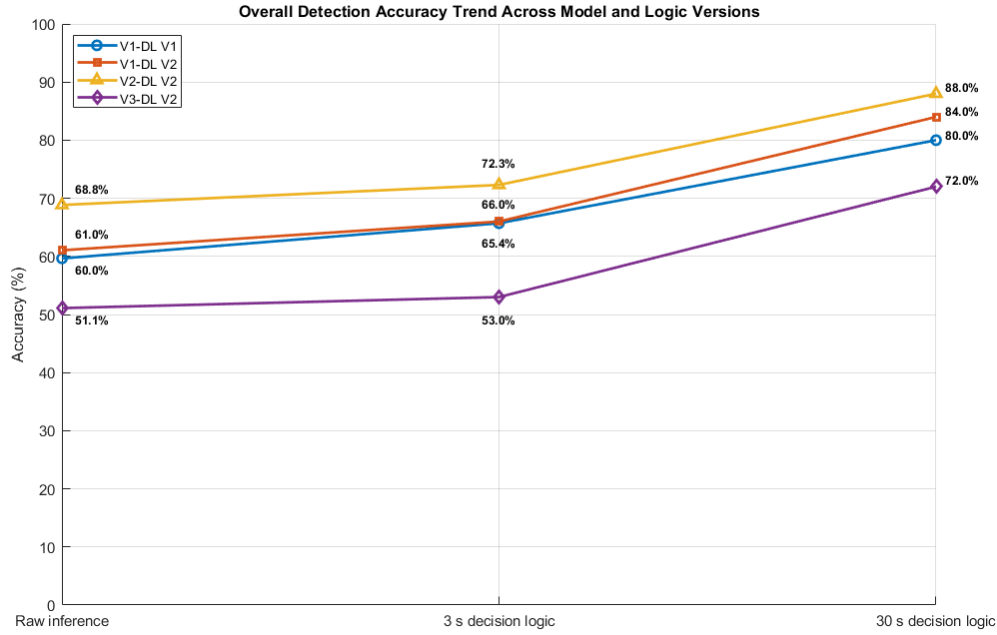


Figure 4.1.4: Overall detection accuracy trend across model and logic versions.

As shown in Figure 4.1.4, all model configurations follow the same general trend, where the overall detection accuracy increases progressively from raw inference to 3s decision logic, and then rises further at the 30s decision logic stage. This confirms that the layered decision structure contributes positively to the final detection outcome, since each additional processing stage refine the result beyond the previous one. Among all evaluated configurations, V2 - DL V2 achieves the highest performance throughout the final stage, reaching 88.0% overall accuracy, which is higher than V1 - DL V1 (80.0%), V1 - DL V2 (84.0%), and V3 - DL V2 (72.0%). For this reason, V2 - DL V2 was selected as the final implemented version for the proposed embedded leak detection system.

The comparison between V1 - DL V1 and V1 - DL V2 shows that the performance improvement in the second version of the decision logic was not caused merely by threshold adjustment, but by a more substantial redesign of the decision structure itself. In the first version, the 3s acoustic and vibration decision rules were already present, but the later 30s stage did not explicitly make use of the 3s leak verdict as a structured intermediate decision. Instead, the V1 30s logic operated mainly on the average leak confidence of each 3s segment and then applied simple count-based rules and a conditional score accumulation scheme before late fusion. The final score in V1 was therefore driven mainly by segment-level average confidence and strong-segment confidence, with a fixed acoustic-to-vibration weighting of 0.70 and 0.30, but without the explicit persistent-segment formulation, leak-segment ratio term, persistent-ratio term, or agreement bonuses introduced in V2.

The threshold values used in V1 were not derived from a closed-form optimization but were selected as interpretable engineering starting points based on the intended behavior of the decision logic. A confidence level around 0.60 was treated as moderate leak evidence, while a level around 0.80 was treated as strong evidence. The associated count thresholds were chosen so that a final leak decision would not be dominated by a single isolated segment or by one short high-confidence event. In this way, the V1 design aimed to reflect a simple and conservative principle, where the system should respond either to repeated leak-like behavior over time or to a sufficient number of strongly leak-like events. This provided a reasonable first operating point before the later embedded validation results were available.

In contrast, V2 redesigned the 30s stage so that it explicitly reused the 3s segment verdict as meaningful intermediate evidence rather than depending mainly on average segment confidence alone. This allowed the final decision to be based not only on confidence magnitude, but also on whether leak evidence was sustained over time, whether strong segments occurred repeatedly, and whether both sensing modalities agreed. The threshold and weighting values in V2 were not obtained from a closed-form analytical optimization, but from a systematic empirical tuning process within the embedded validation workflow, where raw slot outputs, 3s summaries, 30s final decisions, confusion matrices, and performance metrics were compared under the same simulation-rig conditions. In this way, the final V2 values were selected to improve the trade-off between missed leaks and false alarms while preserving an interpretable and lightweight decision structure suitable for embedded implementation. A further revision beyond V2 was not pursued because V2 had already addressed the main structural weakness identified in V1, and increasing the complexity of the logic further without a clearly observed new failure mode would have reduced interpretability without strong justification.

The comparison also suggests that V2 detection model benefits from the expanded and more diverse training data relative to V1 detection model, leading to a stronger model without the instability observed in the largest dataset version. In contrast, although V3 detection model was trained using the largest dataset, its lower accuracy indicates that increasing dataset size alone does not guarantee better performance, especially when the added data may introduce greater complexity or mismatch with the model capacity and decision logic. Therefore, the results in Figure 4.1.4 indicate that V2 - DL V2 provides the most suitable balance between training dataset scale, decision-logic refinement, and final detection accuracy. Figure 4.1.5 shows the normalized confusion matrices for the selected detection model V2 - DL V2 at three decision stages, raw NEAI output, 3s detection logic, and final 30s fusion logic.

Audio NEAI - Raw (Normalized)	Leak	Background	Audio NEAI - Detection Logic 3s (Normalized)	Leak	Background
Leak	0.80	0.20	Leak	0.93	0.07
Background	0.39	0.61	Background	0.52	0.48
Vibration NEAI - Raw (Normalized)	Leak	Background	Vibration NEAI - Detection Logic 3s (Normalized)	Leak	Background
Leak	0.78	0.22	Leak	0.91	0.09
Background	0.55	0.45	Background	0.65	0.35
Fusion NEAI - Detection Logic 30s (Normalized)	Leak	Background			
Leak	1.00	0.00			
Background	0.33	0.67			

Figure 4.1.5: Confusion matrices for detection model v2 - DL v2.

As shown in Figure 4.1.5, a clear pattern can be observed across the matrices. For both the acoustic and vibration channels, the 3s detection logic improves the proportion of correctly identified leak cases compared with the raw model output, as shown by the increase in the normalized Leak-Leak cell from 0.80 to 0.93 for acoustic and from 0.78 to 0.91 for vibration. This indicates that the intermediate logic is effective in strengthening leak recognition. However, the background class remains more challenging, particularly for vibration, where the Background-Background cell remains relatively low at 0.35 even after 3s processing. The final 30s fusion matrix shows the strongest overall performance, with a perfect normalized leak detection rate of 1.00 and no missed leak cases. Although some background cases are still misclassified as leak, the final fusion stage clearly provides the most reliable overall decision among the three levels. The numerical performance calculated using the equations provided in Section 3.8.1, corresponding to these confusion matrices is summarized in Table 4.1.3.

Table 4.1.3: Performance matrices for detection model v2 - DL v2.

Performance matrix	Raw NEAI		3s Detection Logic		30s Detection Logic
	Acoustic	Vibration	Acoustic	Vibration	
Accuracy	0.73	0.65	0.75	0.69	0.88
Precision	0.76	0.69	0.73	0.69	0.84
Recall	0.80	0.78	0.93	0.91	1.00
Specificity	0.61	0.45	0.48	0.35	0.67
F1	0.78	0.73	0.82	0.79	0.91
Balanced Accuracy	0.71	0.61	0.70	0.63	0.83

As shown in Table 4.1.3, the performance of V2 - DL V2 improves progressively as the decision process moves from raw inference to 3s logic and finally to 30s logic. In terms of accuracy, the acoustic channel improves from 0.73 to 0.75, while the vibration channel improves from 0.65 to 0.69 after 3s logic. The final 30s detection logic reaches 0.88, confirming that the layered decision structure substantially improves the final system output. A similar pattern is seen in the F1 score, which rises from 0.78 to 0.82 for acoustics, from 0.73 to 0.79 for vibration, and then to 0.91 at the 30s stage, indicating a better overall balance between positive prediction reliability and leak detection capability. The most notable improvement is in recall, where the acoustic channel increases from 0.80 to 0.93, the vibration channel from 0.78 to 0.91, and the final 30s fusion reaches 1.00. This means that, under the tested conditions, the final implemented system did not miss any actual leak cases, which is particularly important for a monitoring application where false negatives are more critical than false positives.

At the same time, specificity remains lower than recall across all stages, with values of 0.61 and 0.45 for the raw acoustic and vibration outputs, 0.48 and 0.35 after 3s logic, and 0.67 at the final 30s stage. This indicates that the system is comparatively stronger at identifying leak cases than at rejecting background cases, meaning some false alarms are still present. This pattern is also reflected in precision, which remains moderate for the single-modality stages (0.76, 0.69, 0.73, and 0.69) before increasing to 0.84 at the 30s stage. In practical terms, this means that the final fusion logic not only improves leak sensitivity but also produces a more reliable positive alarm than either the raw or 3s outputs alone. The balanced accuracy values further support this interpretation, increasing from 0.71 and 0.61 in the raw acoustic and

vibration stages to 0.70 and 0.63 after 3s logic, and finally to 0.83 after 30s fusion. This indicates that the final version strikes the best balance between detecting real leaks and maintaining acceptable background discrimination. Overall, the results in Table 4.1.3 confirm that the selected V2 - DL V2 configuration provides the strongest practical performance, especially when the full 30s detection logic is applied, and therefore justifies its use as the final implemented model in this project.

Although the results in this section demonstrate strong performance for the selected V2 – DL V2 configuration, they should be interpreted within the context of the current simulation rig. The reported accuracy was obtained using a controlled straight-pipe setup with a maximum sensor-to-leak separation of approximately 1.14 m, which is considerably shorter than the spacing likely to occur in real water distribution networks. In practical systems, attenuation with distance is an important factor affecting leak-signal propagation and the reliability of acoustic or correlation-based detection approaches [118]. In addition, the simulated leaks were generated using replaceable circular holes with limited diameters, whereas real leaks may occur as cracks, irregular openings, or other non-uniform defects. The prototype and leak source were also positioned along the same straight pipe section, while practical installations may include bends, junctions, and more complex pipe routing that can alter vibro-acoustic signal propagation and introduce wave scattering and mode conversion, thereby complicating signal interpretation compared with an ideal straight pipeline [119]. In addition, the simulated leaks were generated using replaceable circular holes with limited diameters, whereas real leaks may occur as cracks, irregular openings, or other non-uniform defects. The prototype and leak source were also positioned along the same straight pipe section, while practical installations may include bends, junctions, and more complex pipe routing that can alter vibro-acoustic signal propagation and introduce wave scattering and mode conversion, thereby complicating signal interpretation compared with an ideal straight pipeline [120]. These constraints do not invalidate the present results; rather, they define the boundary of the current validation and indicate that the measured accuracy should be regarded as performance within a controlled physical test model, with broader generalization to real infrastructure requiring future validation on a more representative rig or field installation.

4.2 AI Model Architecture

Figure 4.2.1 shows the performance matrices extracted from the library reports of the selected acoustic and vibration models.

PERFORMANCE		PERFORMANCE	
Performance	Value	Performance	Value
Balanced Accuracy ⓘ	97.61 %	Balanced Accuracy ⓘ	93.22 %
RAM ⓘ	9.9 KB	RAM ⓘ	12.6 KB
Flash ⓘ	1183.4 KB	Flash ⓘ	79.2 KB
Accuracy ⓘ	97.71 %	Accuracy ⓘ	93.32 %
F1-Score ⓘ	97.67 %	F1-Score ⓘ	92.89 %
Precision ⓘ	97.74 %	Precision ⓘ	92.61 %
MCC ⓘ	95.36 %	MCC ⓘ	85.83 %

Figure 4.2.1: Performance matrices extracted from the acoustic (left) and vibration (right) library report.

As shown in Figure 4.2.1, the library-reported validation performance is much higher than the actual embedded detection performance discussed earlier in Table 4.1.3. For the acoustic

model, the library report indicates 97.71% accuracy, 97.67% F1-score, and 97.61% balanced accuracy, while the vibration model reports 93.32% accuracy, 92.89% F1-score, and 93.22% balanced accuracy. In contrast, the real embedded results in Table 4.1.3 are significantly lower, where the raw NEAI stage achieved 73% accuracy for acoustic and 65% accuracy for vibration, and even after the full decision hierarchy the final 30s detection logic reached 88% accuracy.

This gap indicates that the performance reported during library generation should be interpreted as an internal validation result under the model-development workflow, whereas the values in Table 4.1.3 reflect the more demanding condition of a fully deployed prototype operating on the simulation rig. In practice, the embedded system is influenced by additional factors such as physical sensor coupling, environmental interference, timing behavior, signal variation introduced by the rig, and the accumulation of errors across sensing, buffering, inference, decision logic, and transmission. As highlighted in an embedded-AI deployment reviews, accuracy measured after deployment on target hardware may differ significantly from development-stage validation, and representative on-device validation is necessary to assess real-world performance reliably [121]. Figure 4.2.2 shows the algorithm flow chart of the selected acoustic and vibration models.

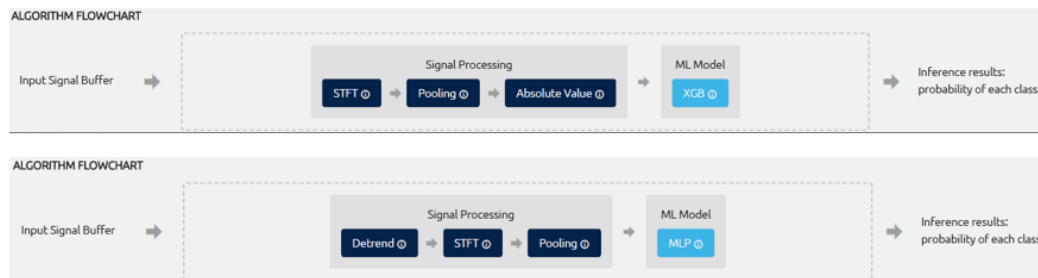


Figure 4.2.2: Algorithm flowchart extracted from the acoustic (top) and vibration (bottom) library report.

From Figure 4.2.2, the acoustic and vibration models also differ in their underlying signal-processing pipelines. For the acoustic model (top), the input signal buffer first passes through STFT, followed by pooling, and then absolute value processing before entering the machine learning model. The classifier used is XGB, indicating that the final decision is produced by an Extreme Gradient Boosting model rather than by a deep neural network with multiple hidden layers.

For the vibration model (bottom), the signal-processing chain begins with detrending, followed by STFT and pooling, before entering an MLP classifier. The vibration model uses a Multi-Layer Perceptron, meaning that its prediction is based on a neural-network structure rather than a tree-based boosting model. The exact number of hidden layers is not stated explicitly in the extracted flowchart, so from the report it can only be concluded that the vibration model is MLP-based, while the acoustic model is XGB-based. In both cases, the feature-extraction stage precedes the classifier, meaning that the raw time-domain signal is not used directly. Instead, each model first converts the input buffer into a more informative feature representation before classification.

From the architecture shown in Figure 4.2.2, it can be seen that both models follow the same broad flow of input signal buffer, to signal processing, to ML model and finally output class probabilities, but they differ in the pre-processing steps and in the type of classifier used. For the acoustic model, the inclusion of STFT, pooling, and absolute value suggests that the classifier is operating on a magnitude-based time-frequency representation, which is consistent with common audio-classification practice using spectral features [122]. For the vibration model, the addition of detrending before STFT suggests that slowly varying trend or drift components are first removed so that the later spectral representation is less influenced by

baseline variation [123]. These differences indicate that the preprocessing chain was adapted to the different characteristics of the acoustic and vibration signals before they were passed to their respective classifiers.

Overall, the library reports show that both architectures are capable of strong validation performance during model development, but the comparison with Table 4.1.3 confirms that deployment on the embedded prototype remains the more meaningful indicator of final system performance. For this reason, the real on-device results discussed in Section 4.1 are more representative of the practical detection capability of the proposed leak detection system than the library-reported metrics alone.

4.3 YAMNet Model Accuracy Comparison

4.3.1 Training Performance Evaluation

Figure 4.3.1 shows the training accuracy of the YAMNet-based acoustic and vibration models as a function of the number of training epochs.

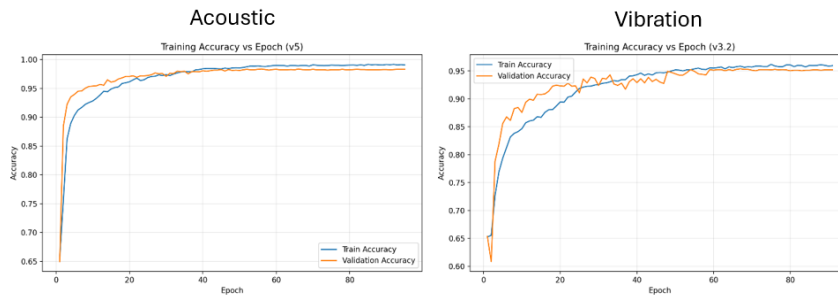


Figure 4.3.1: Training accuracy for YAMNet acoustic and vibration model versus number of epochs used for training.

As shown in Figure 4.3.1, both the acoustic and vibration models demonstrate a steady increase in training accuracy as the number of epochs increases, indicating that the models are progressively learning meaningful features from the dataset. The acoustic model reaches a higher accuracy and converges more smoothly compared to the vibration model, suggesting that the MFCC-based representation provides clearer and more distinguishable patterns for classification. In contrast, the vibration model shows a slightly lower final accuracy and more fluctuation during training, which may be attributed to the more complex and less structured nature of raw vibration signals. Despite this, both models achieve high training performance overall, indicating that the selected model architectures and training parameters are sufficient to capture the underlying characteristics of leak and background conditions under controlled dataset evaluation. Figure 4.3.2 shows the confusion matrices obtained from the trained acoustic and vibration models, respectively, evaluated on the test dataset. Both models demonstrate strong classification performance, with clear separation between leak and background classes.

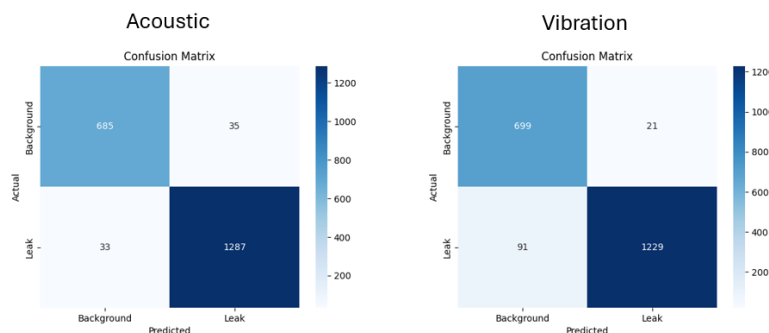


Figure 4.3.2: Confusion matrix for YAMNet acoustic and vibration best model.

As shown in Figure 4.3.2, the acoustic model shows a great classification, with minimal misclassification between classes. Similarly, the vibration model also achieves a high level of classification accuracy, although with slightly more overlap between predicted and actual labels compared to the acoustic channel. Based on the test dataset evaluation, the acoustic model achieved an accuracy of 96.7%, while the vibration model achieved 94.5%. These results indicate that both models are capable of learning meaningful patterns from their respective modalities under controlled conditions.

4.3.2 Model Conversion and Architecture Analysis

Figure 4.3.3 illustrate the model architectures before and after conversion of the YAMNet-based model using STM32Cube AI Studio for the acoustic channels.

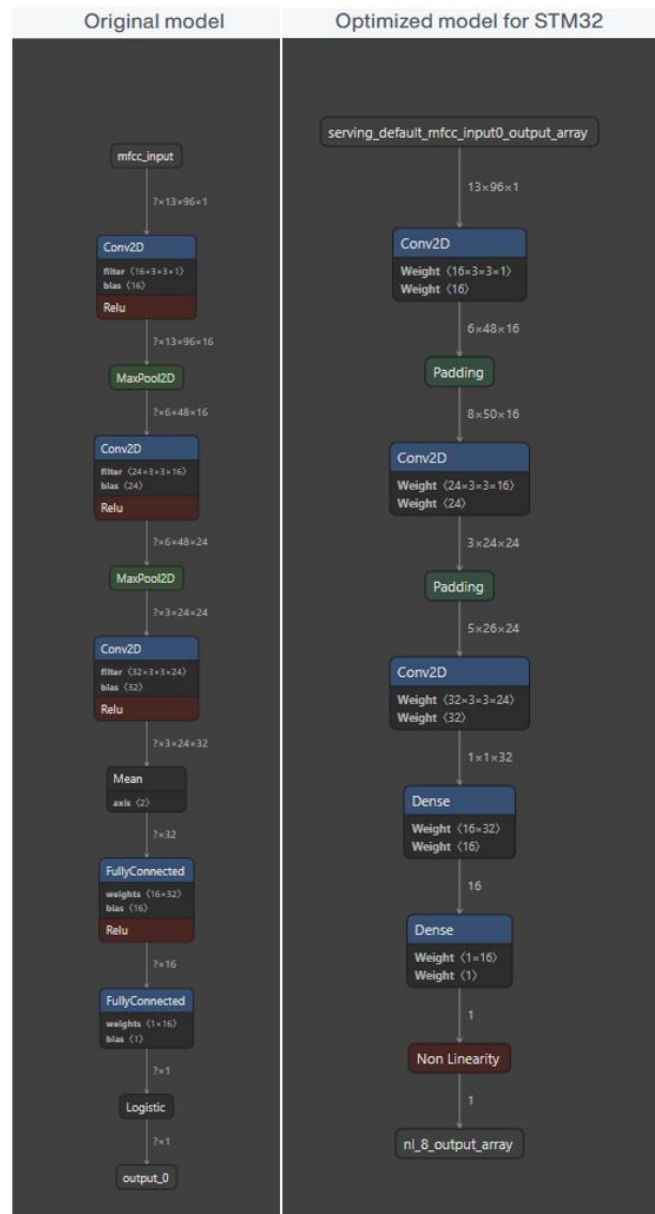


Figure 4.3.3: Architecture of the YAMNet acoustic model before and after the optimization in STM32Cube AI Studio.

From Figure 4.3.3, the overall structure of the acoustic channel remains largely consistent after conversion. The model retains its compact 2D convolutional layers operating on MFCC feature maps, with only minor optimizations applied during conversion. This suggests that the original architecture is already well-suited for embedded deployment. Figure 4.3.4 illustrate the model

architectures before and after conversion of the YAMNet-based model using STM32Cube AI Studio for the vibration channels.

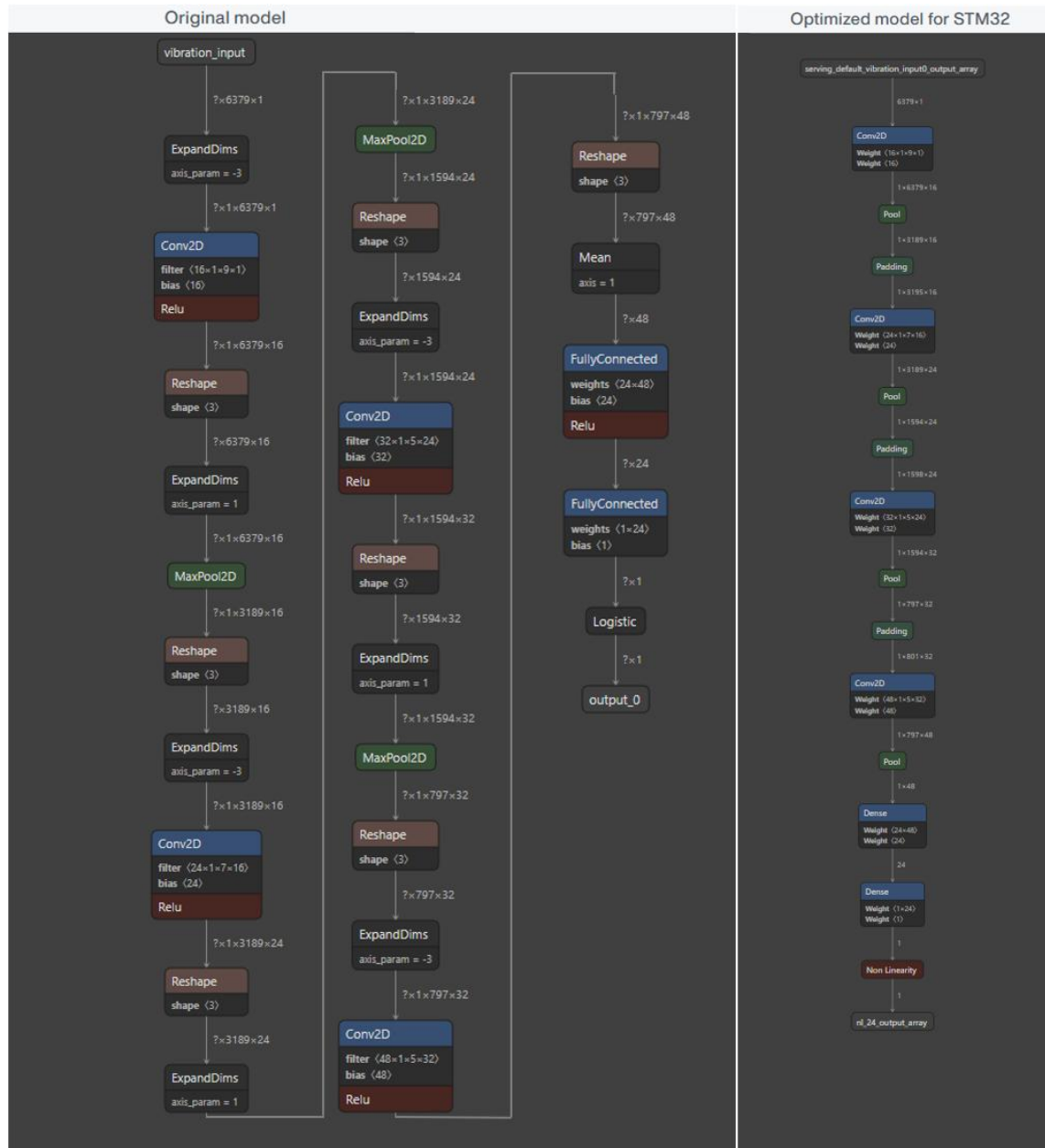


Figure 4.3.4: Architecture of the YAMNet vibration model before and after the optimization in STM32Cube AI Studio.

From Figure 4.3.4, the vibration model shows a more noticeable simplification after conversion. The number of layers is reduced, and certain operations are merged or optimized to meet embedded constraints. This reduction indicates that the original model contained elements that were either unsupported or inefficient for deployment, leading to a streamlined architecture in the final implementation.

Overall, the conversion process highlights the trade-off between model complexity and deployability, particularly for resource-constrained systems.

4.3.3 Embedded Accuracy Evaluation

Figure 4.3.5 shows the accuracy comparison between the original trained YAMNet-based models and the converted YAMNet-based model implemented into the B-U585I-IOT02A. The accuracy of the original trained YAMNet-based models is evaluated on the test dataset running

on PC. On the other side, the evaluation methodology for the converted YAMNet-based models follows a similar approach to the NEAI classification model validation process, where the prototype is loaded with the model and tested directly under real operating conditions.

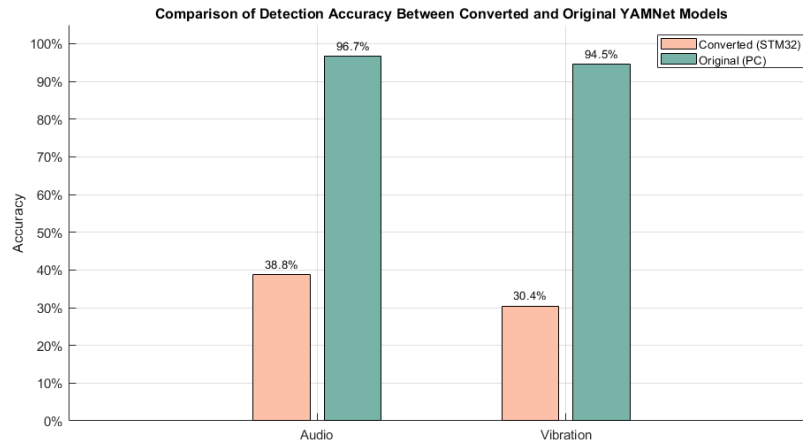


Figure 4.3.5: Comparison of detection accuracy between converted and original YAMNet models.

As shown in Figure 4.3.5, the converted YAMNet-based models exhibit a noticeable reduction in accuracy when deployed on the embedded system from 96.7% to 38.8% for acoustic channel and 94.5% to 30.4% for vibration channel. This drop in performance is mainly due to several practical limitations during deployment. First, model quantization reduces numerical precision, which can affect how well the model represents learned features. Second, some architectural simplifications are required to make the model compatible with the embedded platform. In addition, real-time execution constraints, such as limited memory and processing power, further restrict the model's performance. These factors together explain why the embedded version cannot fully match the accuracy achieved during PC-based evaluation. Figure 4.3.6 shows the accuracy comparison between the original trained YAMNet-based models, the converted YAMNet-based model and the NEAI classification model at different stages.

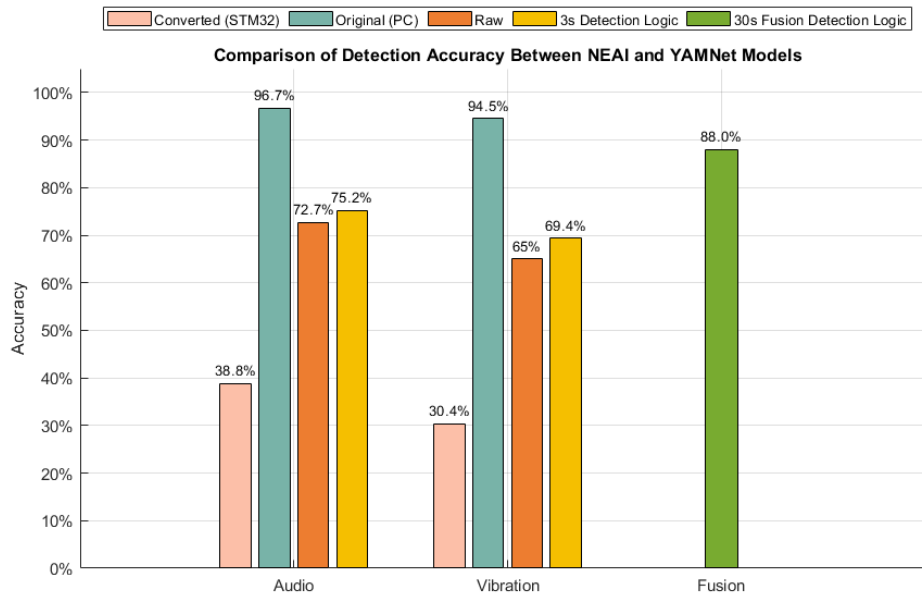


Figure 4.3.6: Comparison of detection accuracy between NEAI and YAMNet models.

As shown in Figure 4.3.6, the YAMNet-based converted model achieves lower accuracy at the raw detection level compared to the NEAI model. After applying the 30-second decision logic, the performance of the NEAI approach improves further, reaching a level close to the original YAMNet-based model evaluated on a PC, although still slightly lower.

This comparison highlights an important observation. While deep learning models such as YAMNet can achieve high accuracy in controlled environments, their performance may degrade when deployed on constrained hardware. While deep learning models such as YAMNet can achieve high accuracy under controlled conditions, their performance may decrease when implemented on resource-limited embedded systems. In contrast, the NEAI approach, when combined with a well-designed decision logic, provides a more practical and robust solution for real-time applications. This makes it more suitable for deployment in real-world scenarios, where consistency and reliability are more important than achieving the highest possible accuracy under ideal conditions.

4.4 Power Consumption

4.4.1 Measurement Overview and Operating States

Power consumption was evaluated to quantify how effectively the device enters low-power operation between periodic wake cycles. Two measurement scopes were used:

1. Entire development board (B-U585I-IOT02A) powered from an external 6 V supply regulated to ≥ 5 V at the board input and the target ER34615 battery connected to the board supply path.
2. MCU-only low-power modes, where the STM32U585 was configured to enter STOP2, STOP3, and SHUTDOWN during the “Power saving” state.

Four system states were considered in the dataset: Power saving, Idle, Detection, and LoRa. Power Saving corresponds to the planned sleep period between scheduled detection cycles. Idle refers to the device being awake but not performing any recording or inference. Detection describes the active phase where sensor data acquisition and model inference are executed. LoRa denotes the transmission phase when the device sends its payload through the LoRa P2P module. Table 4.4.1 shows the measured current drawn from the entire development board.

Table 4.4.1: Entire-board current consumption (measured)

Power source	Mode	Min Voltage (V)	Max Voltage (V)	Min Current (mA)	Max Current (mA)
External supply (6V)	Power saving	5.52	5.53	113.0	113.0
	Idle	5.46	5.47	130.0	131.0
	Detection	5.35	5.36	137.0	138.0
	LoRa	5.34	5.35	137.0	138.0
ER34615 battery	Power saving	3.20	3.24	123.8	129.3
	Idle	3.02	3.06	129.7	131.5
	Detection	3.10	3.14	121.0	135.0
	LoRa	2.86	2.95	130.0	140.1

The full development board (B-U585I-IOT02A) draws approximately 113 – 129 mA even in “Power saving” mode. This indicates that, although low-power firmware states were implemented, the development board platform contributes substantial baseline current that dominates the system’s power budget. Table 4.4.2 shows the measured current drawn by the MCU only at different sleep modes.

Table 4.4.2: MCU-only sleep current drawn by different low-power modes (measured).

MCU mode	Min Voltage (V)	Max Voltage (V)	Min Current (mA)	Max Current (mA)	Notes
STOP 2	3.3	3.3	0.170	0.180	MCU in STOP2

STOP 3	3.3	3.3	0.003	0.008	MCU in STOP3
SHUTDOWN	3.3	3.3	0.001	0.002	Limited by 1 μ A meter resolution (true may be lower)

The MCU-only measurements demonstrate that firmware-controlled deep sleep is achievable at microamp-to-sub-milliamp levels, whereas the full-board baseline ($\approx 100\text{ mA}$) is dominated by non-MCU circuitry on the development kit. Figure 4.4.1 shows the comparison of the maximum sleep current drawn by entire development board vs MCU-only with different sleep modes.

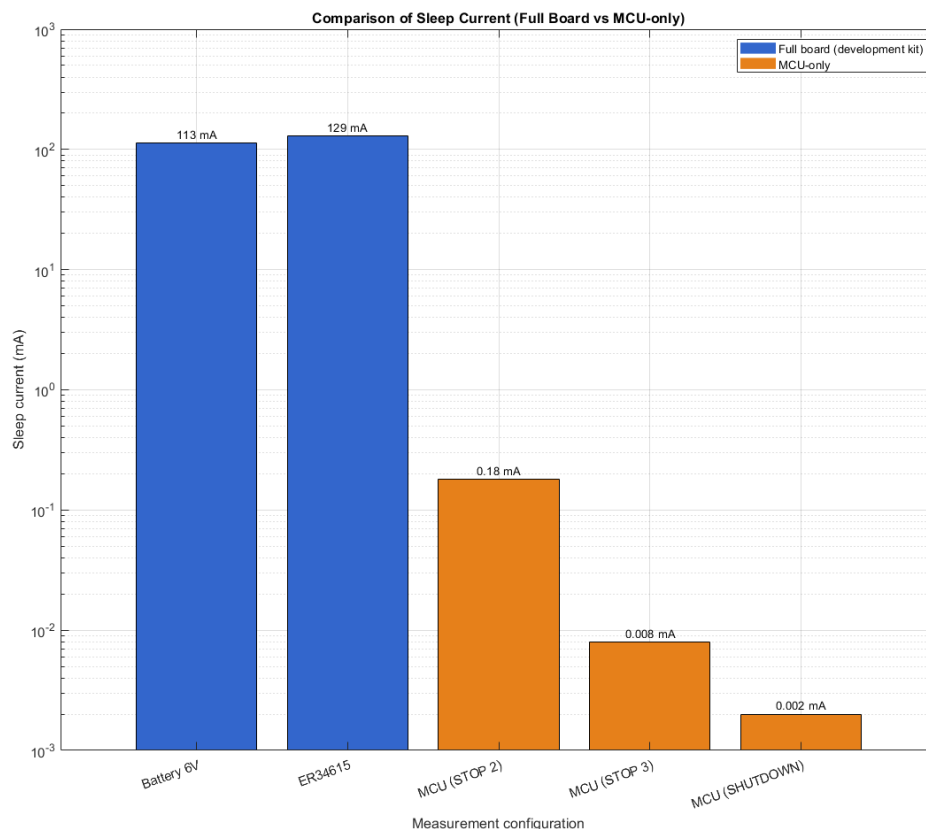


Figure 4.4.1: Comparison of Sleep Current (Full Board vs MCU-only).

Figure 4.4.1 highlights a clear separation between platform-level and MCU-level sleep behavior. The two full-board measurements (Battery 6V and ER34615) remain in the $\sim 10^2\text{ mA}$ order of magnitude during the sleep interval, whereas all MCU-only modes fall between 10^{-1} mA to 10^{-3} mA . Because the y-axis is logarithmic, this figure directly presents the orders-of-magnitude gap rather than just absolute differences.

A key observation is that the full-board “sleep” current ($\sim 113 - 129\text{ mA}$) is approximately three to five orders of magnitude higher than the MCU-only sleep currents. For example, comparing the ER34615 full-board sleep level ($\sim 129\text{ mA}$) with MCU STOP3 mode ($\sim 0.008\text{ mA}$) shows a difference of roughly 16,000 $\times$, and compared with MCU SHUTDOWN mode ($\sim 0.002\text{ mA}$) the difference is about 64,000 $\times$. This means that, in the current prototype, system sleep consumption is not limited by the STM32U585AI low-power modes, but instead by the development board’s always-powered circuitry (power conversion, debug domain, and other board-level loads). As a result, further firmware-only optimization will have limited impact on total sleep current unless the system-level power domains can be reduced or gated.

The figure also demonstrates the value of deeper MCU sleep states. Moving from STOP2 ($\sim 0.18\text{ mA}$) to STOP3 ($\sim 0.008\text{ mA}$) yields an additional reduction of about $22\times$, and STOP3 to SHUTDOWN ($\sim 0.002\text{ mA}$) gives about $4\times$ further reductions. Although these reductions are significant at the MCU level, they become negligible at the total-board level when the platform baseline sits near 100 mA . In other words, the MCU can already achieve microamp-to-sub-milliamp sleep or even lower, but the system will not benefit unless the rest of the hardware is designed to match that low-power capability by custom hardware implementation in the future. Figure 4.4.2 compares the total daily energy consumption for the prototype under a fixed operating schedule of 24 detections per day, where the daily energy includes the combined contribution from sleep + idle + detection + LoRa P2P transmission.

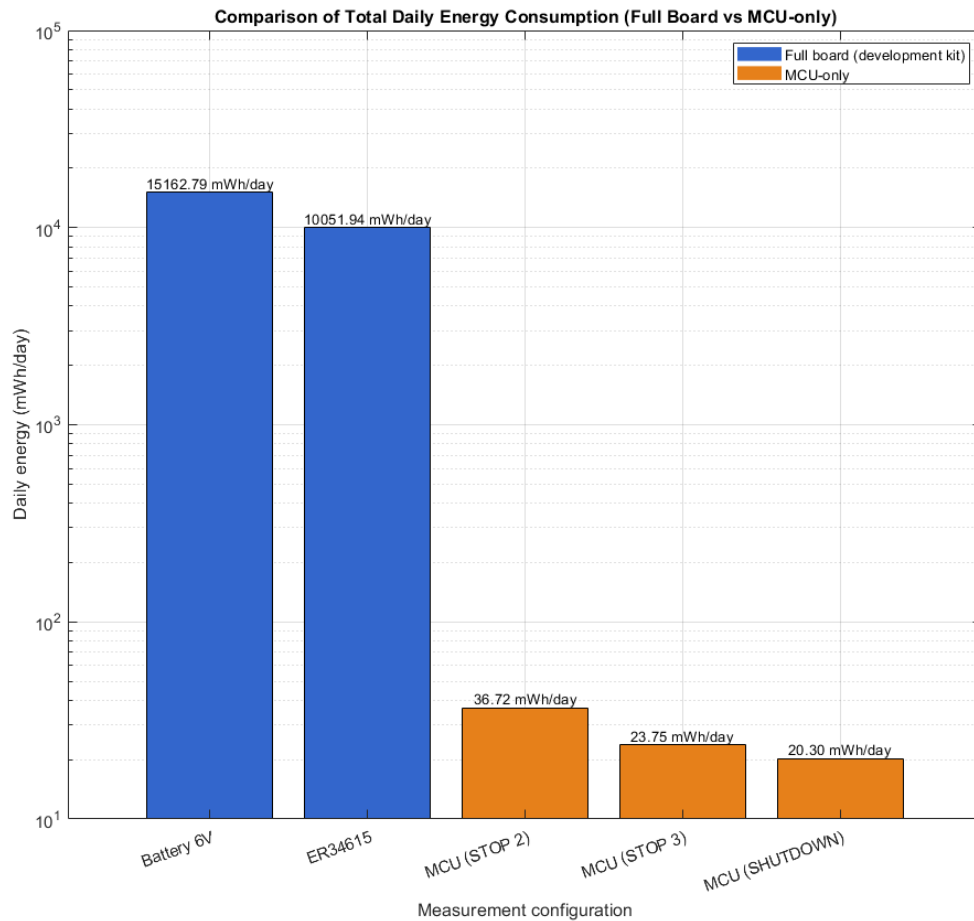


Figure 4.4.2: Comparison of Total Daily Energy Consumption (Full Board vs MCU-only).

A clear outcome from Figure 4.4.2 is that the full-board implementation dominates the energy budget. The development kit operating from the external 6V supply corresponds to approximately $15,162.79\text{ mWh/day}$, while the ER34615-powered case is $10,051.94\text{ mWh/day}$. In contrast, the MCU-only cases are approximately 36.72 mWh/day (STOP2), 23.75 mWh/day (STOP3), and 20.30 mWh/day (SHUTDOWN). This indicates a reduction of roughly two to three orders of magnitude when comparing the system energy dominated by the development board platform versus the MCU-only low-power operation.

The difference between the two “full board” bars (Battery 6V vs ER34615) also highlights why energy (mWh/day) is a more appropriate metric than current when voltages differ. Although both full-board cases show similarly high current during sleep, the daily energy differs because the measured supply voltages are not the same and because the power path and regulation losses can vary across configurations. Presenting results in energy terms therefore avoids misleading conclusions that could arise from comparing currents measured at different voltages.

Importantly, the daily energy values in Figure 4.4.2 should be interpreted as a rough estimation rather than a precise energy audit. The estimate is based on the observed duration of a single detection cycle and then extrapolated to 24 detections per day as shown in Table 4.4.3. Any variation in real operation, such as longer inference time, lagging, etc. would change the total daily energy. Nevertheless, this approach remains valuable for preliminary assessment, as it captures the expected trend and provides a consistent framework for comparing operating modes under a common assumed duty cycle.

Table 4.4.3: Estimated maximum duration for each state of the device per day.

No. of detection per day	1	24
Power saving duration	86,304.38 s/day	84,795.12 s/day
Idle duration	60.00 s/day	750.00 s/day
Detection duration	33.32 s/day	799.68 s/day
LoRa duration	2.30 s/day	55.20 s/day

4.4.2 Datasheet Comparison (STM32U585)

The STM32U585 family is specified for ultra-low-power operation, with typical currents in deep low-power modes in the μA or sub- μA range depending on SRAM retention, RTC, and wake configuration [15]. Table 4.4.4 shows the current drawn by the MCU at different power modes based on the datasheet [15].

Table 4.4.4: Current Drawn by MCU at Different Power Mode from Datasheet [15].

Power Mode	Current
Stop 2 mode with full SRAM	8.95 μA
Stop 2 mode with 16-Kbyte SRAM	4.0 μA
Stop 3 mode with full SRAM	4.3 μA
Stop 3 mode with 16-Kbyte SRAM	1.9 μA
Shutdown mode (24 wake-up pins)	160 nA

In practice, measured currents can exceed datasheet “typical” values due to factors explicitly highlighted by ST, including retained domains, enabled clocks/peripherals, regulator configuration, and debug circuitry [124]. ST’s power optimization guidance notes that Stop modes place most domains into low-leakage states, but the actual current depends strongly on software configuration and what remains powered/clocked [124].

4.4.3 Battery Lifetime Estimation

The battery lifetime presented in this section should be interpreted as an analytical model based on measured electrical consumption and nominal battery capacity, rather than as a direct long-duration run-down measurement. This analytical model is useful for comparing operating modes and estimating the effect of design decisions, even though the absolute lifetime will still depend on real deployment conditions and battery behavior over time. The battery (ER34615) lifetime can be estimated using the equation,

$$\text{Lifetime (days)} = \frac{\text{Capacity} \times \text{nominal voltage}}{\text{Total Daily Energy for detection}}$$

Using ER34615 capacity $\approx 19,000 \text{ mAh}$ (nominal) and nominal voltage = 3.3V , The lifetime of the battery can be calculated. Figure 4.4.3 shows the estimated lifetime based on ER34614 capacity.

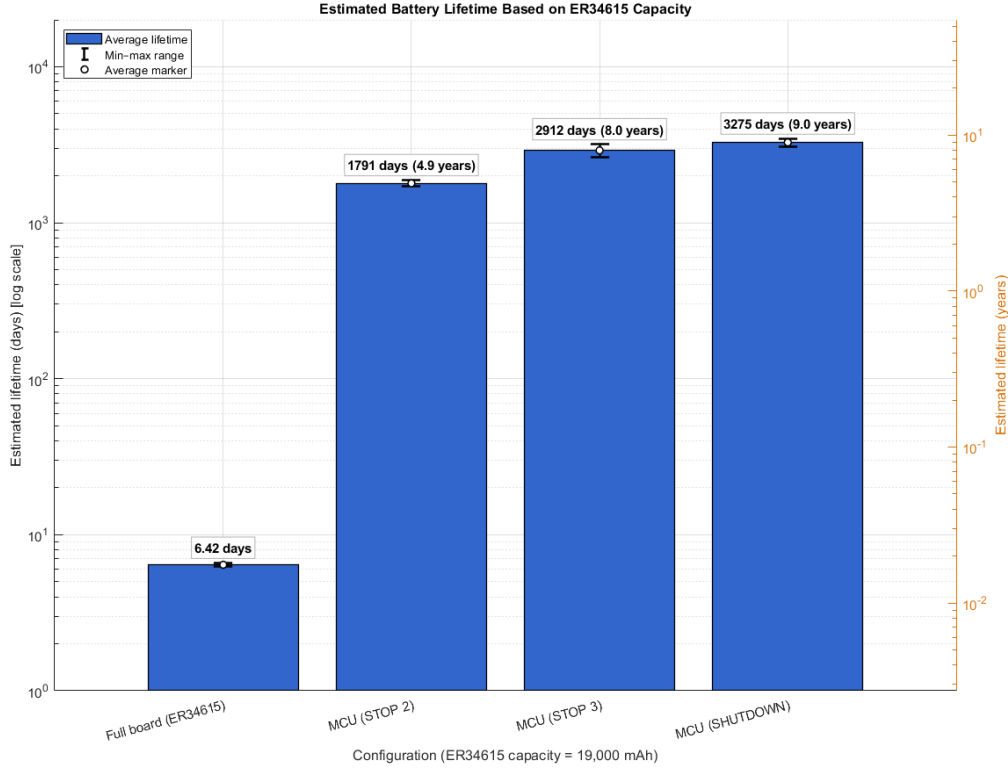


Figure 4.4.3: Estimated lifetime based on ER34614 capacity.

From Figure 4.4.3, the bars represent the average lifetime, while the error bars indicate the min-max lifetime computed from the measured current variation. A strong contrast is observed between the full development board and MCU-only low-power modes. The full-board configuration yields only ~ 6.4 days of expected lifetime, whereas the MCU-only cases extend to ~ 4.9 years (STOP2), ~ 8.0 years (STOP3), and ~ 9.0 years (SHUTDOWN). This figure therefore illustrates that battery lifetime is primarily limited by board-level baseline consumption rather than the STM32U585 low-power capability.

4.4.4 Effect of Detection Frequency on Daily Energy Consumption and Battery Lifetime

This section extends the power analysis by examining how the number of detections per day affects the worst-case total daily energy consumption and the corresponding minimum battery lifetime. In this analysis, the worst-case condition was estimated using the maximum measured power together with the maximum observed duration for each state, namely sleep, idle, detection, and LoRa P2P transmission. Therefore, these plots should be interpreted as conservative design estimates rather than exact field lifetime predictions. Figure 4.4.4 shows the worst case total daily energy consumption versus the number of detections per day for the entire board powered by the ER34615 battery.

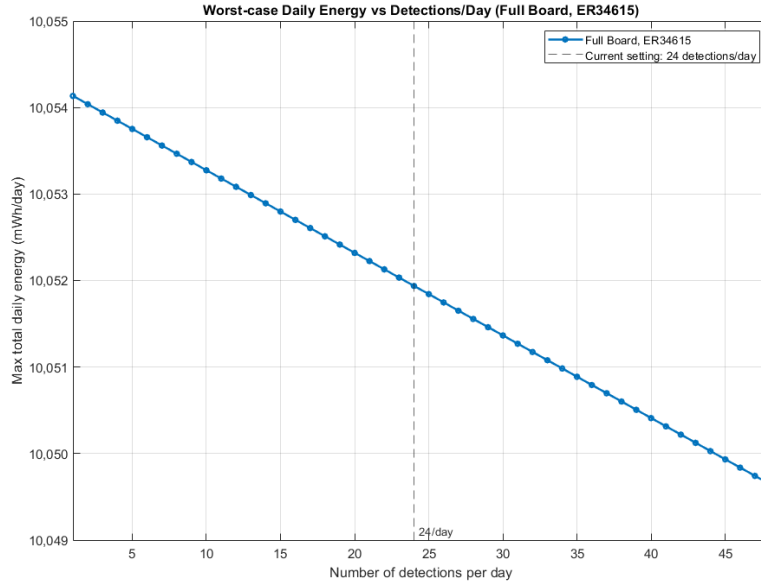


Figure 4.4.4: Total daily energy consumption of the full board versus number of detections per day.

For the full-board implementation, Figure 4.4.4 shows that the total daily energy remains almost constant at approximately 10,052 mWh/day across 1 to 48 detections per day. This indicates that the energy budget is overwhelmingly dominated by the board’s baseline consumption during non-active periods, rather than by the incremental energy required for each additional detection cycle. In other words, increasing the detection frequency has only a very small effect on the total daily energy when the system is deployed on the complete B-U5851-IOT02A platform, because the development board itself already consumes a large amount of energy even when the application is intended to be in a low-power state. This trend is reflected again in Figure 4.4.5 which shows the minimum lifetime versus the number of detections per day for the entire board powered by the ER34615 battery.

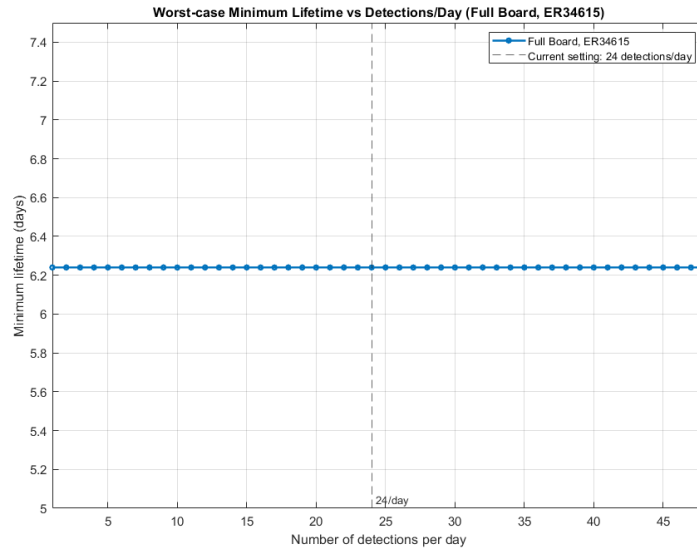


Figure 4.4.5: Minimum lifetime of the full board versus the number of detections per day.

As shown in Figure 4.4.5, the estimated minimum lifetime of the full-board system stays almost flat at around 6.24 days regardless of whether the device performs 1 detection or 48 detections per day. Figure 4.4.5 shows that, under the current development-board implementation, battery lifetime is effectively constrained by the persistent platform-level load. As a result, adjusting the detection frequency alone does not provide a meaningful extension of operating life. This supports the earlier conclusion that the development board is suitable for validating firmware

functionality and wake-sleep operation, but it is not representative of the final energy profile expected from a custom low-power product design. In contrast, the MCU-only results reveal a much clearer relationship between detection frequency and energy usage. Figure 4.4.6 shows the worst case total daily energy consumption versus the number of detections per day for the MCU-only configurations, STOP2, STOP3, and SHUTDOWN.

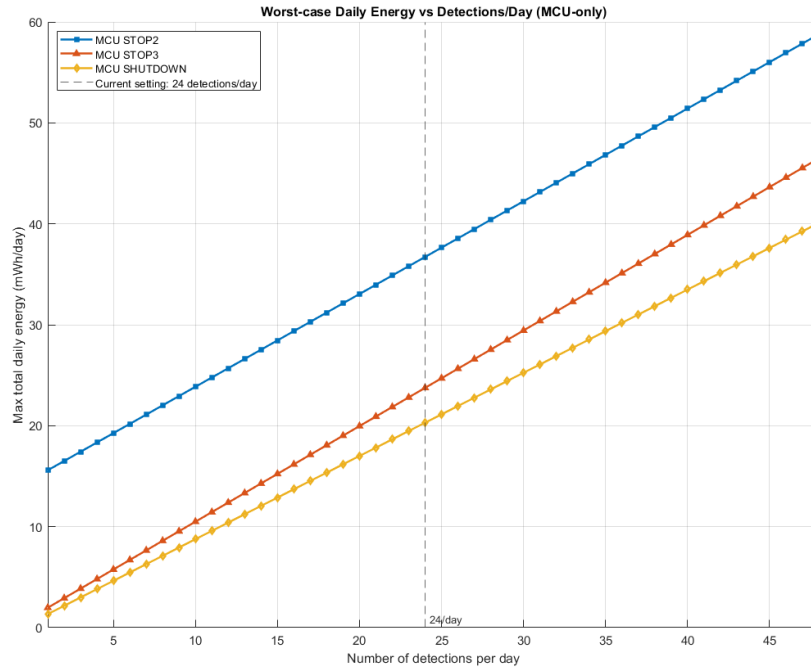


Figure 4.4.6: Total daily energy consumption of the MCU-only configurations versus number of detections per day.

Figure 4.4.6 shows that the worst-case total daily energy increases approximately linearly as the number of detections per day increases. Among the three MCU-only sleep modes, STOP2 exhibits the highest daily energy, followed by STOP3, while SHUTDOWN gives the lowest daily energy throughout the entire range. Similar trends appear in Figure 4.4.7 which shows the minimum lifetime versus the number of detections per day for the MCU-only configurations, STOP2, STOP3, and SHUTDOWN.

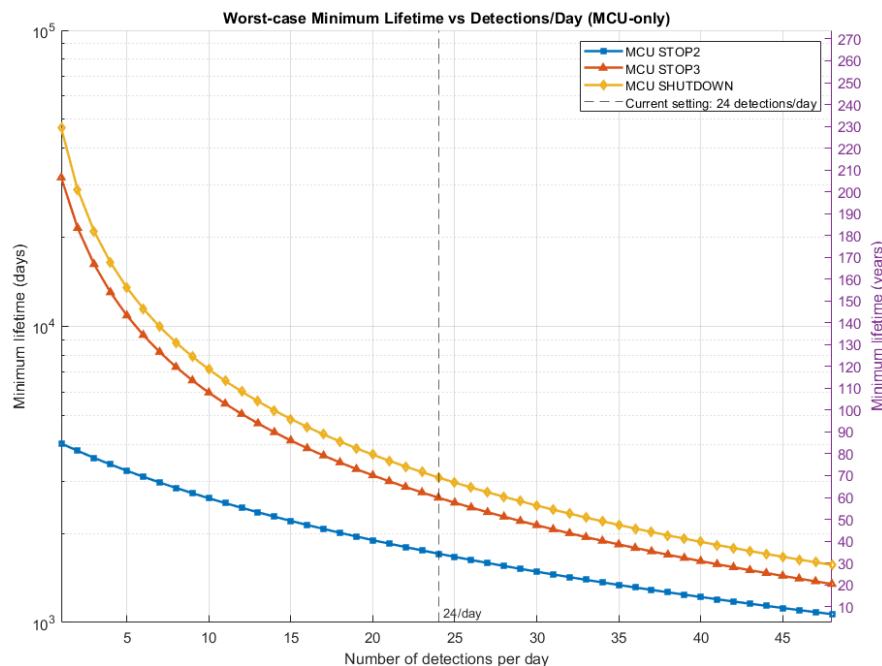


Figure 4.4.7: Minimum lifetime of the MCU-only configurations versus the number of detections per day.

From Figure 4.4.7, the minimum lifetime of the MCU-only system decreases as the number of detections per day increases. At lower detection rates, the estimated lifetime is very long because the MCU spends most of its time in deep sleep. As the detection frequency increases, more time is spent in active operation, causing the lifetime to drop progressively. Even so, the relative ranking remains unchanged, where the SHUTDOWN mode provides the longest lifetime, followed by STOP3, then STOP2.

This confirms that, for a future custom hardware implementation, selecting a deeper sleep mode remains beneficial, but the final lifetime will still depend strongly on how often the node is required to wake and perform detection.

The vertical dashed line in Figure 4.4.4 to Figure 4.4.7 marks the current selected operating point of 24 detections per day, which corresponds to one detection every hour. This setting was chosen as a practical middle ground to balance between battery lifetime and monitoring effectiveness. One detection every hour interval is frequent enough to provide regular monitoring of the water distribution system while avoiding the much higher energy cost associated with very frequent wake schedules.

The results also show that the daily detections rate is a configurable design parameter rather than a fixed requirement. It can be adjusted according to operational needs. For example, a company may select a higher detection frequency for crucial facilities or sites with a high leakage risk, where earlier leak event awareness is more valuable than maximum battery life. In contrast, a lower detection frequency may be preferred in lower-risk deployments to extend maintenance intervals and reduce battery replacement cost. This means that the final detection rate can be aligned with broader deployment strategy, service expectations, and maintenance planning, provided that the associated reduction in lifetime is clearly understood from the trends shown in Figure 4.4.4 to Figure 4.4.7.

4.4.5 Development Board Limitation in Low-Power Operation

The measured full-board sleep current ($\sim 113 - 129\text{ mA}$) indicates that the prototype, as implemented on the B-U585I-IOT02A Discovery kit, cannot meet that requirement in its current hardware form. This outcome should be presented as a platform limitation rather than a firmware failure for two reasons.

First, the discovery kit is not a low-power product design. The development boards typically include always-available debug and support circuits such as the ST-LINK domain, power switches, multiple regulators, indicator LEDs, and sensor expansion circuitry, that raise baseline current.

Secondly, the MCU-only results confirm low-power capability at the controller level. When measuring the MCU low-power modes, STOP2, STOP3 and SHUTDOWN modes, the current drawn is much lower than the full-board baseline. Also, the SHUTDOWN mode reading is constrained by the $1\text{ }\mu\text{A}$ meter resolution, so the real SHUTDOWN mode current could be lower. This aligns with the low-power positioning and datasheet low-power currents for STM32U585 deep sleep states [15].

The prototype successfully demonstrates the critical architectural elements of low-power Edge-AI operation:

1. Periodic wake-up (RTC),
2. Execution of on-device inference,
3. Wireless communication (LoRa P2P),
4. Returning to low-power mode.

However, the results also show that achieving a persuasive battery lifetime requires moving from a development kit to a custom low-power hardware design.

4.4.6 Implication for the final design

A practical path to meet the low power operation is to design a dedicated PCB containing only the required components/peripherals for this project and to explicitly power-gate everything else. Based on the current system requirements, the essential components/peripherals are:

1. STM32U585 MCU (core processing + RTC/backup domain)
2. Microphone front-end (only powered during sampling for inference)
3. Accelerometer (only powered during sampling for inference)
4. LoRa P2P module such as RA-08H (powered only around transmission)
5. Efficient power regulation

Therefore, the development kit results are best interpreted as proof of functionality for the low-power firmware strategy, while the MCU-only measurements indicate that a future custom board can realistically achieve the intended low-power behavior when the peripheral power domains are specifically designed for battery operation.

4.5 LoRa P2P Transmission

This section presents the implementation and validation of the wireless data transmission Stretch Goal III, namely: “Utilize the onboard ARDUINO® Uno V3 expansion interface to send data (e.g., detection results and timestamps) via LoRa peer-to-peer (P2P) communication.”. In the current prototype, the communication was implemented as a LoRa P2P link between the sensing node and a base receiver, rather than through a gateway-based network. This arrangement is consistent with the communication architecture described earlier in Section 3.2.3 and provides a practical way to validate wireless transmission of leak-detection results with reduced system complexity.

Communication validation showed that the handshake mechanism described in the methodology was able to confirm successful bidirectional wireless communication before payload transfer. After successful link establishment, the STM32U585AI MCU transmitted a multi-line detection payload containing the start date and time, recording duration, acoustic per-segment average leak confidence (confidence level of each segment), vibration per-segment average leak confidence (confidence level of each segment), final fused score and verdict, and end time. The payload was sent from the STM32U585AI MCU to the node LoRa through UART and then forwarded over the radio link to the base receiver. The example shown in Figure 4.5.1 confirms that the base LoRa was able to receive and display the transmitted detection information correctly and perform the handshake during wake. This demonstrates that the device is capable not only of local embedded inference, but also of wirelessly forwarding summarized leak-detection results to an external receiver for monitoring and logging purposes.

```
RX:START: 07/02/2026 00:01:03.478 - Detect 30s (10*3s chunks)
RX:AUD_chunkAvgLeak: 0.022, 0.068, 0.019, 0.035, 0.103, 0.093, 0.159, 0.066, 0.357, 0.032
RX:VIB_chunkAvgLeak: 0.994, 0.998, 0.998, 0.996, 0.998, 0.999, 0.998, 0.997, 0.867, 0.997
RX:Fuse Score: 0.277, Verdict: Background (0.460)
RX:END: 00:03:40.337
RX:SLEEP: 07/02/2026 00:03:41.707
RX:PING:18626
TX:PONG:18626
RX:WAKE: 07/02/2026 00:04:12.347
RX:MODE: DETECTION
```

Figure 4.5.1: Example payload received by the base LoRa.

Communication range testing further showed that, under obstructed conditions, smooth and reliable transmission was maintained up to approximately 40m, as shown in Figure 4.4.2. In this test, the node LoRa P2P module was placed inside a room while the base receiver was located inside a box, such that both ends were surrounded by solid obstructions. Beyond this range, the link could still reach up to approximately 60m, but with an increased risk of intermittent packet loss where some messages were not received successfully. Although the reliable range is not long, maintaining performance within 40m is sufficient for the present proof-of-concept because the future deployment concept does not depend on a single direct transmission from every node to one base station.

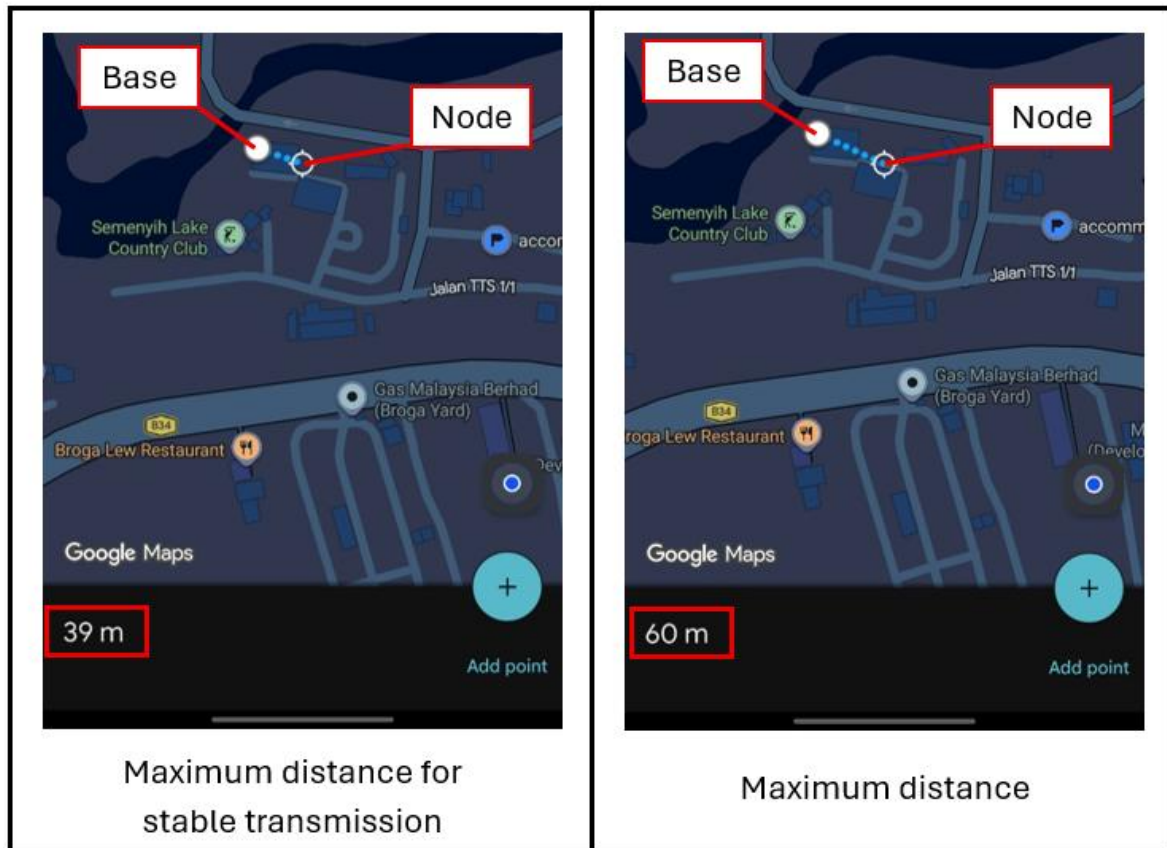


Figure 4.5.2: LoRa P2P communication range test under obstructed conditions.

As discussed earlier in Section 3.2.3, a larger deployment could instead adopt a multi-node forwarding strategy, where data are relayed from one node to another until they reach the base node. Under such an arrangement, each hop only needs to remain within a manageable communication range, making the present result acceptable as an initial validation of the LoRa P2P transmission concept.

4.6 Device Holder

The device holder developed in this project successfully supported Objective IV, to design and assemble a fully functional prototype for development on a controlled test rig. By integrating the B-U585I-IOT02A development board, battery-powered hardware, and LoRa P2P communication module into a single mounted structure, the holder enabled the sensing device to be attached directly to the pipe during dataset collection and embedded testing. In this way, the holder is not only a mechanical accessory, but also an important component of the prototype system, enabling the leak detection workflow to be evaluated in a real installation configuration. The inner view of the complete fabricated prototype is shown in Figure 4.6.1.

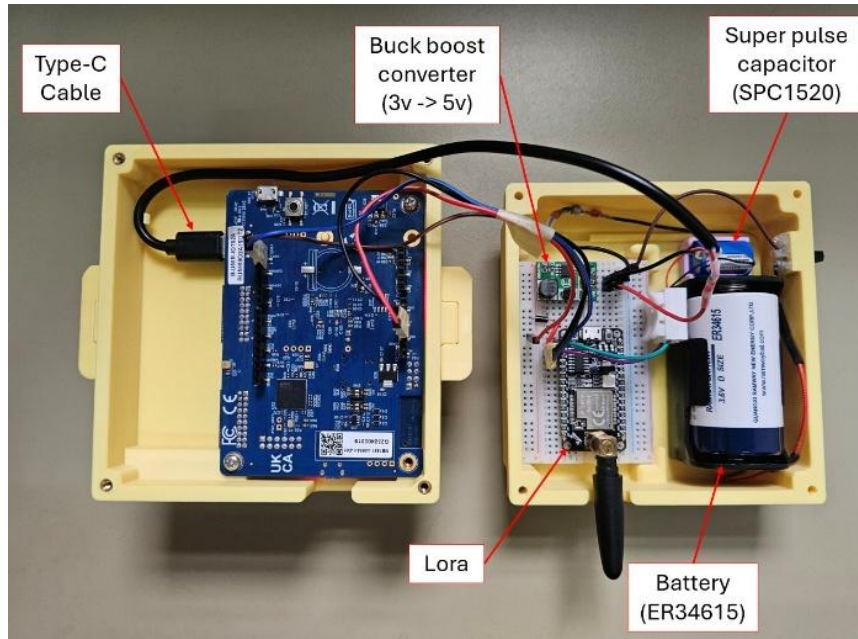


Figure 4.6.1: Inner view of the prototype device.

In practical use, the holder provided a stable and repeatable mounting mechanism on the pipe, which is important for maintaining consistent sensor coupling during experiments. This is particularly relevant for the vibration channel, where changes in mechanical contact can affect the transmitted signal characteristics. The outer mounting arrangement is illustrated in Figure 4.6.2, where the enclosure is clamped onto the pipe using the two-point fastening arrangement described earlier in the methodology.

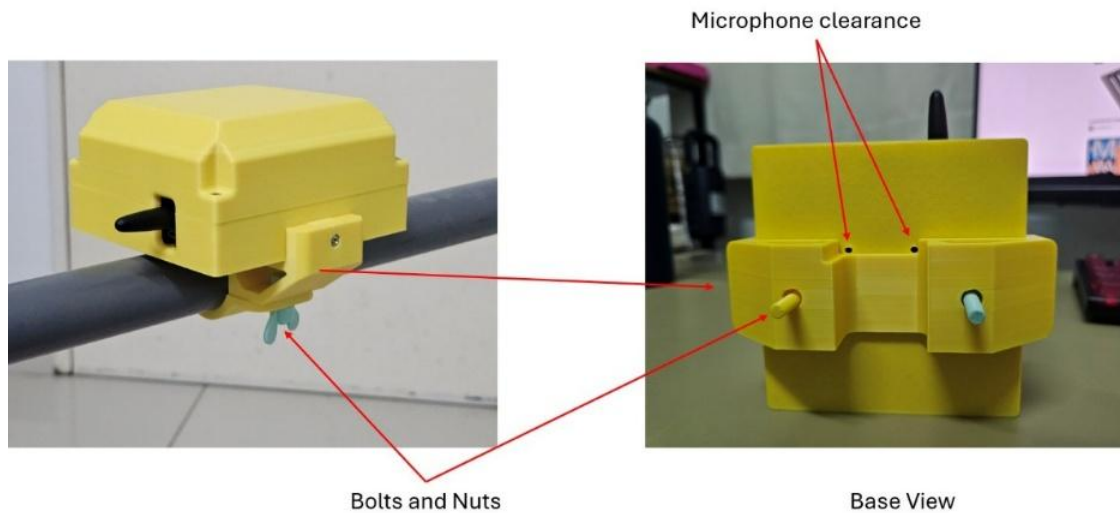


Figure 4.6.2: Outer view and mounting mechanism of the holder.

As shown in Figure 4.6.2, the successful mounting and operation of the prototype demonstrates that the holder fulfilled its intended role for controlled rig-based development and testing. At the same time, the assembled holder kept the major hardware components physically organized and functional during operation, thereby improving the overall robustness of the prototype during repeated experiments. The holder also contributed to the overall usability of the prototype by making the system easier to install, remove, and reposition during repeated experiments. This was beneficial during dataset collection and leak validation, where the device had to be mounted consistently across multiple trials. As shown in Figure 4.6.3, the enclosure design maintained access to important external parts such as the LoRa antenna, ST-

LINK port and power switch, allowing the prototype to be operated without disassembly once mounted. From a prototype-development perspective, the holder therefore helped bridge the gap between circuit-level functionality and a more complete deployable system form. This makes it an important enabling component of the project, even though its primary role is mechanical rather than algorithmic.

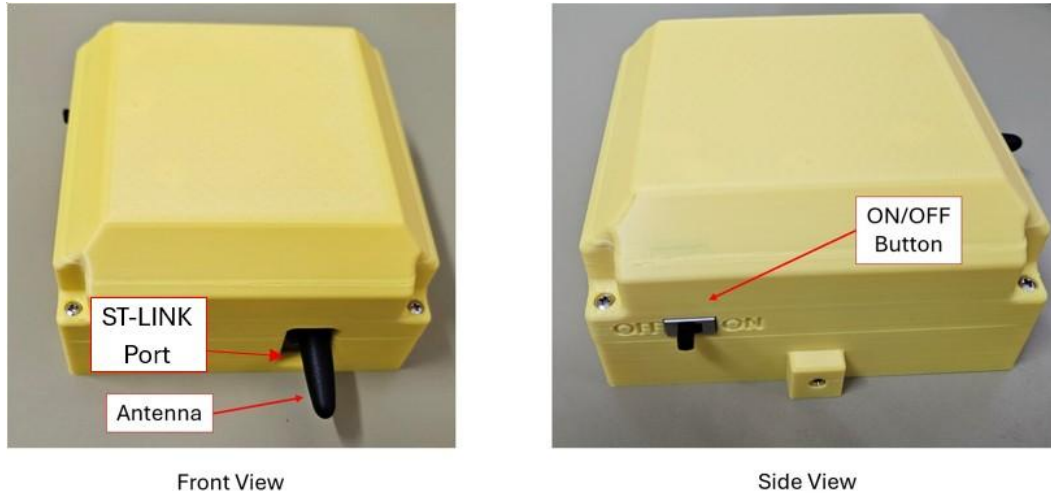


Figure 4.6.3: Components that are accessible from outside.

Although the holder meets the needs of controlled laboratory testing, it should still be regarded as a first-generation laboratory prototype rather than a finalized field-ready enclosure. The pipe mounting geometry is optimized for the 25mm pipe used in the simulation rig, so adaptation would be required for other diameters or for pipes with poor accessibility. In addition, clamp torque and long-term polymer creep could change the contact pressure over time, potentially affecting vibration transmissibility and therefore the measured features [108].

Finally, although microphone clearance is provided, a real-world installation would benefit from an acoustic vent or membrane solution to reduce the risk of dust or water ingress while maintaining acoustic transparency. Commercial acoustic vents are widely used to protect microphone ports while minimizing insertion loss, and they are commonly specified to achieve ingress-protection ratings in harsh environments [125]. Consequently, further refinement would still be needed before the holder could be considered suitable for wider field deployment.

Nevertheless, within the scope of the current work, the holder successfully enabled a complete and functional prototype to be assembled and tested on the simulation rig. It provided the necessary mechanical integration, external accessibility, and mounting repeatability required for the present study, and therefore satisfied its intended role in supporting the overall leak detection system.

5. Conclusion and Future Works

5.1 Detection System Performance

The performance of the developed leak detection system demonstrates that Objective II, which aims to develop an AI-based leak detection algorithm using both acoustic and vibration signal analysis, has been successfully achieved. The proposed approach combines modality-specific classification models with temporal decision logic to improve robustness. While the raw classification outputs show moderate accuracy, the integration of time-based decision strategies significantly enhances overall detection performance. The final system achieves a

high level of accuracy and consistency, indicating that the combination of acoustic and vibration sensing provides complementary information that strengthens leak detection reliability under practical operating conditions.

The results further show that system-level performance is not solely dependent on model accuracy, but also on how detection decisions are aggregated over time. By incorporating structured decision logic over multiple time windows, the system is able to reduce false detections and better capture sustained leak behavior. This highlights the importance of designing not only the AI model, but also the surrounding inference framework, particularly for embedded applications where signals are dynamic and noisy.

In relation to Stretch Goal V, which involves benchmarking the deployed AI model against alternative approaches such as YAMNet, the comparison demonstrates that the NEAI-based model is more suitable for embedded deployment. Although the YAMNet-based models achieve high accuracy during training under controlled conditions, their performance degrades significantly after conversion and deployment onto the embedded platform. In contrast, the NEAI model maintains stronger on-device performance and benefits further from the applied decision logic. This indicates that models specifically optimized for embedded systems can provide more reliable real-time performance than more complex deep learning models when operating under hardware constraints.

The reported detection performance validation was conducted on the same simulation rig developed for Objective I, which was used to collect acoustic and vibration data under defined leak conditions and controlled variables. The use of a simplified simulation rig with limited leak geometries and relatively short sensor-to-leak distances provides a consistent environment for evaluation but does not fully represent real-world pipeline conditions. Factors such as longer propagation distances, complex pipe structures, and irregular leak formations may influence signal characteristics and detection accuracy. As such, the current results should be interpreted as validation within a controlled environment, with further testing required to confirm performance in real deployment scenarios.

Overall, the system demonstrates a strong balance between accuracy, robustness, and deployability. The integration of multi-modal sensing, embedded AI models, and application-specific decision logic enables reliable leak detection within the constraints of a low-power embedded platform. These results confirm the feasibility of the proposed approach as a practical solution for real-time water leak detection and provide a solid foundation for future improvements and field deployment.

5.2 Power Consumption and Low-Power Operation

In relation to Objective III, to ensure low power operation, the project demonstrates that low-power behavior was successfully achieved at the MCU and firmware level, but not yet at the level of the complete development-board-based prototype. The implemented power strategy enabled the STM32U585AI to periodically wake from sleep, perform sensing and on-device inference, transmit the detection result through the LoRa P2P communication, and then return to a low-power state. This confirms that the proposed periodically edge-AI operating concept is feasible and that the firmware architecture supports low-power scheduling as intended.

The measurement results further show that STM32U585AI itself is capable of very low current operation in deep sleep modes, as evidenced by the MCU-only STOP2, STOP3, and SHUTDOWN current measurements. However, the full-board measurements clearly indicate that the assembled prototype based on the B-U585I-IOT02A Discovery kit remains dominated by platform-level power consumption. Even during the intended sleep interval, the full-board current remained at approximately 113 – 129 mA, leading to short estimated battery lifetime

when evaluated using the ER34615 battery. This means that, although low-power operation was demonstrated conceptually, the present prototype cannot yet be regarded as a truly low-power field-ready node. Therefore, the limiting factor in the current prototype is not the low-power capability of the MCU itself, but the overhead introduced by the general-purpose development platform.

Accordingly, this objective is best conclude as partially achieved. The project has successfully verified the required low-power operating sequence and has shown that the controller can be driven into deep sleep modes appropriate for periodically embedded monitoring. At the same time, the measured full-board consumption demonstrates that additional hardware optimization is essential before the design can deliver persuasive long-term battery operation in practice. A future deployment-oriented implementation should therefore be developed using a custom hardware platform that includes only the required functions and explicitly accounts for the power demand of the major peripherals, including the microphone, accelerometer, LoRa P2P communication module, power-conversion stage, and supporting circuitry.

Further reduction in controller-side power may also be possible through selection of an MCU family with even lower published deep-sleep current. As noted earlier in the project overview, STM32L562QEI6Q is a possible alternative candidate. Nevertheless, MCU substitution alone would not fully solve the overall power issue, because the final system lifetime will still depend on the combined contribution of the sensors, radio, regulators, and supporting hardware. For this reason, the main conclusion of this study is that low-power operation is technically achievable, but its practical realization requires both firmware-level sleep management and hardware specifically designed for battery-powered deployment.

5.3 Holder Design

The holder design can be concluded to have successfully supported Objective IV, by enabling the leak detection system to exist as an integrated prototype rather than as a collection of separate electronic parts. Through the CAD modelling, assembly planning, and final fabrication stages, the holder provided a practical mechanical platform for combining the MCU board, battery-powered hardware, and LoRa P2P communication module into a single pipe-mounted device. This was important for the project because it allowed the proposed sensing and detection concept to be developed in a form that more closely reflects a deployable engineering prototype.

From the overall project perspective, the main contribution of the holder was to translate the electronic and embedded subsystems into a usable physical structure suitable for controlled experimental work. It provided the foundation for repeatable installation on the pipe and supported the operation of the prototype as one complete unit during testing. In this sense, the holder was not only a packaging component, but also an enabling part of the prototype-development process, since it allowed the leak detection device to be evaluated under mounted conditions relevant to the intended application.

However, the holder should still be regarded as a prototype-stage design rather than a finalized field enclosure. Its present form is suitable for the laboratory rig and the pipe configuration used in this study, but further development would be required before broader deployment. Future refinement should focus on improving compactness, environmental protection, adaptability to different pipe installations, and closer integration with a custom low-power PCB. Nevertheless, within the scope of this project, the holder design fulfilled its intended role by enabling a complete, functional, and testable prototype system.

5.4 Overall Conclusion

As discussed in Sections 5.1 to 5.3, the main technical objectives of this project have largely been addressed. Objective I was achieved through the development of a controlled leak simulation rig for repeatable dataset collection and validation. Objective II was achieved through the development of an embedded AI leak detection system using acoustic and vibration sensing, with the selected V2 - DL V2 configuration providing the best overall performance. Objective III was partially achieved, as low-power operation was demonstrated successfully at MCU and firmware level, although a custom hardware platform is still required for full battery-powered deployment in the future. Objective IV was also achieved through the successful development of a holder that enabled the complete prototype to be mounted and tested as an integrated device.

Objective V was also achieved within the scope of this project. The complete system was evaluated through embedded validation of detection performance, low-power operation, LoRa P2P communication, and mechanical integration on the simulation rig. This confirms that the final prototype was not only developed at subsystem level but was also tested as one complete proof-of-concept system. In terms of stretch goals, LoRa P2P wireless transmission and AI model benchmarking against YAMNet were successfully completed. By contrast, Bluetooth connectivity, refined LED mode indication, and leak localization were not completed within the available project duration, as priority was given to the core detection, validation, and low-power aspects of the project. Table 5.4.1 shows a summary of the objective and stretch goal status.

Table 5.4.1: Summary of the objective and stretch goal status.

Objective/Stretch Goal	Status	Evidence
Objective I: Controlled experimental environment	Achieved	Simulation rig developed and used for dataset collection and validation
Objective II: AI leak detection using acoustic and vibration sensing	Achieved	Embedded NEAI system deployed; best configuration achieved 88.0% final accuracy
Objective III: Ensure low power operation	Partially achieved	MCU low-power behavior demonstrated; full board is still limited by development-board consumption
Objective IV: Functional prototype on controlled test rig	Achieved	Holder and integrated prototype successfully mounted and tested
Objective V: Evaluate complete system implementation	Achieved	Detection, power, LoRa, and holder performance validated on the prototype
Stretch Goal I: LED indication refinement	Not achieved	Deferred due to time constraints
Stretch Goal II: LoRa P2P	Achieved	Wireless payload successfully transmitted and received
Stretch Goal III: Bluetooth	Not achieved	Deferred due to priority on core objectives
Stretch Goal IV: Leak localization	Not achieved	Beyond current project scope
Stretch Goal V: YAMNet benchmarking	Achieved	Benchmarking completed against NEAI

Overall, this project has demonstrated the feasibility of an edge AI based low-power water leak detection system that combines acoustic sensing, vibration sensing, embedded intelligence, periodic low-power operation, wireless communication, and prototype mechanical integration into a single proof-of-concept platform. The results show that leak detection can be performed locally on a resource-constrained STM32 microcontroller with meaningful performance under controlled conditions. More importantly, the project shows that practical embedded leak

detection depends not only on the AI model itself, but also on the quality of the dataset, the decision logic, the sensing configuration, the power-management strategy, the communication method, and the physical prototype design. Although further refinement is still required before real deployment, the work has achieved its core purpose and established a credible foundation for future development of compact, low-power, and field-oriented smart water leak detection systems.

5.5 Future Improvement

Although the developed prototype has demonstrated the feasibility of embedded leak detection under controlled conditions, several areas remain for future improvement in order to strengthen the system's realism, deployability, and practical performance.

The first important direction is to expand the diversity of the simulation rig and dataset conditions. In the present study, the simulation rig was sufficient for controlled comparative testing, but future work should broaden the range of physical variables represented during data collection. This includes incorporating different leak forms, such as cracks, seal-related leakage, and other irregular defect shapes, in addition to circular holes. The pipe network geometry should also be made more representative by including bends, loops, branches, and other structural features that can alter signal propagation. Further variation in pipe material, pipe diameter, and water flow rate would also improve the realism of the collected signals. Such expansion would allow the detection system to be evaluated under conditions that are closer to practical water distribution environments. At the same time, these additions should be introduced in a controlled and well-documented way so that data consistency is preserved and the effect of each variable can still be interpreted clearly.

A second area of improvement concerns the mechanical design of the prototype holder. The current holder is suitable for the specific pipe configuration used in this study, but future versions should adopt a more flexible mounting mechanism so that the device can be attached securely to pipes of different diameters. This would improve the adaptability of the system for wider testing and future deployment. In addition, the enclosure should be further developed into a more protective waterproof design to improve environmental robustness. This is particularly important if the device is to be used outside controlled laboratory environment. The present microphone opening is also a simple exposed hole, and future work may explore reshaping this region into a structure that provides mechanical acoustic amplification or improved sound guidance. Such refinement may help improve acoustic sensitivity while maintaining enclosure protection.

Another major improvement is the development of a custom hardware board. While the present project successfully demonstrated the intended embedded operating sequence and low-power concept, a dedicated board would be needed to fully realize the low-power objective in practice. A future design should include only the necessary components required for this application, such as the MCU, microphone, accelerometer, LoRa communication module, and an efficient power-management circuit. Removing unnecessary components would provide a much more realistic platform for long-term battery-powered deployment.

Wireless transmission performance could also be further improved through antenna refinement. The current LoRa setup was sufficient for the proof of concept, but future versions could use a better antenna or a more suitable antenna placement to achieve longer transmission range and more reliable communication. This would be beneficial for larger deployment scenarios, especially where obstacles, longer distances, or more complex installation environments are expected.

Overall, these improvements would move the current prototype from a controlled experimental prototype toward a more robust and deployment-oriented leak detection system. Future work should therefore focus not only on improving individual subsystems, but also on ensuring that sensing performance, mechanical design, hardware efficiency, and wireless reliability are improved together as one integrated engineering solution.

5.6 Reflection on Project Management

This project gave several useful lessons in project management, especially in hardware planning, time management, budget use, and communication with external parties. Although the main technical objectives were mostly achieved, the overall progress of the project was affected not only by engineering challenges, but also by how well the resources, schedule, and coordination were managed.

One important lesson was related to hardware procurement. The STM32L562E-DK board was ordered on 22 October 2025 but was only received on 8 April 2026, which was already close to the project deadline. Because of this delay, the board could not be used meaningfully in the main development work. However, as identified earlier in the risk management plan, the backup B-U585I-IOT02A board was available and allowed the project to continue successfully. This showed that the backup-board strategy was effective, but it also highlighted the importance of checking stock availability, delivery source, and expected shipping time more carefully before making a purchase, especially when overseas delivery and customs clearance may be involved.

Another lesson was related to time and resource management during experimental work. Dataset collection was carried out over many separate sessions instead of being completed in one more organized phase. Although this allowed the model to be improved gradually, it also meant that the simulation rig and recording setup had to be prepared repeatedly, which used more time than expected. In addition, the project budget was used quite conservatively, with much of the spending focused on the STM32L562E-DK board. Looking back, it would have been better to spend more of the budget on improving the simulation rig, since a better rig could have supported better dataset quality and stronger model validation.

The project also showed the importance of earlier communication with the company. More detailed information about the intended system design, including the preference for the STM32L series board, ER34625 battery, and SPC1520 pulse-assist capacitor, was only clarified later in the project. Earlier discussion on these technical requirements would have made planning more consistent and reduced the need for later adjustments.

Finally, this project also showed the importance of focusing on the main deliverables first and writing more consistently throughout the project. Some parts of the work, such as debugging, decision-logic refinement, dataset expansion, and validation, took more time than originally expected. This made it necessary to prioritize the core objectives over some stretch goals. At the same time, much of the thesis writing was completed near the later stage of the project, which increased the workload close to the deadline. A better approach would have been to write and update sections more steadily alongside the technical progress.

Overall, this project showed that completing an engineering project successfully depends not only on solving technical problems, but also on planning purchases carefully, managing time well, using resources effectively, and maintaining clear communication from the beginning.

6. References

- [1] W. B. GROUP, "The World Bank and the International Water Association to Establish a Partnership to Reduce Water Losses," 1 September 2016. [Online]. Available: <https://www.worldbank.org/en/news/press-release/2016/09/01/the-world-bank-and-the-international-water-association-to-establish-a-partnership-to-reduce-water-losses>. [Accessed 11 October 2025].
- [2] R. L. P. M. Bill Kingdom, "The Challenge of Reducing Non-Revenue Water (NRW) in Developing Countries," *How the Private Sector Can Help: A Look at Performance-Based Service Contracting*, p. 2, 2006.
- [3] Damian, "Comparing Traditional vs. Modern Water Leak Detection Methods," Wizard Leak Detection, 16 June 2025. [Online]. Available: <https://www.wizardleakdetection.com.au/comparing-traditional-vs-modern-water-leak-detection-methods/>. [Accessed 11 October 2026].
- [4] M. R. Islam, S. Azam, B. Shanmugam and D. Mathur, "An Intelligent IoT and ML-Based Water Leakage Detection System," *IEEE Access*, 2023.
- [5] S. M. Ekolama, "Application of Artificial Intelligence (AI) Model to Mitigate Security threats of Internet of Things (IoT): A Review," *Journal of Engineering, Emerging Technologies and Applied Sciences*, vol. 1, no. 2, pp. 88-96, 2023.
- [6] S. E. Najm Aleessawi, "Harnessing AI and IoT for Advancing Sustainable Development Methods," *Journal of the Association of Arab Universities for Higher Education Research (JAARU-RHE)*, vol. 45, no. 3, pp. 333-346, 2025.
- [7] Anon, "Water Loss Management," SPML, 2018. [Online]. Available: <https://www.spml.co.in/post/water-loss-management>. [Accessed 11 October 2025].
- [8] F. a. C. R. a. N. L. a. R. M. a. P. P. Haque, *EDGE COMPUTING FOR DATA-DRIVEN AI MODELS*, 2019.
- [9] J. Greg, "CRITICAL INFRASTRUCTURE PROTECTION IN SMART CITIES (ENERGY, WATER, TRANSPORT)," pp. 1-8, 2025.
- [10] W. L. Detection, "How Do Acoustic Water Leak Detectors Work?," *Water Leak Detection*, 27 September 2020. [Online]. Available: <https://www.waterleakdetection.net.au/how-do-acoustic-water-leak-detectors-work>. [Accessed 11 October 2025].
- [11] A. M. N. B. M. Z. Othmane Atanane, "Smart Buildings: Water Leakage Detection Using TinyML," *Sensors*, vol. 23, pp. 1-17, 2023.

- [12] U. Nations, "Ensure availability and sustainable management of water and sanitation for all," United Nations, [Online]. Available: <https://sdgs.un.org/goals/goal6>. [Accessed 14 April 2026].
- [13] L. D. A. E. S. S. Pierpaolo Dini, "Overview of AI-Models and Tools in Embedded IIoT Applications," *Electronics*, vol. 13, no. 12, 2024.
- [14] J.-L. Aufranc, "STM32U5 Cortex-M33 MCU gets more performance, 2D graphics accelerator, and advanced security," CNX Software, 27 February 2021. [Online]. Available: <https://www.cnx-software.com/2021/02/27/stm32u5-cortex-m33-mcu-gets-more-performance-2d-graphics-accelerator-and-advanced-security/>. [Accessed 1 March 2026].
- [15] STMicroelectronics, "STM32U585xx," STMicroelectronics, 2024.
- [16] W. D. H. L. K. Z. Y. L. Y. H. Z. H. Niannian Wang, "Fine-Grained Leakage Detection for Water Supply Pipelines Based on CNN and Selective State-Space Models," *Water*, vol. 17, no. 8, 2025.
- [17] R. Lauer, "Create a Self-Training Edge AI Solution on STM32 with NanoEdge AI Studio," blues, 17 July 2024. [Online]. Available: <https://dev.blues.io/blog/edge-ai-stm32-nanoedge-ai-studio/>. [Accessed 19 October 2025].
- [18] STMicroelectronics, "AI solution for leakage detection with Altyor," STMicroelectronics, [Online]. Available: https://www.st.com/content/st_com/en/st-edge-ai-suite/case-studies/ai-solution-for-leakage-detection-with-altyor.html. [Accessed 19 October 2025].
- [19] D. o. O. S. a. H. M. o. H. R. Malaysia, "HIRARC," 2008.
- [20] W. Winarni, "Infrastructure Leakage Index (ILI) as Water Losses Indicator," *Civil Engineering Dimension*, vol. 11, no. 2, pp. 126-134, 2009.
- [21] R. L. P. M. Bill Kingdom, "The Challenge of Reducing Non-Revenue Water (NRW) in Developing Countries How the Private Sector Can Help: A Look at Performance-Based Service Contracting," December 2006. [Online]. Available: <https://documents1.worldbank.org/curated/en/385761468330326484/pdf/394050Reducing1e0water0WSS81PUBLIC1.pdf>. [Accessed 9 April 2026].
- [22] R. L. Rudolf Frauendorfer, "The Issues and Challenges of Reducing Non-Revenue Water," Asian Development Bank, Mandaluyong City, Philippines, 2010.
- [23] EPA, "Control and Mitigation of Drinking Water Losses in Distribution Systems," November 2010. [Online]. Available: <https://www.epa.gov/sites/default/files/2015-05/documents/epa816r10019.pdf>. [Accessed 9 April 2026].

- [24] N. H. S. P. G. T. O. Wulan Seizarwati, "Impacts of water supply leakage and inter-basin water transfer on groundwater dynamics in Greater Jakarta," *Journal of Hydrology: Regional Studies*, vol. 64, 2026.
- [25] S. T. Malcolm Farley, *Losses in Water Distribution Networks*, London: IWA Publishing, 2003.
- [26] Y. W. H. Z. X. J. Jingxin Mao, "Study on the Influence of Urban Water Supply Pipeline Leakage on the Scouring Failure Law of Cohesive Soil Subgrade," *Water*, vol. 16, no. 1, 2023.
- [27] A. N. A. H. M. Edi Septea, "A Review of Corrosion Assessment Model and Parameters of Drinking Water Distribution Pipelines," *Jurnal Teknologi*, 2014.
- [28] N. T. F. R. D. G. K. A. L. Athanasios V. Serafeim, "Leakages in Water Distribution Networks: Estimation Methods, Influential Factors, and Mitigation Strategies - A Comprehensive Review," *Water*, vol. 16, no. 11, 2024.
- [29] T. Z. Samer El-Zahab, "Leak detection in water distribution networks: an introductory overview," *Smart Water*, vol. 4, 2019.
- [30] A. W. W. Association, "Water Loss Control," American Water Works Association, [Online]. Available: <https://www.awwa.org/resource/water-loss-control/>. [Accessed 9 April 2026].
- [31] K. A. K. S. R. v. S. P. W. J. C. S. S. Joseph, "Application of Software and Hardware-Based Technologies in Leaks and Burst Detection in Water Pipe Networks: A Literature Review," *Water*, vol. 15, no. 11, 2023.
- [32] M. I. a. D. R. a. A. N. a. M.-S. F. a. B. N. a. M. I. M. A. Mohd Ismail, "A Review of Vibration Detection Methods Using Accelerometer Sensors for Water Pipeline Leakage," *IEEE Access*, 2019.
- [33] S. T. T. Z. Zhongyu Hu, "A comprehensive review of acoustic based leak localization method in pressurized pipelines," *Mechanical systems and signal processing*, vol. 161, 2021.
- [34] I. T. a. S. A. a. A. F. a. K. F. a. Z. H. a. T. Zayed, "Improving the leak detection efficiency in water distribution networks using noise loggers," *Science of The Total Environment*, vol. 821, 2022.
- [35] Y. G. a. M. B. a. P. J. a. J. M. a. O. Hunaidi, "On the selection of acoustic/vibration sensors for leak detection in plastic water pipes," *Journal of Sound and Vibration*, vol. 283, pp. 927-941, 2005.
- [36] B. B. T. Z. Salman Tariq, "Data-driven application of MEMS-based accelerometers for leak detection in water distribution networks," *Science of The Total Environment*, vol. 809, pp. 1-17, 2022.

- [37] B. K. Suan Lee, "Machine Learning Model for Leak Detection Using Water Pipeline Vibration Sensor," *sensors*, vol. 2, 2023.
- [38] W. T. C. Osama Hunaidi, "Acoustical characteristics of leak signals in plastic water distribution pipes," *Applied Acoustics*, vol. 58, pp. 235-254, 1999.
- [39] O. B. M. J. d. A. F. C. L. I. M. K. M. J. M. J. P. F. & G. Y. Scussel, "Key Factors That Influence the Frequency Range of Measured Leak Noise in Buried Plastic Water Pipes: Theory and Experiment," *Acoustics*, vol. 5, no. 2, 2023.
- [40] R. L. a. T. Z. a. R. Xiao, "Advanced acoustic leak detection in water distribution networks using integrated generative model," *Water Research*, vol. 254, 2024.
- [41] R. Z. T. & X. R. Liu, "Contrastive learning method for leak detection in water distribution networks," *Clean Water*, vol. 118, 2024.
- [42] E. a. B.-T. D. a. B.-B. R. Martinez-Ríos, "Water Leakage Classification With Acceleration, Pressure, and Acoustic Data: Leveraging the Wavelet Scattering Transform, Unimodal Classifiers, and Late Fusion," *IEEE Access*, 2024.
- [43] H. F. a. S. T. a. T. Zayed, "Acoustic leak detection approaches for water pipelines," *Automation in Construction*, vol. 138, 2022.
- [44] B. B. a. T. Zayed, "Review of Water Leak Detection and Localization Methods through Hydrophone Technology," *Pipeline systems engineering and practice*, vol. 12, 2021.
- [45] T. a. A. C. a. M. L.-P. Baltrušaitis, "Multimodal Machine Learning: A Survey and Taxonomy," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, pp. 423-443, 2019.
- [46] P. A. S. S. B. I. L. M. P. Huang Shih-Cheng, "Fusion of medical imaging and electronic health records using deep learning: a systematic review and implementation guidelines," *npj Digital Medicine*, vol. 3, no. 1, 2020.
- [47] S. a. D. R. a. T. V. Chhabria, "Survey of Fusion Techniques for Design of Efficient Multimodal Systems," in *2013 International Conference on Machine Intelligence and Research Advancement*, 2013, pp. 486-492.
- [48] X. T. a. J. L. a. C. W. a. Y. Liu, "A water pipe leakage detection method based on Multi-view Embedding Distance Feature Fusion Network," *Water Research*, 2025.
- [49] F. D. R. He HUANG, "All On-Device anomaly detection in NanoEdge AI Studio," [Online]. Available: https://cms.tinymml.org/wp-content/uploads/ew2023/He-Huang_final.pdf. [Accessed 11 April 2026].

- [50] R. L. a. T. Z. a. R. Xiao, "A critical systematic review of machine learning based models for water leak detection using vibroacoustic technology," *Engineering Applications of Artificial Intelligence*, vol. 158, 2025.
- [51] J. W. a. C. Z. a. J. Q. a. X.-C. Z. a. Z. B. a. D. L. a. S. Z. a. Q. Wang, "An optimized VMD-wavelet denoising method for leakage detection in water supply networks from acoustic emission signals," *International Journal of Pressure Vessels and Piping*, vol. 217, 2025.
- [52] C. W. a. X. C. a. Y. X. a. W. Y. a. Y. Y. a. Y. S. a. T. Yu, "A two-stage leak monitoring framework for water distribution networks based on acoustic signals," *Mechanical Systems and Signal Processing*, vol. 225, 2025.
- [53] F. a. A. Z. a. K. J.-M. Saleem, "Real-Time Pipeline Leak Detection: A Hybrid Deep Learning Approach Using Acoustic Emission Signals," *Applied Sciences*, vol. 15, 2025.
- [54] T. A. a. L. M. a. D. L. a. W. L. a. H. Luo, "Feature extraction for acoustic leakage detection in water pipelines," *Automation in Construction*, vol. 176, 2025.
- [55] S. a. A. Z. a. K. C.-H. a. K. J.-M. Ahmad, "A Method for Pipeline Leak Detection Based on Acoustic Imaging and Deep Learning," *Sensors*, vol. 22, 2022.
- [56] U. a. K. M. a. P. C. H. Rajasekaran, "Novel EMD with Optimal Mode Selector, MFCC, and 2DCNN for Leak Detection and Localization in Water Pipeline," *Applied Sciences*, vol. 13, 2023.
- [57] P. a. H. J. a. H. W. a. Z. J. a. Y. Y. a. C. B. a. Y. Z. a. X. Y. a. Y. Y. a. W. C. a. C. H. a. Z. L. Zhang, "Water Pipeline Leak Detection Based on a Pseudo-Siamese Convolutional Neural Network: Integrating Handcrafted Features and Deep Representations," *Water*, vol. 15, 2023.
- [58] J. L. A. B. J. L. E. K. Anthony Bagnall, "The great time series classification bake off: a review and experimental evaluation of recent algorithmic advances," *Data Mining and Knowledge Discovery*, vol. 31, 2017.
- [59] STMicroelectronics, "Datalogging guidelines for a successful NanoEdge AI project," STMicroelectronics, 1 December 2023. [Online]. Available: https://wiki.st.com/stm32mcu/wiki/AI%3ADatalogging_guidelines_for_a_successful_NanoEdge_AI_project. [Accessed 12 April 2026].
- [60] S. S. a. S. S. a. S. M. Saha, "Machine Learning for Microcontroller-Class Hardware: A Review," *IEEE Sensors Journal*, vol. 22, pp. 21362-21390, 2022.
- [61] Y. W. G. G. S. L. X. W. Y. X. J. F. H. W. L. X. Xingke Ma, "Acoustic Leak Detection in Water Networks: A Two-Stage Temporal Segmentation Approach for Non-Stationary Signals," The University of Sheffield, 2025.

- [62] M. a. E. M. J. a. J. A. Essouabni, "Multi-Class Leak Detection in Water Pipelines Using a Wavelet-Guided Frequency-Informed Transformer," *Applied System Innovation*, vol. 9, 2026.
- [63] R. a. I. S. a. R. F. a. S. T. a. P. C. a. O. J. a. L.-P. C. Müller, "Acoustic Leak Detection in Water Networks," in *Proceedings of the 13th International Conference on Agents and Artificial Intelligence*, SCITEPRESS - Science and Technology Publications, 2021, p. 306–313.
- [64] N. a. M. L. a. T. C. W. a. W. G. I. a. F. G. a. S. M. Mohammadi Foumani, "Deep Learning for Time Series Classification and Extrinsic Regression: A Current Survey," *ACM Computing Surveys*, vol. 56, no. 9, 2024.
- [65] D. U. a. M. S. a. B. P. a. M. M. a. T. S. Leonzio, "Water Leak Detection and Classification With Complex-Valued Neural Networks and Sensor Fusion," *IEEE Access*, vol. 13, pp. 115669-115685, 2025.
- [66] P. P. Ray, "A review on TinyML: State-of-the-art and prospects," *Journal of King Saud University - Computer and Information Sciences*, vol. 34, pp. 1595-1623, 2022.
- [67] M. S. a. B. { . A. a. P. R. a. V. Raja, "Integrating machine learning models and acoustic signals for water pipeline leakage localization," *Ain Shams Engineering Journal*, vol. 17, 2026.
- [68] L. T. a. L. B. a. R. R. a. J. L. a. Z. L. a. A. Z. a. X. Kong, "Acoustic water pipe leak detection using transformer-based variational autoencoder," *NDT & E International*, vol. 160, 2026.
- [69] J. D. A. J. V. J. R. N. J. J. L. N. K. I. N. M. N. S. R. R. R. T. W. P. W. Robert David, "TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems," *CoRR*, 2022.
- [70] S. a. I. M. M. a. C. G. a. C. R. a. M. M. S. a. S. S. a. I. K. a. A. J. S. a. R. S. A. a. D. A. a. D. S. Somvanshi, "From Tiny Machine Learning to Tiny Deep Learning: A Survey," 2025.
- [71] B. a. K. S. a. C. B. a. Z. M. a. T. M. a. H. A. a. A. H. a. K. D. Jacob, "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704-2713.
- [72] N. a. K. A. a. K. C. a. S. S. Schizas, "TinyML for Ultra-Low Power AI and Large Scale IoT Deployments: A Systematic Review," *Future Internet*, vol. 14, no. 12, 2022.
- [73] S. a. M. Q. H. Heydari, "Tiny Machine Learning and On-Device Inference: A Survey of Applications, Challenges, and Future Directions," *Sensors*, vol. 25, 2025.

- [74] W.-M. C. Y. j. L. J. C. C. G. S. H. Ji Lin, "MCUNet: Tiny Deep Learning on IoT Devices," *CoRR*, 2020.
- [75] STMicroelectronics, "Automated Machine Learning (ML) tool for STM32 developers," STMicroelectronics, [Online]. Available: <https://www.st.com/en/development-tools/nanoedgeaistudio.html>. [Accessed 13 April 2026].
- [76] STMicroelectronics, "NanoEdge AI Studio," STMicroelectronics, 29 July 2025. [Online]. Available: https://wiki.st.com/stm32mcu/wiki/AI:NanoEdge_AI_Studio. [Accessed 19 October 2025].
- [77] STMicroelectronics, "How to Build an Anomaly Detection Project for Predictive Maintenance with NanoEdge AI Studio," STMicroelectronics, 1 December 2023. [Online]. Available: https://wiki.st.com/stm32mcu/wiki/AI%3ADatalogging_guidelines_for_a_successful_NanoEdge_AI_project. [Accessed 13 April 2026].
- [78] E. a. B.-C. P. a. M.-C. F. a. T. A. a. C.-T. R. Hidalgo-Fort, "Low-Cost, Low-Power Edge Computing System for Structural Health Monitoring in an IoT Framework," *Sensors*, vol. 24, no. 15, 2024.
- [79] F. A. a. M. H. a. A. Abuarqou, "A Survey on MAC Protocols for Duty-cycled Wireless Sensor Networks," *Procedia Computer Science*, vol. 73, pp. 482-489, 2015.
- [80] S. S. a. V. A. a. P. K. Bera, "A Delay-Tolerant low-duty cycle scheme in wireless sensor networks for IoT applications," *International Journal of Cognitive Computing in Engineering*, vol. 4, pp. 194-204, 2023.
- [81] M. a. F.-H. D. Pointl, "Assessing the Potential of LPWAN Communication Technologies for Near Real-Time Leak Detection in Water Distribution Systems," *Sensors*, vol. 21, no. 1, 2021.
- [82] F. C. a. F. M. a. M. M. a. T. E. a. Z. V. Andriulo, "Edge Computing and Cloud Computing for Internet of Things: A Review," *Informatics*, vol. 11, no. 4, 2024.
- [83] D. O. N. E. H. M. Diane Ass, "A systematic and comprehensive review on low power wide area network: characteristics, architecture, applications and research challenges," *Discover Internet of Things*, vol. 5, 2025.
- [84] N. N. a. I. D. M. Alajlan, "TinyML: Enabling of Inference Deep Learning Models on Ultra-Low-Power IoT Edge Devices for AI Applications," *Micromachines*, vol. 13, no. 6, 2022.
- [85] E. F. K. M. M. A. m. Atheer M Alghamdi, "LoRaWAN Performance Analysis for a Water Monitoring and Leakage Detection System in a Housing Complex.," *Sensors*, vol. 22, no. 19, 2022.

- [86] N. S. B. Solomon Ould, "Energy Performance Analysis and Modelling of LoRa Prototyping Boards," *Sensors*, vol. 21, no. 23, 2021.
- [87] R. a. N. P. a. S. M. Berto, "A LoRa-Based Mesh Network for Peer-to-Peer Long-Range Communication," *Sensors*, vol. 21, no. 13, 2021.
- [88] G. KENT, "Corporate Profile," GEORGE KENT, [Online]. Available: <https://georgekent.net/corporate-profile/>. [Accessed 19 October 2025].
- [89] D. Isamudin, "George Kent, China's Qingdao to develop Malaysia's first ultrasonic water meter," in *New Straits Times*, 2025.
- [90] C. Bolgar, "AI tool uses sound to pinpoint leaky pipes, saving precious drinking water," Microsoft, 19 September 2024. [Online]. Available: <https://news.microsoft.com/source/features/sustainability/ai-tool-uses-sound-to-pinpoint-leaky-pipes-saving-precious-drinking-water/>. [Accessed 19 October 2025].
- [91] L. Space, "Tackling Water Leakage from Space," Lacuna Space, 1 July 2025. [Online]. Available: <https://lacuna-space.com/tackling-water-leakage-from-space/>. [Accessed 19 October 2025].
- [92] STMicroelectronics, "MP23DB01HP," STMicroelectronics, [Online]. Available: <https://www.st.com/resource/en/datasheet/mp23db01hp.pdf>. [Accessed 4 March 2026].
- [93] P. Vaidyanathan, "Generalizations of the sampling theorem: Seven decades after Nyquist," *IEEE Transactions on Circuits and Systems I: Fundamental Theory and Applications*, vol. 48, pp. 1094-1109, 2001.
- [94] STMicroelectronics, "STM32U5 Series power optimization using LPBAM," [Online]. Available: https://www.st.com/resource/en/application_note/an5645-stm32u5-series-power-optimization-using-lpbam-stmicroelectronics.pdf. [Accessed 4 March 2026].
- [95] STMicroelectronics, "ISM330DHCX," [Online]. Available: <https://www.st.com/resource/en/datasheet/ism330dhcx.pdf>. [Accessed 4 March 2026].
- [96] STMicroelectronics, "LSM6DSM: always-on 3D accelerometer and 3D gyroscope," [Online]. Available: https://www.st.com/resource/en/application_note/dm00352102-lsm6dsm-alwayson-3d-accelerometer-and-3d-gyroscope-stmicroelectronics.pdf. [Accessed 4 March 2026].
- [97] STMicroelectronics, "Parameters and calibration of a low-g 3-axis accelerometer," [Online]. Available: https://www.st.com/resource/en/application_note/an4508-parameters-and-calibration-of-a-lowg-3axis-accelerometer-stmicroelectronics.pdf. [Accessed 4 March 2026].

- [98] Ai-Thinker, "Ra-08H-Kit Specification V1.2.0," [Online]. Available: <https://www.lcsc.com/datasheet/C2991930.pdf>. [Accessed 9 March 2026].
- [99] DigiKey, "RA-08H," DigiKey, [Online]. Available: <https://www.digikey.my/en/products/detail/ai-thinker/RA-08H/16688823>. [Accessed 15 March 2026].
- [100] Ai-Thinker-Open, "Ai-Thinker-LoRaWAN-Ra-08," Ai-Thinker-Open, [Online]. Available: <https://github.com/Ai-Thinker-Open/Ai-Thinker-LoRaWAN-Ra-08>. [Accessed 16 April 2026].
- [101] ANYWAVES, "What is Antenna Gain in Satellite Communications? (Explained simply)," ANYWAVES, 19 December 2025. [Online]. Available: <https://anywaves.com/resources/blog/what-is-antenna-gain-satellite-communications/>. [Accessed 23 March 2026].
- [102] Dolph, "Which type of antenna gives highest gain," Dolph Microwave, 13 April 2024. [Online]. Available: <https://dolphmicrowave.com/default/which-type-of-antenna-gives-highest-gain/>. [Accessed 23 March 2026].
- [103] E. GigaByte, "Antenna Gain Calculator," Electronics GigaByte, [Online]. Available: <https://electronicsgb.com/antenna-gain-calculator/>. [Accessed 23 March 2026].
- [104] admin, "ER34615+1530, ER34615 SPC1520, and ER34615 with HPC1550 ER34615+1550 3.6V D Size Batteries For Smart water meter IoT device," Guangzhou Ser Battery Technology, 29 September 2024. [Online]. Available: <https://www.serui-battery.com/News/gongsidongtai/er34615-1530--er34615-spc1520-and-er34615-with-hpc1550-er34615-1550-3-6v-d-size-batteries-for-smart-water-meter-iot-device.html>. [Accessed 15 March 2026].
- [105] EVE, "ER34615," EVE Energy, [Online]. Available: https://panda-bg.com/resources/prod_8759_2767-360935-lithium-cylindrical-battery-eve-er34615cnr.pdf. [Accessed 15 March 2026].
- [106] W. Henderson, "PVC Pipe Sizes: A Complete Guide for Plumbing and Industrial Use," Audit Bureau of Circulations Malaysia, [Online]. Available: <https://abcm.org.my/a-complete-guide-to-pvc-pipe-sizes-for-plumbing-and-industrial-use/>. [Accessed 2 March 2026].
- [107] JKR, "Internal Cold Water and Sanitary Plumbing System," 1 February 2016. [Online]. Available: https://crr.kkr.gov.my/system/files/2024-02/60%20JKR20500-0048-17-CWS_Specification.pdf. [Accessed 2 March 2026].
- [108] HBK, "Avoiding Vibration Measurement Errors," HBK, [Online]. Available: <https://www.hbkworld.com/en/knowledge/resource-center/articles/avoid-errors-using-accelerometers>. [Accessed 2 March 2026].

- [109] J. H. E. C. Kenton Hummel, "Mounted Accelerometer Frequency Response of Adhesive Products and Aluminum Frame Quick Mounts," *Vibration*, 2025.
- [110] Knowles, "SiSonic Design Guide," [Online]. Available: https://media.digikey.com/pdf/data%20sheets/knowles%20acoustics%20pdfs/sisonic_design_guide.pdf. [Accessed 2 March 2026].
- [111] Laird, "Laird RM1262 LoRaWAN Module User Guide," [Online]. Available: <https://manuals.plus/m/ce2446e4434ed7ee34da0caa83ee13faa8f359bf137c0d5b549d59bdbcedcb14>. [Accessed 2 March 2026].
- [112] STMicroelectronics, "STM32Cube AI Studio documentation," STMicroelectronics, 19 February 2026. [Online]. Available: https://wiki.st.com/stm32mcu/wiki/AI:STM32Cube_AI_Studio_documentation. [Accessed 8 April 2026].
- [113] P. a. P. A. a. C. Y. Giannakopoulos, "Improving Post-Processing of Audio Event Detectors Using Reinforcement Learning," *IEEE Access*, vol. 10, pp. 84398-84404, 2022.
- [114] Z. B. M. A. M. E. Manel Boujelben, "An efficient system for water leak detection and localization based on IoT and lightweight deep learning," *Internet of Things*, vol. 24, 2023.
- [115] Google, "Classification: Accuracy, recall, precision, and related metrics," Google, [Online]. Available: <https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall>. [Accessed 24 March 2026].
- [116] s. learn, "3.4. Metrics and scoring: quantifying the quality of predictions," scikit learn, [Online]. Available: https://scikit-learn.org/stable/modules/model_evaluation.html. [Accessed 25 March 2026].
- [117] STMicroelectronics, "Discovery kit for IoT node with STM32U5 series," STMicroelectronics, [Online]. Available: https://www.st.com/resource/en/user_manual/um2839-discovery-kit-for-iot-node-with-stm32u5-series-stmicroelectronics.pdf. [Accessed 15 March 2026].
- [118] S. C. D. C. Y. H. Ping Lu, "Estimation of the characteristics of wave propagation in the water pipe using continuous wavelet transform.," *Sci Rep*, vol. 15, 2025.
- [119] H. D. S. Z. Wenjun Wu, "Scattering of guided waves propagating through pipe bends based on normal mode expansion," *Sci Rep*, vol. 12, 2022.
- [120] N. Hamamed, C. Mechri, T. Mhammedi, N. Yaakoubi, R. El Guerjouma, S. Bouaziz and M. Haddar, "Comparative Study of Leak Detection in PVC Water Pipes Using Ceramic, Polymer, and Surface Acoustic Wave Sensors.," *Sensors*, vol. 23, 2023.

- [121] H. W. S. Q. S.-K. T. Xiaoyuan Huang, "Embedded Artificial Intelligence: A Comprehensive Literature Review," *Electronics*, vol. 14, 2025.
- [122] R. A. M. S. & P. K. S. Mahendra Kumar Gourisaria, "Comparative analysis of audio classification with MFCC and STFT features using machine learning techniques," *Discover Internet of Things*, vol. 4, 2024.
- [123] C. Yan, "A review on spectral data preprocessing techniques for machine learning and quantitative analysis," *iScience*, vol. 28, no. 7, 2025.
- [124] STMicroelectronics, "How to optimize power consumption on STM32U5 MCUs," [Online]. Available: https://www.st.com/resource/en/application_note/an5652-how-to-optimize-power-consumption-on-stm32u5-mcus-stmicroelectronics.pdf. [Accessed 11 March 2026].
- [125] GORE, "GORE® Acoustic Vents," GORE, [Online]. Available: <https://www.gore.co.uk/node/27286>. [Accessed 2 March 2026].

Appendices

Appendix A: Turnitin similarity score.

TurnitinUK Originality Report

Processed on: 18-Apr-2026 3:04 PM BST
ID: 280137903
Word Count: 40519
Submitted: 1

Thesis By Jing Siang Cheng

[Document Viewer](#)

Similarity Index	Similarity by Source
11%	Internet Sources: 9% Publications: 7% Student Papers: 6%

exclude quoted

exclude bibliography

exclude small matches

mode: quickview (classic) report

print

download

1% match (student papers from 23-Oct-2025)
[Submitted to University of Nottingham on 2025-10-23](#)

<1% match (student papers from 23-Oct-2025)
[Submitted to University of Nottingham on 2025-10-23](#)

<1% match (student papers from 22-Oct-2025)
[Submitted to University of Nottingham on 2025-10-22](#)

<1% match (student papers from 23-Oct-2025)
[Submitted to University of Nottingham on 2025-10-23](#)

<1% match (student papers from 28-Apr-2024)
[Submitted to University of Nottingham on 2024-04-28](#)

<1% match (student papers from 06-Sep-2024)
[Submitted to University of Nottingham on 2024-09-06](#)

<1% match (student papers from 24-Feb-2026)
[Submitted to University of Nottingham on 2026-02-24](#)

<1% match (student papers from 23-Oct-2025)
[Submitted to University of Nottingham on 2025-10-23](#)

<1% match (Internet from 25-Feb-2026)
<https://www.mdpi.com/2571-5577/9/2/47>

<1% match (Internet from 06-Jan-2026)

97

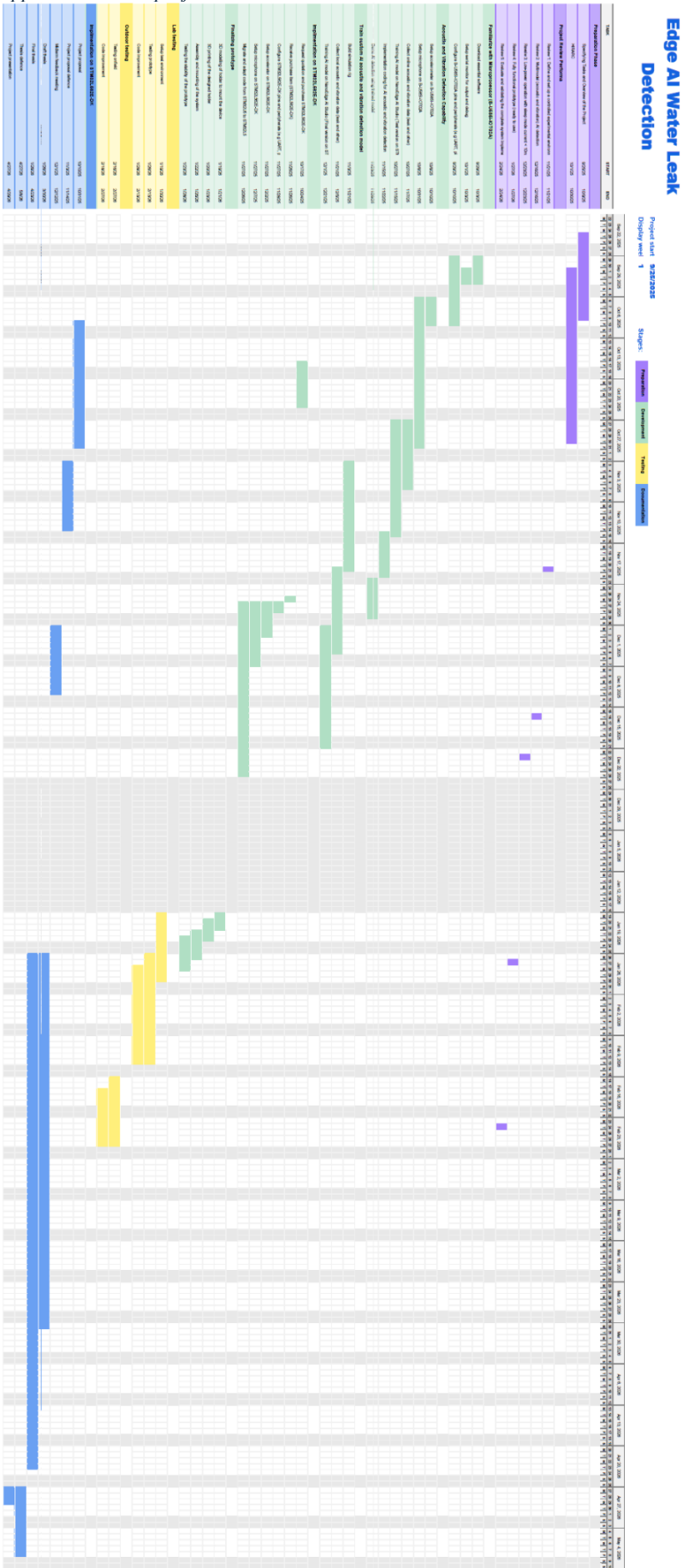
Appendix B: Project description.

Project Title	Low-Power Edge AI Acoustic Water Leak Detection System								
Project Code	HN-MEng-25-01								
Supervisor	Dr Hermawan								
Moderator	Prof Dr Haider Abbas								
Project Description	The project partner is GeorgeKent Sdn Bhd and the aim is to develop a low-power edge artificial intelligence system that uses acoustic signal processing to identify water leaks in piping infrastructure. The system will need to use a microcontroller with ultra-low deep sleep current consumption ($\leq 19\text{mA}$) to enable continuous monitoring with minimal power requirements. By processing acoustic data locally on the edge device without requiring cloud connectivity, the system ensures real-time leak detection while maintaining operational efficiency suitable for deployment in smart water management systems. A microphone sensor captures sound signals from water pipes, which are analyzed by an optimized lightweight neural network trained to distinguish between leak and non-leak acoustic patterns, delivering immediate detection results on the microcontroller hardware.								
Project Objectives	Research Objectives: 1. Collect, preprocess, and validate acoustic dataset representing leak and non-leak conditions 2. Develop and optimize a neural network model for edge deployment (size $\leq 500\text{KB}$, accuracy $\geq 80\%$) 3. Implement AI model on microcontroller meeting power constraints ($\leq 19\text{mA}$ deep sleep) 4. Integrate components into functional prototype and validate real-time leak detection performance								
Project Deliverables	1. Acoustic Dataset & Preprocessing Pipeline • Data collection and labeling protocol documentation 2. Optimized Edge AI Model • Neural network with $\geq 80\%$ accuracy on validation set • Model architecture specification 3. Embedded Firmware • Microcontroller code for audio acquisition and AI inference • Power management system ($\leq 19\text{mA}$ specification) 4. Functional Prototype Device • Integrated microphone, microcontroller, power management, alert interface 5. Technical Documentation & Validation Report • System design documentation • Performance metrics (accuracy, latency, power consumption, false positives/negatives) • Deployment guidelines and user manual								
Ratio of HW/SW/Research	<table border="1"> <tr> <th>HW</th><th>SW</th><th>Research/Investigation/Design</th></tr> <tr> <td>50%</td><td>30%</td><td>20%</td></tr> </table>	HW	SW	Research/Investigation/Design	50%	30%	20%		
HW	SW	Research/Investigation/Design							
50%	30%	20%							
Lab Space Requirements	☐ Fixed Space ☐ Ad-hoc Space ☐ Others.								
Software (This is software usually supported in the Software Lab)									

Prepared by: Dr Hermawan
 Date: 17 October 2025
 (Supervisor)

Reviewed by: Haider
 Date: 20 Oct 2025
 (Moderator)

Appendix C: Ideal project Gantt chart.





2025-2026

Student Name(s) : Cheng Jing Siang
Project Title : Edge AI low-power water leak detection
Supervisor(s) : Dr Hermawan Nugroho
Date of meeting : 6/11/2025 **Length of meeting** : 1 hour

Progress made since the last meeting

- Initialized microphone, accelerometer , LEDs, user button.
- Record 1s audio and vibration data from microphone and accelerometer.
- Transmit recorded data to PC via st-link.
- Wrote a python script to read the data send from the board and
- Convert and save the transmitted data to .wav/.csv/.png

Items discussed /planned for the week

- Research to find out why use 2 sensors (question from modulator as sensor fusion are only useful when sensor 1 compliment sensor 2)
- Design of the simulation rig

Remarks (if any)

Student's signature: 

Supervisor's signature(s) : 

Date: 6/11/2025



2025-2026

Student Name(s) : Cheng Jing Siang
Project Title : Edge AI low-power water leak detection
Supervisor(s) : Dr Hermawan Nugroho
Date of meeting : 13/11/2025 **Length of meeting** : 2 hours

Progress made since the last meeting

- Modelled simulation rig and confirmed dimensions.
- Done training acoustic AI detection model.

Items discussed /planned for the week

- Email Mr. Vincent to clarify the use of 2 sensors
- Purchase the materials for the simulation rig.
- Possible approach to do localization to estimate leak location.
- Possible approach to estimate leak size

Remarks (if any)

Student's signature: 

Supervisor's signature(s): 

Date: 13/11/2025



2025-2026

Student Name(s) : Cheng Jing Siang
Project Title : Edge AI low-power water leak detection
Supervisor(s) : Dr Hermawan Nugroho
Date of meeting : 20/11/2025 **Length of meeting** : 1 hour

Progress made since the last meeting

- Receive conceptual system slide from Mr. Vincent
- Manage to detect leak/background on edge using the detection model output from NEAI.
- Created 2 decision logic (Temporal hysteresis and strong fraction) to account for different types of leaks and low detection accuracy.

Items discussed /planned for the week

- Improve the decision logic and recording intervals.
- Whether the own leaks simulated can be classified based on frequency.
- Arrange the date for Performa review.
- Research on the library and model used by NEAI.
- Start building simulation rig.

Remarks (if any)

Student's signature: 

Supervisor's signature(s) : 

Date: 20/11/2025



The University of
Nottingham

UNITED KINGDOM • CHINA • MALAYSIA

2025-2026

Student Name(s) : Cheng Jing Siang

Project Title : Edge AI low-power water leak detection

Supervisor(s) : Dr Hermawan Nugroho

Date of meeting : 22/1/2026

Length of meeting : 0.5 hours

Progress made since the last meeting

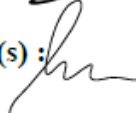
- Purchased remaining components needed:
 - 2 Lora board (RA-08H-Kit + antenna)
 - Battery (ER34615)
 - Super pulse capacitor (SPC1520)
 - Battery holder (D-Size)
 - 2 pins type C male to 2 core open end cable
 - Schottky diode
 - Breadboard
 - Bolts and Nuts
 - USB power meter
- Loaned 5V buck boost converter

Items discussed /planned for the week

- Start collecting datasets from simulation rig
- Late delivery of STM32L562E-DK board due to documentation (approximately 1 and a half months to receive)

Remarks (if any)

Student's signature: 

Supervisor's signature(s): 

Date: 22/1/2026



2025-2026

Student Name(s) : Cheng Jing Siang
Project Title : Edge AI low-power water leak detection
Supervisor(s) : Dr Hermawan Nugroho
Date of meeting : 29/1/2026 **Length of meeting** : 0.5 hours

Progress made since the last meeting

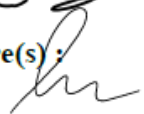
- Finalized full framework of detection logic.
- Collected total 1200 seconds of dataset
- Trained new NEAI model for both acoustic and vibration using the datasets collected
- Tested Lora board on simple transfer and receive.

Items discussed /planned for the week

- Continuing to collect more datasets
- Start connecting all components together (STM32 board, Lora, battery, etc.)
- Start designing 3D holder for the device and mounting mechanism on the pipe.

Remarks (if any)

Student's signature: 

Supervisor's signature(s): 

Date: 29/1/2026



2025-2026

Student Name(s) : Cheng Jing Siang

Project Title : Edge AI low-power water leak detection

Supervisor(s) : Dr Hermawan Nugroho

Date of meeting : 5/2/2026

Length of meeting : 0.5 hours

Progress made since the last meeting

- Designed and printed out the holder for the device.
- Connected all components together and mounted on the device.
- Lora working to handshake and send payload.

Items discussed /planned for the week

- Continuing to collect more datasets
- Create sleep mode for the device
- Start measuring power consumption

Remarks (if any)

Student's signature: 

Supervisor's signature(s) : 

Date: 5/2/2026



2025-2026

Student Name(s) : Cheng Jing Siang
Project Title : Edge AI low-power water leak detection
Supervisor(s) : Dr Hermawan Nugroho
Date of meeting : 26/2/2026 **Length of meeting** : 0.7 hours

Progress made since the last meeting

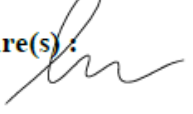
- Done sleep modes: Stop2, Stop3, Shutdown.
- Board current drawn measured about 110mA during sleep mode.

Items discussed /planned for the week

- Continuing to collect more datasets
- Measure current drawn by the MCU solely.
- Modify objective and stretch goal based on the board B-U585I-IOT02A since it implementing to the board STM32L562E-DK is unlikely due to late delivery.

Remarks (if any)

Student's signature: 

Supervisor's signature(s): 

Date: 26/2/2026



2025-2026

Student Name(s) : Cheng Jing Siang
Project Title : Edge AI low-power water leak detection
Supervisor(s) : Dr Hermawan Nugroho
Date of meeting : 5/3/2026 **Length of meeting** : 0.7 hours

Progress made since the last meeting

- Measured the current consumption of MCU for Stop 2, Idle and Off mode.
- Further expanded datasets.

Items discussed /planned for the week

- Continuing to collect more datasets.
- Complete all power consumption reading and calculation.
- Plot graphs for power consumption related comparison to be included in draft thesis.
- Start writing result and discussion, power consumption part for draft thesis.
- Start evaluating the accuracy of the first version of the NEAI detection model.

Remarks (if any)

Student's signature:

Supervisor's signature(s):

Date: 5/3/2026



2025-2026

Student Name(s) : Cheng Jing Siang
Project Title : Edge AI low-power water leak detection
Supervisor(s) : Dr Hermawan Nugroho
Date of meeting : 12/3/2026 **Length of meeting** : 1 hours

Progress made since the last meeting
- Plotted the power consumption related graphs (current drawn by each configuration, daily energy consumption, lifetime vs No. of detection per day, daily energy consumption vs No. of detection per day) in MATLAB. - Completed 50% of results and discussion for power consumption part. - Evaluated the accuracy first version of NEAI model. - Created second version of decision logic.

Items discussed /planned for the week
- Continue writing draft thesis. - Continuing to collect more datasets and start training second version of NEAI model.

Remarks (if any)

Student's signature: 

Supervisor's signature(s): 

Date: 12/3/2026



2025-2026

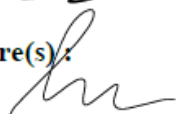
Student Name(s) : Cheng Jing Siang
Project Title : Edge AI low-power water leak detection
Supervisor(s) : Dr Hermawan Nugroho
Date of meeting : 2/4/2026 **Length of meeting** : hours

Progress made since the last meeting
- Trained few versions for the NEAI model. - Plotted the accuracy related graphs in MATLAB. - Completed result and discussion part for: - o AI model accuracy (NEAI) - o AI Model architecture (NEAI) - o Power consumption - o LoRa P2P transmission - o Device Holder - Completed methodology part for: - o Design and Hardware Integration - o Simulation rig - o Device holder - o Decision Logic - o Edge AI Model Development - Completed conclusion part for: - o Power Consumption and Low-Power Operation - o Holder Design - Submitted draft thesis

Items discussed /planned for the week
- Refine and clarify the decision logic section for clearer and better presentation.

Remarks (if any)

Student's signature: 

Supervisor's signature(s): 

Date: 2/4/2026



2025-2026

Student Name(s) : Cheng Jing Siang
Project Title : Edge AI low-power water leak detection
Supervisor(s) : Dr Hermawan Nugroho
Date of meeting : 6/4/2026 **Length of meeting** : 1 hours

Progress made since the last meeting

- Completed methodology part for:
 - o Deployment on Embedded Hardware
 - o Decision Logic (refine and clarification)

Items discussed /planned for the week

- Train and evaluate YAMNet detection model.
- Complete the results and discussion for YAMNet model accuracy comparison.
- Discussed on the decision logic structure and threshold/scoring/weight value and come out with questions to be answered.
- Refine and clarify the decision logic section to answer the question discussed on the value and method/structure/equation chosen. To be more defensible.
- Continue writing thesis.

Remarks (if any)

Student's signature: 

Supervisor's signature(s): 

Date: 6/4/2026



2025-2026

Student Name(s) : Cheng Jing Siang
Project Title : Edge AI low-power water leak detection
Supervisor(s) : Dr Hermawan Nugroho
Date of meeting : 13/4/2026 **Length of meeting** : 1 hours

Progress made since the last meeting

- Trained and evaluated the YAMNet detection model.
- Plotted the YAMNet accuracy related graphs in MATLAB.
- Completed methodology part for:
 - o Decision Logic (refine and clarification)
- Completed result and discussion part for:
 - o YAMNet Model Accuracy Comparison
- Completed conclusion part for:
 - o Detection System Performance
- Started writing literature review.

Items discussed /planned for the week

- Continue writing thesis.
- Check the root cause of accuracy drop of YAMNet-based model detection.

Remarks (if any)

Student's signature:

Supervisor's signature(s):

Date: 13/4/2026

Appendix P: Project review proforma - 1

Project Review Proforma – Review number: 1 Date: 4/12/2025

Summarised Planned State of Project: <i>Objective I: Define and set up a controlled experimental environment, including a leak simulation rig, to collect acoustic and vibration data under various leak conditions and controlled variables.</i> <i>Build the simulation rig that will be used in dataset collection and AI model performance evaluation.</i> <i>Code the development board to be able to sense and output the acoustic and vibration signal. These signals needed to be extracted and formatted into a format required for training AI detection model.</i>	Actual Progress Since Last Review <i>Built the simulation rig to simulate pipe leaking with different leak size. The controlled variables are the same PVC pipe used throughout the simulation, distance of the device from location of leak is kept constant and the source of water is kept to same to have the same flowrate.</i> <i>After testing with few samples' dataset recorded, late fusion is chosen over early fusion but will keep collecting datasets for both fusion methods.</i> <i>Coding in STM32Cube IDE to record 10s of acoustic and vibration data with a sample rate of 16kHz and 6.6kHz respectively with a single press on user button. Added a flexibility to choose how many 10s data to record per press of the user button.</i> <i>Wrote python code to extract the recorded data sent over UART and convert them into correct format required by NEAI.</i>
Next Steps and Supervisor Feedback <i>Next Step:</i> <i>Collect as many datasets as possible and start training in NEAI for both acoustic and vibration AI detection model. Used the output models to implement AI detection using late fusion method on the STM32 board. Finalise the decision logic to increase accuracy and logically implement on real life application.</i> <i>Supervisor Feedback:</i> <i>Try to collect data with noises and take picture on the data collection progress.</i>	

Appendix Q: Project review proforma – 2

Project Review Proforma – Review number: 2 Date: 4/12/2025

Summarised Planned State of Project: <i>Objective III: Ensure low power operation.</i> <i>Measure the current current drawn by the entire development board and by the MCU solely at different configuration (idle, detection, sleep, transmission mode).</i>	Actual Progress Since Last Review <i>Code written to let the prototype goes into sleep mode after a timeout to conserve energy.</i> <i>Measured and recorded the current drawn by the entire development board and by the MCU solely at different configuration (idle, detection, sleep, transmission mode).</i> <i>The entire development board draws high current ($\approx 113\text{mA}$) which result in a lifetime of a few days without replacing the battery (ER34615).</i> <i>The MCU solely draws very low current ($<0.2\text{mA}$) and affected by which sleep mode (Stop 2, Stop 3, Shutdown mode) implemented.</i>
Next Steps and Supervisor Feedback <i>Next step:</i> <i>Use the collected dataset to train the first version of the NEAI-based acoustic and vibration model and evaluate on the performance directly on the resource-limited embedded platform</i> <i>Supervisor feedback:</i> <i>Since the MCU, microphone, and accelerometer are all declared as "low-power" components, and the current readings of the MCU show very low current consumption, low power consumption can be demonstrated by using only the necessary components, without using the entire development board, as the current on the development board would likely be consumed by unused components. Therefore, future work such as constructing a custom hardware implementation can better achieving the objective.</i>	

Appendix R: Project review proforma - 3

Project Review Proforma – Review number: 3 Date: 13/4/2026

Summarised Planned State of Project: <i>Objective II: Develop an AI-based leak detection algorithm utilizing both acoustic and vibration signal analysis.</i>	Actual Progress Since Last Review <i>Developed a NEAI-based acoustic and vibration model has been created and evaluated on the performance directly on the resource-limited embedded platform. Decision logic built to support the lower performance of the detection model when implemented to the resource-limited embedded platform. Graphs plotted to visualise the comparison of the performance of NEAI-based acoustic and vibration model with the decision logic implemented. It was proved that the decision logic able to help with improving the accuracy.</i>
Next Steps and Supervisor Feedback <i>Next step: Expand the dataset in term of quantity and diversity and repeat the procedure to train and evaluate the NEAI-based acoustic and vibration model. Using the expanded dataset to develop a YAMNet-based acoustic and vibration model and evaluate on the performance directly on the resource-limited embedded platform to compare with the NEAI-based acoustic and vibration model.</i> <i>Supervisor feedback: Develop the YAMNet-based acoustic and vibration model to compare the accuracy with the NEAI-based acoustic and vibration model. Make a clearer explanation on the decision logic created and answer to these questions: Why does the decision logic help improving the accuracy? How are the value for threshold, weight, score and bonus chosen? Why are those specifically defined duration of signal passing into the periodical detection chosen and why does 512 samples chosen as the training and inference input size?</i>	

Appendix S: Project review proforma - 4

Project Review Proforma – Review number: 4 Date: 13/4/2026

Summarised Planned State of Project: <i>Stretch Goal V: AI Model Performance Comparison: Benchmark the device's deployed AI model against alternative models (e.g., YAMNet) to evaluate and compare detection performance.</i>	Actual Progress Since Last Review <i>Developed a YAMNet-based acoustic and vibration model has been created and evaluated on the performance directly on the resource-limited embedded platform. Graphs plotted to visualise the comparison of the performance between the NEAI-based acoustic and vibration model with the decision logic implemented and the YAMNet-based acoustic and vibration model. Results shows that the chosen NEAI-based acoustic and vibration model outperformed the YAMNet-based acoustic and vibration model.</i>
Next Steps and Supervisor Feedback <i>Next step: Try to find out why the accuracy of YAMNet- based model was very low (>40%).</i> <i>Supervisor feedback: Check if the input format in the device is the same with the input format used to train the AI or not, especially the MFCC conversion for acoustic channel. Check if the length of input (currently 0.1s) is enough for detecting leak or not.</i>	

Appendix T: Project review proforma - 5

Project Review Proforma – Review number: 5 Date: 13/4/2026

Summarised Planned State of Project: <i>Objective V: Evaluate the complete system implementation by validating the data collection and analysis intervals and verifying that power consumption and leak detection performance meet the defined project goal.</i>	Actual Progress Since Last Review <i>Completed the integration of the overall prototype system, including acoustic sensing, vibration sensing, AI detection, decision logic, low-power operation, and LoRa P2P result transmission. Finalized the evaluation of the developed NEAI-based detection model and the comparison with the YAMNet-based model on the embedded platform. Completed the analysis and presentation of the final accuracy, power consumption, and LoRa communication results. Progressed the final thesis writing, including the results, discussion, conclusion, and reflection sections.</i>
Next Steps and Supervisor Feedback <i>Next step: Finalize the thesis and supporting materials for submission. Prepare for the final presentation, including project demonstration and explanation of key findings.</i> <i>Supervisor feedback: Continue refining the thesis writing and ensure the discussion clearly explains the final system performance, limitations, and contribution of the project.</i>	

Section A: Details Information**Important Note:**

Users are required to re-submit the HIRARC for approval and review every three (3) years OR whenever there are changes in process or activities or methodology.

Reference No	UNMC/HSE/HIRARC/2025/8574
Staff / Student	Student
Assessor Name	Cheng Jing Siang
Assessor Email	efyjc23@nottingham.edu.my
Assessor Phone Number	0125488250
Student / Staff ID No	20409588
Activity Category	Undergraduate experiment (i.e Bachelor, MEng, MSc)
Work / Event	FOSE - Engineering
Title	Edge AI low-power water leak detection
Block	D - FOSE
Area name	DA08, DA19, DLG
Activity Date	2025-10-20

Section B: Details of other person involved

Name	Staff or Student ID No.	Email	HP Number
------	-------------------------	-------	-----------

Section C: Assessment Information

Assessment	Work Procedure (i.e standard operation procedure [SOP], experiment methodology, proposal paper)
Assessment Detail	The project will be developed with the STM32L562E-DK discovery board as the base platform. Most of the work will be coding on software such as STM32CubeIDE and NanoEdge AI Studio. Programming of the board will be conducted at home or at DA08. Prototyping involves 3D modelling and 3D printing using a 3D printer at DLG. Soldering might be involved if embedded nuts are used with the 3D printed part at DA19. This project collaborates with George Kent (Malaysia) Berhad, the final prototype testing might be conducted at lab/home or at the test rig built by the company side.
Attachment (pdf,word)	HIRARC_Assessment details.pdf

Section D: HIRARC

Activity	Type of Hazards	Description	Cause and Effect	Existing Risk Control	Description	Likelihood	Severity	Risk	Control Measure	Description
Programming/Coding	Ergonomic	Long time sitting in the same posture during coding in front of computer.	Back pain, RSI, eye strain	Administrative control - Supervision	Take frequent break to prevent long use of computer. Practice good posture to sit, good lighting and do not sit too close to the computer	2	2	4		

Activity	Type of Hazards	Description	Cause and Effect	Existing Risk Control	Description	Likelihood	Severity	Risk	Control Measure	Description
Setup/Connecting microcontroller to computer	Electrical	exposed to a power source, electrical circuit, or unintentionally contact with high voltage, an excessively connected load, or damaged insulated wiring	Circuit short circuit/electrical shock, overdose, electrocution, mild damage, unconsciousness, or, in the case of a major case, collapse.	Administrative control - SOP	Don't touch the circuit with bare hand, wear a glove. Ensure correct connection before power on.	1	2	2		
Soldering embedded nuts on 3D printed object	Temperature	Expose to high temperature soldering iron	Injury and may cause skin burn	Administrative control - SOP	Place the soldering correctly at the correct location. Follow SOP for soldering and solder at ventilated area.	2	2	4		
Test prototypes at field site (home/lab/outdoor/company site)	Natural	Expose to uncertain weather change (hot sun, rain, storm, windy, cold, haze, etc.) If testing site is at outdoor	May get sick, fever, headache, cold, cough, faint, etc.	Administrative control - SOP	Observe the safety regulations whilst on the job site. Bring a raincoat, hat, umbrella, and water to drink. Obtain the most recent weather forecast information. If the weather is bad, call off work.	2	1	2		

Section E: Declaration on Security Issues

I/we hereby declare that we did not bring hazardous or dangerous item(s) into the UNM and followed Malaysian legal requirements.

I/we hereby agreed to be responsible for indemnifying for any effect of our event caused by us, our participant, or student/staff resulting from the above exposure or accident. In the event of an incident, a professional investigation will be conducted to determine who is at fault. The organiser will only be held responsible for what the invited guest says at the venue and will not be held responsible for any statement made outside the venue. The UNM reserves the right to cancel or take necessary action in breach of this declaration.

Name Cheng Jing Siang
IC Number / Passport Number 030623010799
Declaration Yes, I agree with the terms & conditions

Section F: Sustainable Events Guidelines Acknowledgement Statement

I hereby acknowledge that I have read and fully understood the contents of the UNM Sustainable Events Guidelines (GL-ESC/Rev01, 18 September 2025).

I understand the principles, objectives, and requirements outlined in this document, including the commitment to minimizing waste, promoting responsible resource use, and supporting sustainable practices throughout all stages of event planning and execution at the University of Nottingham Malaysia (UNM).

I recognize the importance of integrating sustainability into every aspect of event management to align with the University's Strategic Plan 2022–2026 and the United Nations Sustainable Development Goals (SDGs).

Accordingly, I pledge to adhere to and actively support these guidelines in all events under my responsibility or involvement. I will also encourage others within my team, department, or organization to do the same, ensuring that every event contributes positively to UNM's sustainability goals and fosters a culture of environmental responsibility within our campus community.

Sustainable Events Guidelines

Name Cheng Jing Siang
Department/ Club/ Organisation Department of Electrical and Electronic
Position/Role Mechatronic Year 4 Student
Declaration Yes, I agree with the terms & conditions

Section G: Declaration on Health and Safety Policy Statement

Approver Name HERMAWAN NUGROHO

I understand and acknowledge the University of Nottingham Malaysia Campus Health & Safety Policy Statement.

The responsibility for health and safety is both an individual and a shared responsibility of all users of the UNMC community.

All users of the UNMC community are (1) expected to work, and encourage others to work, in a safe and healthy manner and discourage others from working in an unsafe way, (2) expected to cooperate to support and promote health and safety in the work place and report any unsafe conditions that come to their attention at work.

It is the duty of all people on campus to conform to the Health and Safety policy, safety training, instructions given, codes of safe practice and to accept and carry out their responsibilities with regard to health and safety.

For any work activity in group, this is declare that me, myself and all group members that listed in Section B are understood all contents of hazard and risk that identified and assessed in this H|RARC e-form and will act according to counter measures/controls that stated including the use of PPE when carry out our work,

At the same time, we have also read and understood the contents of SDS (safety data sheet) before using any chemical/substance/gas.

We will be liable and responsible for our own safety by adhere to any rules and guidelines applied in our work

[UNMC Health and Safety Policy Statement](#)

Declaration : Yes, I agreed with the terms & conditions

Draft or Submit Form? Submit

Log

Name	Action	Date
Jing Cheng	New Submission	2025-10-19 21:05:25,0
HERMAWAN NUGROHO	Approved by Approver	2025-10-21 13:38:37,0
YAU YEE KOON	Reviewed by Reviewer	2025-10-22 09:48:11,0

Approval

Approved by HERMAWAN NUGROHO

Status Approved

Date 21/10/2025 1:38:36 PM

Remarks

Reviewer

Status Reviewed

Date 22/10/2025 9:48:10 AM

Remarks Make sure electrical safety when testing the rig/prototype, avoid contact the circuit with wet hands or wet materials.

[Back](#)